

Informationsverarbeitung

Moderne CPUs und heterogene Systeme

Dr.-Ing. Tanja Harbaum

Institut für Technik der Informationsverarbeitung (ITIV)



Übersicht – Vorlesung Nr. 05

- Ripes Case Study
- Ausblick:
 - Moderne CPUs und heterogene Systeme

Workshop Informationsverarbeitung

- Einführung in Präsenz am 13.05.
 - Bringen Sie bitte Ihren Laptop/Tablet mit
- Aufbau des Workshops
 - Termine
 - Inhalte
 - Support
 - Bewertung
- Tutorials
 - Tutorial 1 – Wo finde ich die Aufgaben?
 - Tutorial 2 – Wie ist ein Notebook aufgebaut?
 - Tutorial 3 – Wie gebe ich eine Aufgabe ab?

FORWARDING
LOGIC

CONTROL
LOGIC

CSR

KEYBOARD
CONTROLLER

AND

COLOR
DISPLAY
CONTROLLER

CLOCK

CPU Entwicklung in
Factorio:
Von D-Flip-Flop bis
zum Betriebssystem
von Philipp @ 39c3

Q W E R T Z U I O P <- 7 8 9
A S D F G H J K L 4 5 6
V X C B N M , . - / 1 2 3
SPACE 0



ROM 0 ROM 1

CPU Specs / Features

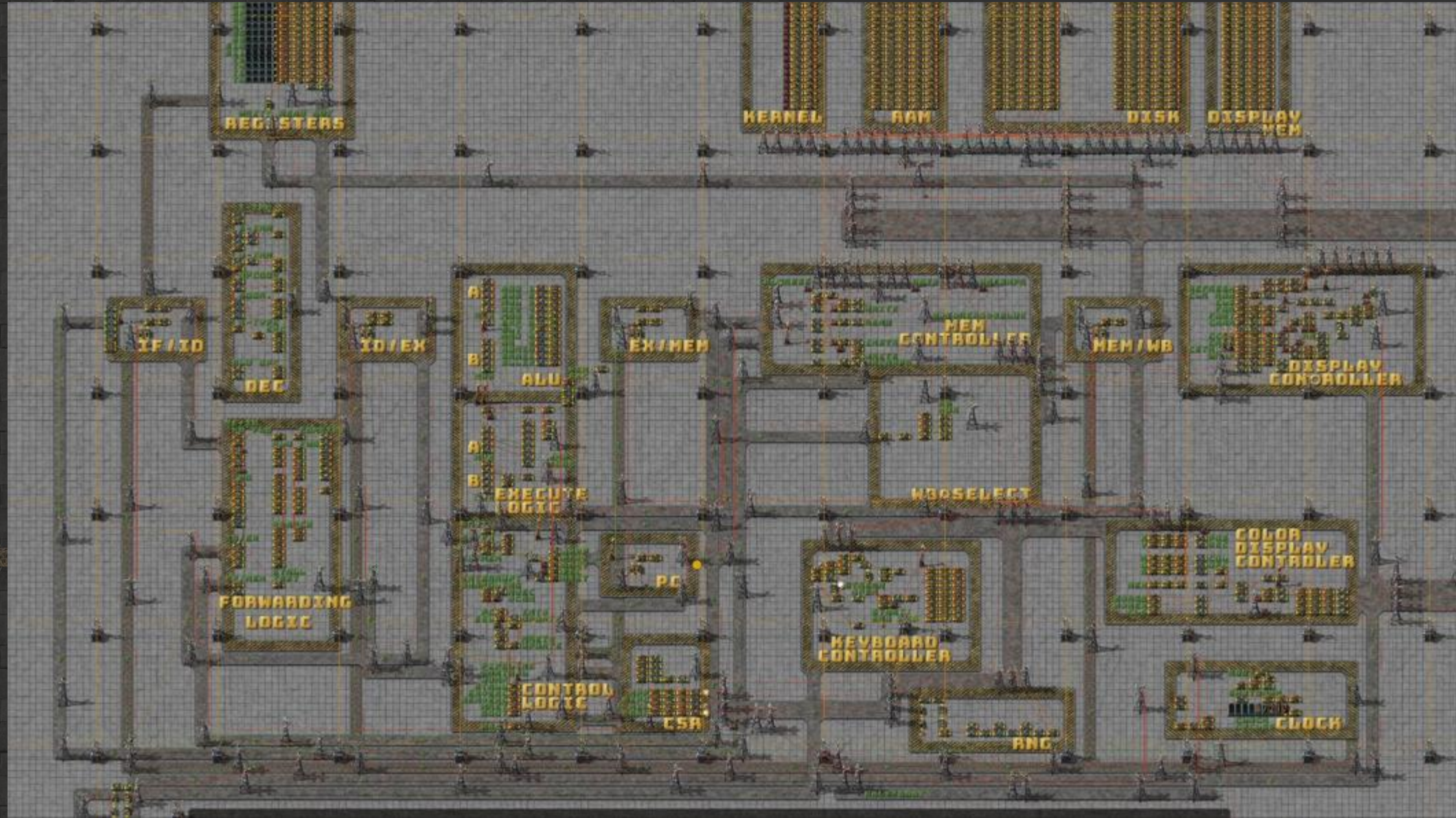
Hardware

- 5 Stage Pipelined Architektur
- 32 General Purpose Register
- 64 kB RAM
- 128 kB Hard Drive
- Interrupt Handler
- ~700 Hz Taktfrequenz
- Peripherie
 - ASCII Character Display
 - Color Display
 - Tastatur
 - RNG

Software

- Python Tools:
 - Assembler
 - Simulator (500 kHz Taktfrequenz)
- Assembly Programme
 - FactOS:
Kommandozeilenumgebung
 - Chest32: FAT-Style Dateisystem
 - Minigames

Pipeline



Join!

39C3 - CPU Entwicklung in
Factorio: Vom D-Flip-Flop bis
zum eigenen Betriebssystem



Github Repository:

github.com/PhilippDahlinger/byte_factory



Evaluation der Vorlesung

- Offen bis zum 12.05.
- Bitte auch teilnehmen, wenn Sie nur die Videos gucken
- Gerne Kommentare bei 5.1
 - Freitext nutzen
- Workshop und Übung werden separat evaluiert

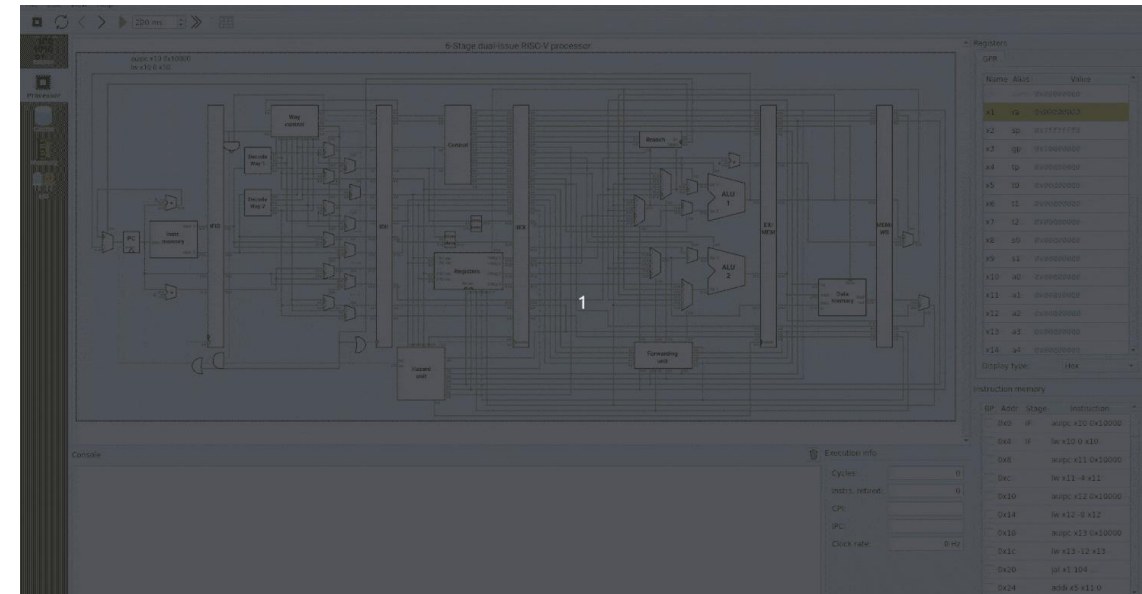


**DIE RÄUME
SIND BEKANNT,
SAG UNS WIE
DIE VORLESUNG
WAR!**



Ripes Case Study

- Wir implementieren die Funktion $f(x) = x!$
- Angefangen bei C Code werden wir alle Schritte im Detail durchgehen
- Um selbst zu experimentieren und das Verständnis zu vertiefen, empfehlen wir <https://github.com/mortbopet/ripes>



n!

- Es gilt: $n! = \prod_{k=1}^n k = 1 \cdot 2 \cdot 3 \cdots n$
- Algorithmisch müssen wir also nur die Reihe durchiterieren und jeweils das aktuelle Element mit dem bisherigen Produkt multiplizieren

Implementierung in C - Programmlogik

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

Gesamtprodukt fängt bei 1 an

iteriere durch alle Zahlen von 1 bis n

Multipliziere aktuellen Wert mit Gesamtprodukt

Implementierung in C – Äußeres Programm

```
int main()  
{  
    int n = 5; ← Beispielwert n=5  
    int ans = factorial(n); ← Jetzt können wir die Funktion aufrufen  
}
```

Implementierung in C – Vollständiges Programm

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Hinweis: Compiler & Assembly

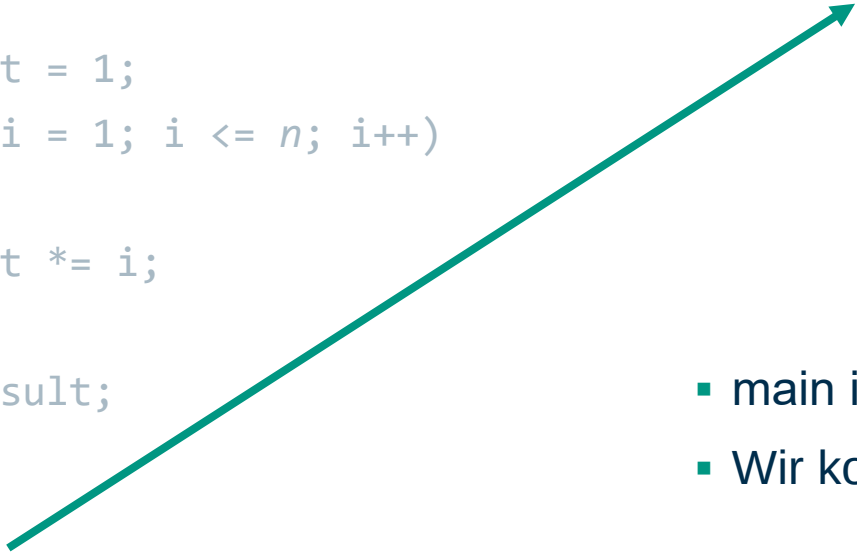
- Wir wollen von C nach RISC-V Assembly kompilieren
- Es gibt viele Compiler. Empfehlung zum ausprobieren: <https://godbolt.org/>
- **Aber:** Compiler optimieren den Code, wodurch er schwer lesbar wird
 - Zu Erklärungszwecken wird in den nächsten Folien manuell geschriebenes Assembly benutzt

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

`.globl main`
`main:`



- main ist ein „label“
- Wir kommen drauf zurück

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

```
int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Wir wollen 5! bestimmen.
Speichere 5 in Register a0

```
.globl main
```

```
main:
```

```
    addi    a0, x0, 5
    jal     ra, factorial
```

```
    addi    a7, x0, 10
    ecall
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

```
.globl main
```

```
main:
```

```
    addi    a0, x0, 5
    jal     ra, factorial
```

```
    addi    a7, x0, 10
    ecall
```

Danach Sprung zu der
Fakultätsfunktion



Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

```
int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Ergebnis wird auf 1
initialisiert in Register a1

Aktuelle Iteration i wird
auf 1 initialisiert in
Register a2

```
.globl main
```

```
main:
```

```
addi    a0, x0, 5
al       ra, factorial
```

```
addi    a7, x0, 10
ecall
```

```
factorial:
```

```
addi    a1, x0, 1
addi    a2, x0, 1
```

```
loop:
```

```
blt     a0, a2, done
mul     a1, a1, a2
addi    a2, a2, 1
jal     x0, loop
```

```
done:
```

```
mv      a0, a1
ret
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

```
int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

blt = Branch if Less Than

Ist a0 (also 5) kleiner als
a2 (Iterationsvariable i)?

-> Falls ja: Schleife ist fertig

```
.globl main
```

```
main:
```

```
    addi    a0, x0, 5
    jal     ra, factorial
```

```
    addi    a7, x0, 10
    ecall
```

```
factorial:
```

```
    addi    a1, x0, 1
    addi    a2, x0, 1
```

```
Loop:
```

```
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
```

```
done:
```

```
    mv     a0, a1
    ret
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Multipliziere aktuelle
Iteration i mit dem
Zwischenergebnis

```
.globl main

main:
    addi    a0, x0, 5
    jal     ra, factorial

    addi    a7, x0, 10
    ecall

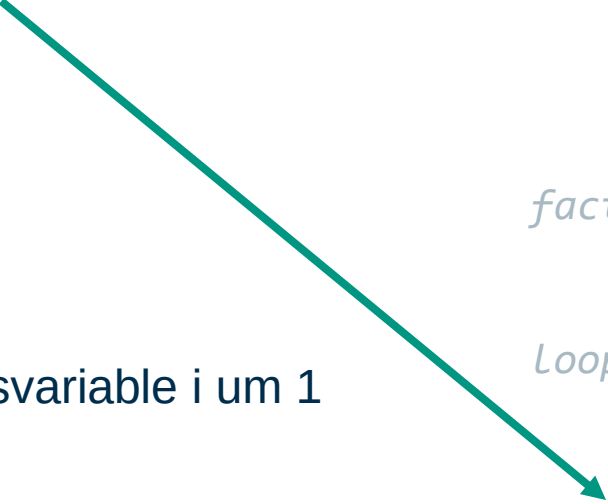
factorial:
    addi    a1, x0, 1
    addi    a2, x0, 1
loop:
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
done:
    mv      a0, a1
    ret
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

```
int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Erhöhe
Iterationsvariable i um 1



```
.globl main
```

```
main:
```

```
    addi    a0, x0, 5
    jal     ra, factorial
```

```
    addi    a7, x0, 10
    ecall
```

```
factorial:
```

```
    addi    a1, x0, 1
    addi    a2, x0, 1
```

```
loop:
```

```
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
```

```
done:
```

```
    mv     a0, a1
    ret
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```



```
.globl main

main:
    addi    a0, x0, 5
    jal     ra, factorial

    addi    a7, x0, 10
    ecall

factorial:
    addi    a1, x0, 1
    addi    a2, x0, 1
loop:
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
done:
    mv      a0, a1
    ret
```

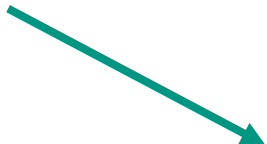


Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}
```

```
int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Speichere
Zwischenergebnis* (a1)
in a0



```
.globl main
```

```
main:
```

```
    addi    a0, x0, 5
    jal     ra, factorial
```

```
    addi    a7, x0, 10
    ecall
```

```
factorial:
```

```
    addi    a1, x0, 1
    addi    a2, x0, 1
```

```
loop:
```

```
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
```

```
done:
```

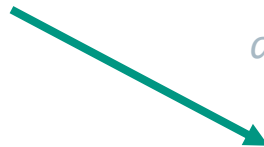
```
    mv     a0, a1
    ret
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```

Kehre zurück* zu der
return address



```
.globl main

main:
    addi    a0, x0, 5
    jal     ra, factorial

    addi    a7, x0, 10
    ecall

factorial:
    addi    a1, x0, 1
    addi    a2, x0, 1
loop:
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
done:
    mv      a0, a1
    ret
```

Pseudoinstruktionen

- *mv* und *ret* sind Pseudoinstruktionen
- *mv* rd, rs1 kopiert den Wert von rs1 nach rd
 - Der Assembler übersetzt das in *addi* rd, rs1, 0
- *ret* springt zurück an die return address, die von jal gesetzt wurde
 - Der Assembler übersetzt das in *jalr* x0, 0(ra)

Pseudoinstruktionen

- `mv` und `ret` sind Pseudoinstruktionen
- `mv rd, rs1` kopiert den Wert von `rs1` nach `rd`
 - Der Assembler übersetzt das in `addi rd, rs1, 0`
- `ret` springt zurück an die return address, die von `jal` gesetzt wurde
 - Der Assembler übersetzt das in `jalr x0, 0(ra)`
- Pseudoinstruktionen machen den Code lesbarer
- Es gibt keine Performance Nachteile, da der Assembler sie vor der Ausführung in normale RISC-V Instruktionen übersetzt

Labels in Assembly

- Sprungbefehle erwarten echte Offsets und Adressen

```
main:
    ...
    jal    x1, factorial
    ...

factorial:
    ...

loop:
    blt    a0, a2, done
    ...
    jal    x0, loop

done:
    ...
```

Labels in Assembly

- Sprungbefehle erwarten echte Offsets und Adressen
- Der Assembler muss Labels in Adressen übersetzen

```
main: 0
    ...
    jal    x1, factorial
    ...

factorial: 16
    ...

loop: 24
    blt    a0, a2, done
    ...
    jal    x0, loop

done: 40
    ...
```

Labels in Assembly

- Sprungbefehle erwarten echte Offsets und Adressen
- Der Assembler muss Labels in Adressen übersetzen
- Die Sprungbefehle müssen ebenfalls vom Assembler angepasst werden
(Adressen sind relativ zum Program Counter)

```
main: 0
    ...
    jal      x1, 12
    ...

factorial: 16
    ...

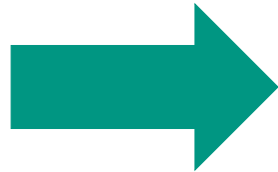
loop: 24
    blt      a0, a2, 16
    ...
    jal      x0, -12

done: 40
    ...
```

Implementierung in Assembly

```
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }
    return result;
}

int main()
{
    int n = 5;
    int ans = factorial(n);
}
```



```
.globl main

main:
    addi    a0, x0, 5
    jal     ra, factorial

    addi    a7, x0, 10
    ecall

factorial:
    addi    a1, x0, 1
    addi    a2, x0, 1
loop:
    blt     a0, a2, done
    mul     a1, a1, a2
    addi    a2, a2, 1
    jal     x0, loop
done:
    mv      a0, a1
    ret
```

Ausführung bis zum ersten Sprung

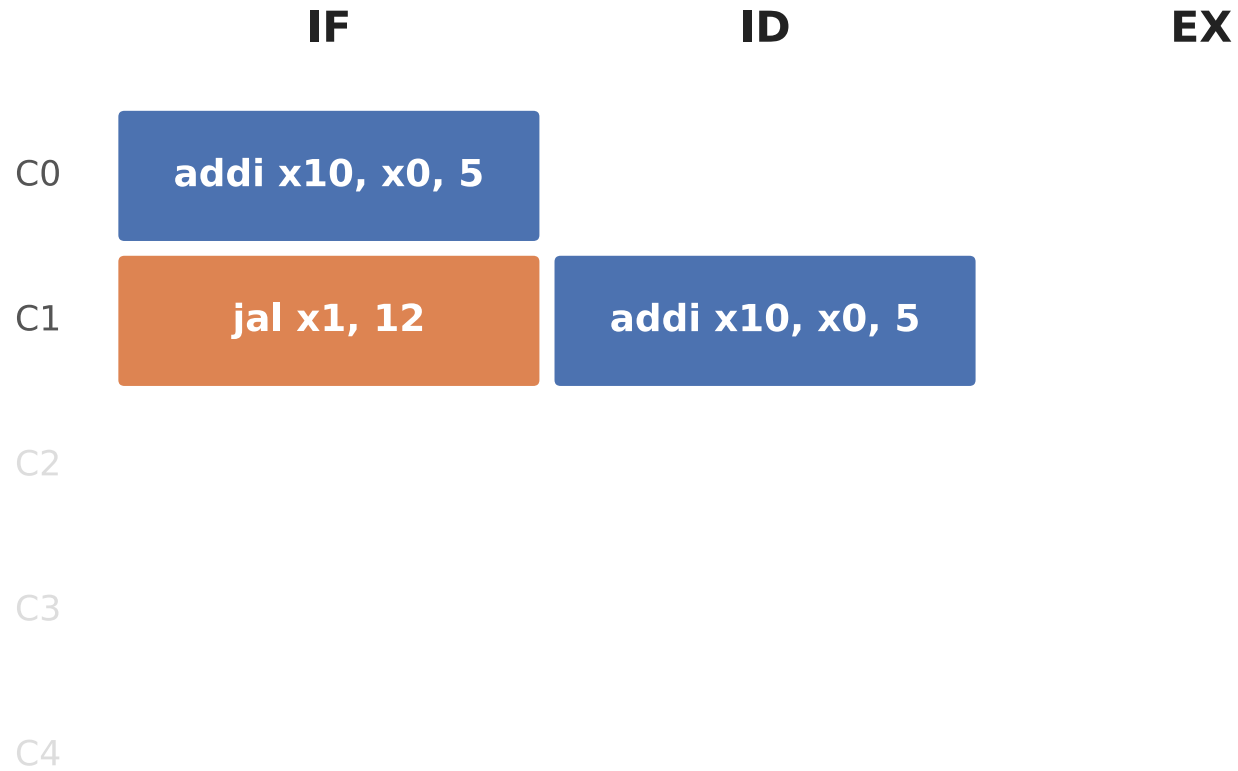


```
main:
    addi    a0, x0, 5
    jal     ra, factorial

    addi    a7, x0, 10
    ecall

factorial:
    addi    a1, x0, 1
```

Ausführung bis zum ersten Sprung

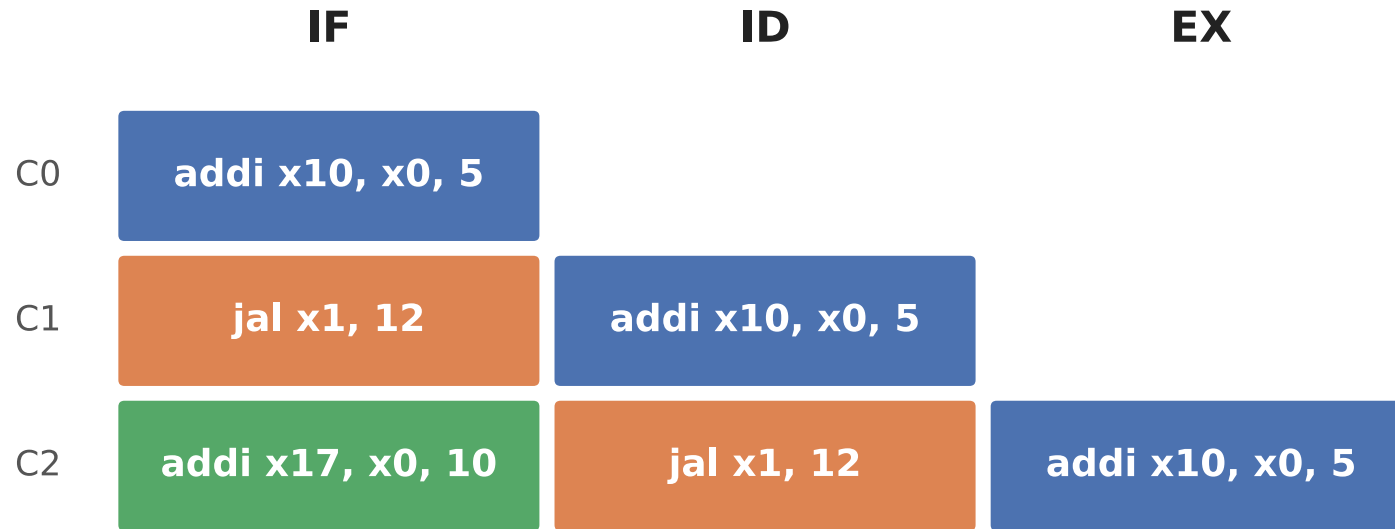


```
main:
    addi    a0, x0, 5
    jal     ra, factorial

    addi    a7, x0, 10
    ecall

factorial:
    addi    a1, x0, 1
```

Ausführung bis zum ersten Sprung



```
main:
    addi    a0, x0, 5
    jal     ra, factorial
```

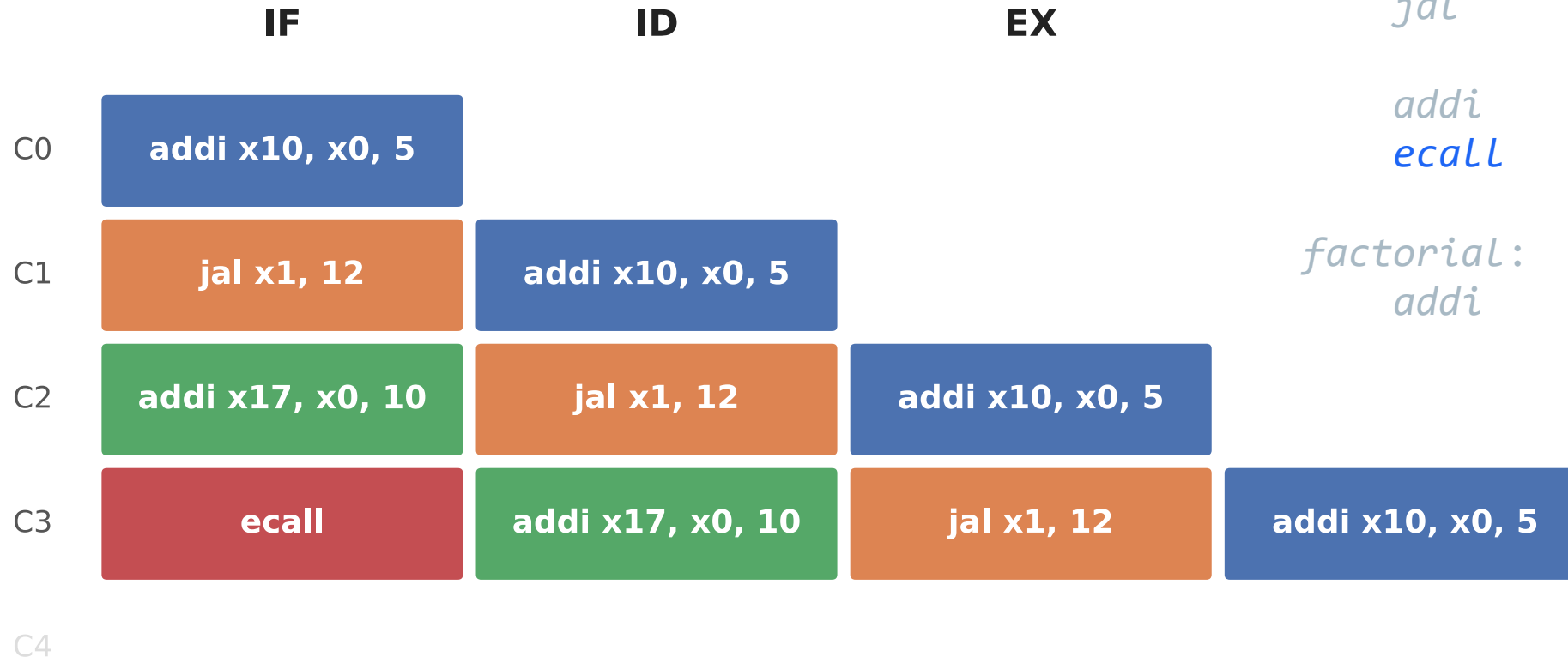
```
    addi    a7, x0, 10
    ecall
```

```
factorial:
    addi    a1, x0, 1
```

Wir hätten bereits zu factorial springen sollen

-> die folgenden Instruktionen sollten eigentlich nicht ausgeführt werden

Ausführung bis zum ersten Sprung

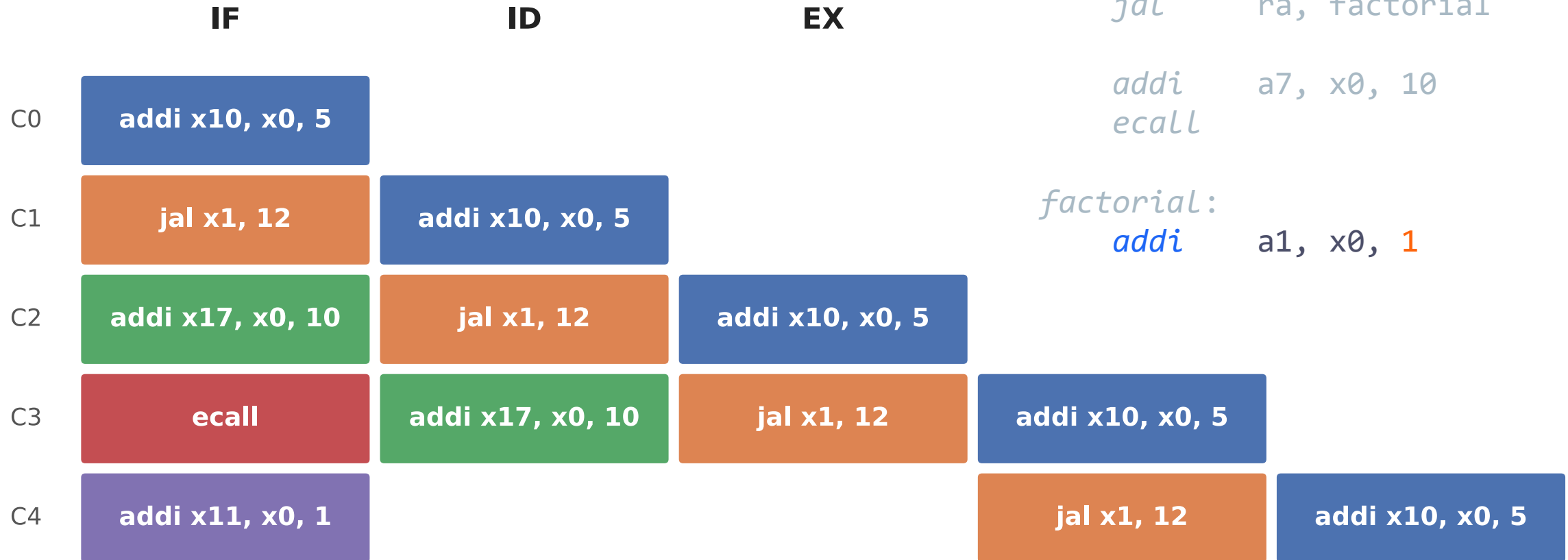


```
main:
    addi    a0, x0, 5
    jal     ra, factorial

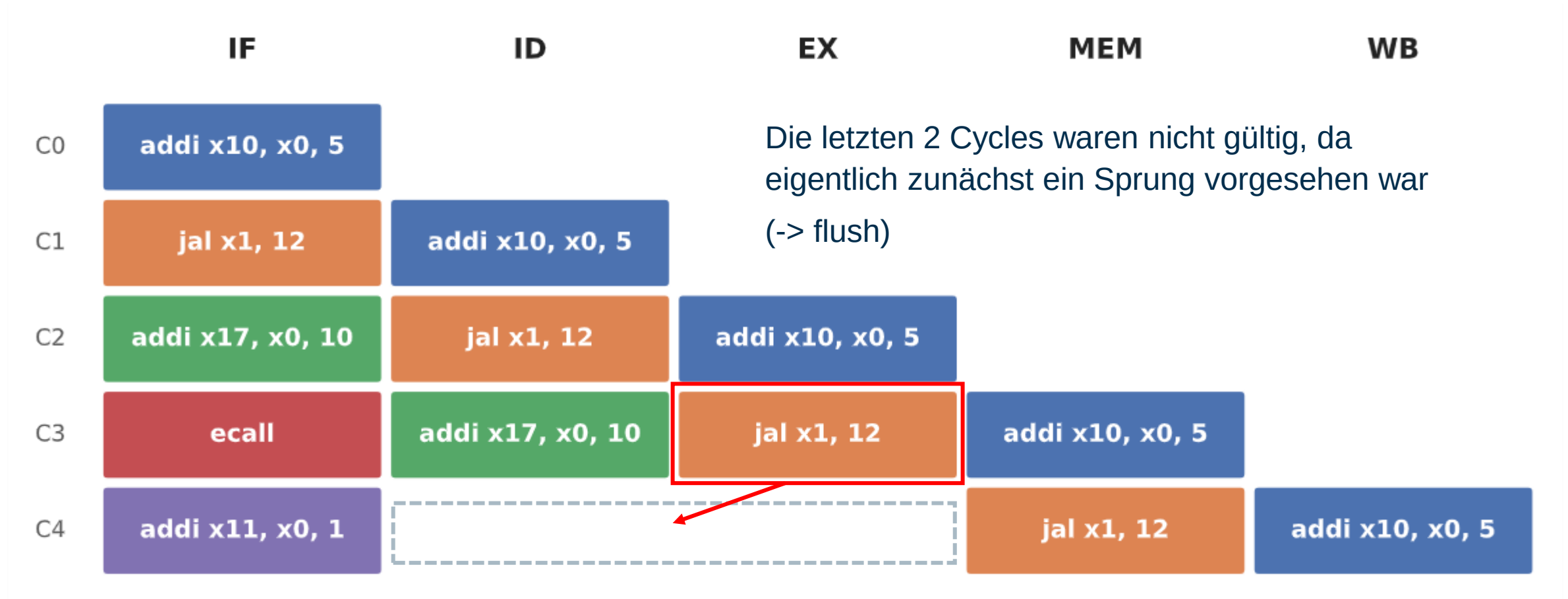
    addi    a7, x0, 10
    ecall

factorial:
    addi    a1, x0, 1
```

Ausführung bis zum ersten Sprung



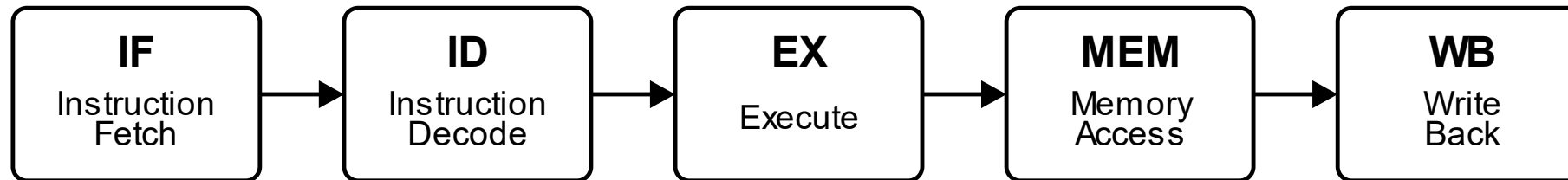
Ausführung bis zum ersten Sprung



Lebenszyklus einer Instruktion

- Jede Instruktion muss alle 5 Stages durchlaufen
- Wir schauen uns den gesamten Ablauf für die erste Instruktion an

addi a0, x0, 5



Lebenszyklus einer Instruktion

- Jede Instruktion muss alle 5 Stages durchlaufen
- Wir schauen uns den gesamten Ablauf für die erste Instruktion an

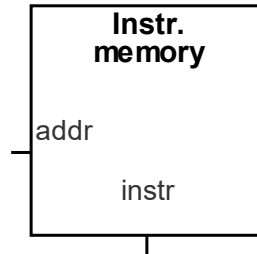
addi a0, x0, 5

addi a0, x0, 5



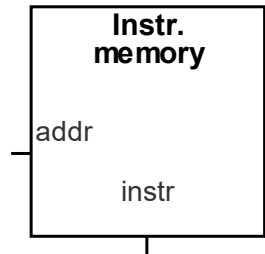
IF Stage im Detail

- Ziel der ersten Stage: Instruktionen aus Instruction Memory laden



- Vereinfachung in Ripes: „Instruction Memory“ hält alle Instructions
- In der Realität kommen die Instruktionen aus Caches/RAM etc.

IF Stage im Detail



```
0 <main>:
0: 00500513 addi x10 x0 5
4: 00c000ef jal x1 12 <factorial>
8: 00a00893 addi x17 x0 10
12: 00000073 ecall

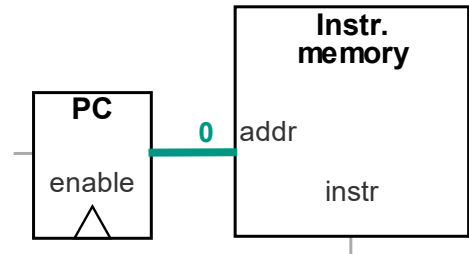
16 <factorial>:
16: 00100593 addi x11 x0 1
20: 00100613 addi x12 x0 1

24 <loop>:
24: 00c54863 blt x10 x12 16 <done>
28: 02c585b3 mul x11 x11 x12
32: 00160613 addi x12 x12 1
36: ff5ff06f jal x0 -12 <loop>

40 <done>:
40: 00058513 addi x10 x11 0
44: 00008067 jalr x0 x1 0
```

IF Stage im Detail

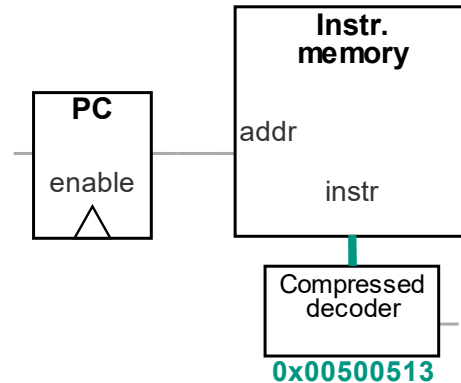
- PC Register hat die Adresse der nächsten Instruktion



```
0 <main>:  
0: 00500513 addi x10 x0 5  
4: 00c000ef jal x1 12 <factorial>  
8: 00a00893 addi x17 x0 10  
12: 00000073 ecall  
  
16 <factorial>:  
16: 00100593 addi x11 x0 1  
20: 00100613 addi x12 x0 1  
  
24 <loop>:  
24: 00c54863 blt x10 x12 16 <done>  
28: 02c585b3 mul x11 x11 x12  
32: 00160613 addi x12 x12 1  
36: ff5ff06f jal x0 -12 <loop>  
  
40 <done>:  
40: 00058513 addi x10 x11 0  
44: 00008067 jalr x0 x1 0
```

IF Stage im Detail

- Instruktion wird aus dem Speicher geladen



- Falls sie komprimiert war, wird sie dekomprimiert (nicht zutreffend)

```
0 <main>:
```

```
0: 00500513 addi x10 x0 5
```

```
4: 00c000ef jal x1 12 <factorial>
```

```
8: 00a00893 addi x17 x0 10
```

```
12: 00000073 ecall
```

```
16 <factorial>:
```

```
16: 00100593 addi x11 x0 1
```

```
20: 00100613 addi x12 x0 1
```

```
24 <loop>:
```

```
24: 00c54863 blt x10 x12 16 <done>
```

```
28: 02c585b3 mul x11 x11 x12
```

```
32: 00160613 addi x12 x12 1
```

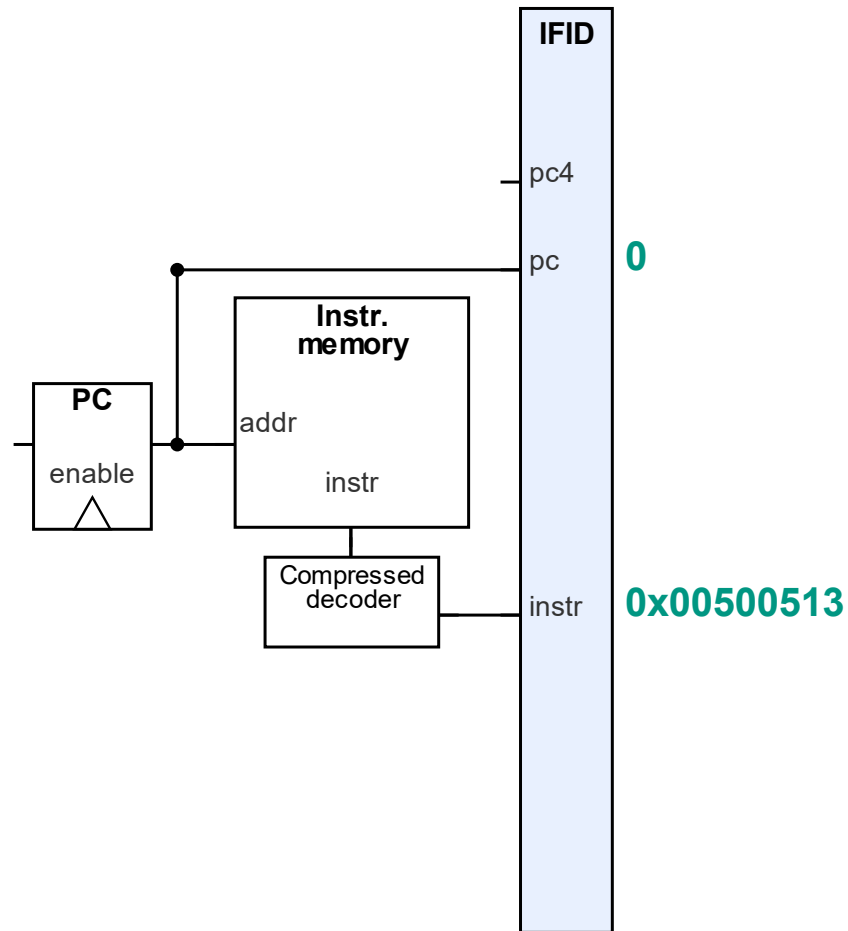
```
36: ff5ff06f jal x0 -12 <loop>
```

```
40 <done>:
```

```
40: 00058513 addi x10 x11 0
```

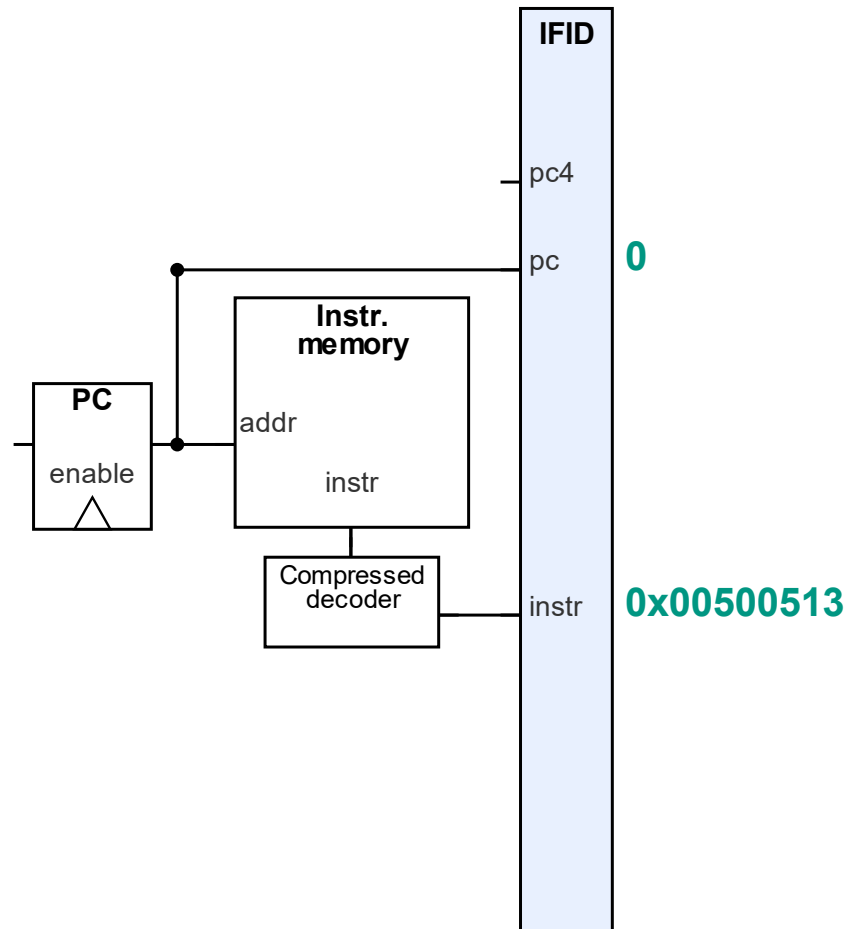
```
44: 00008067 jalr x0 x1 0
```

IF Stage im Detail



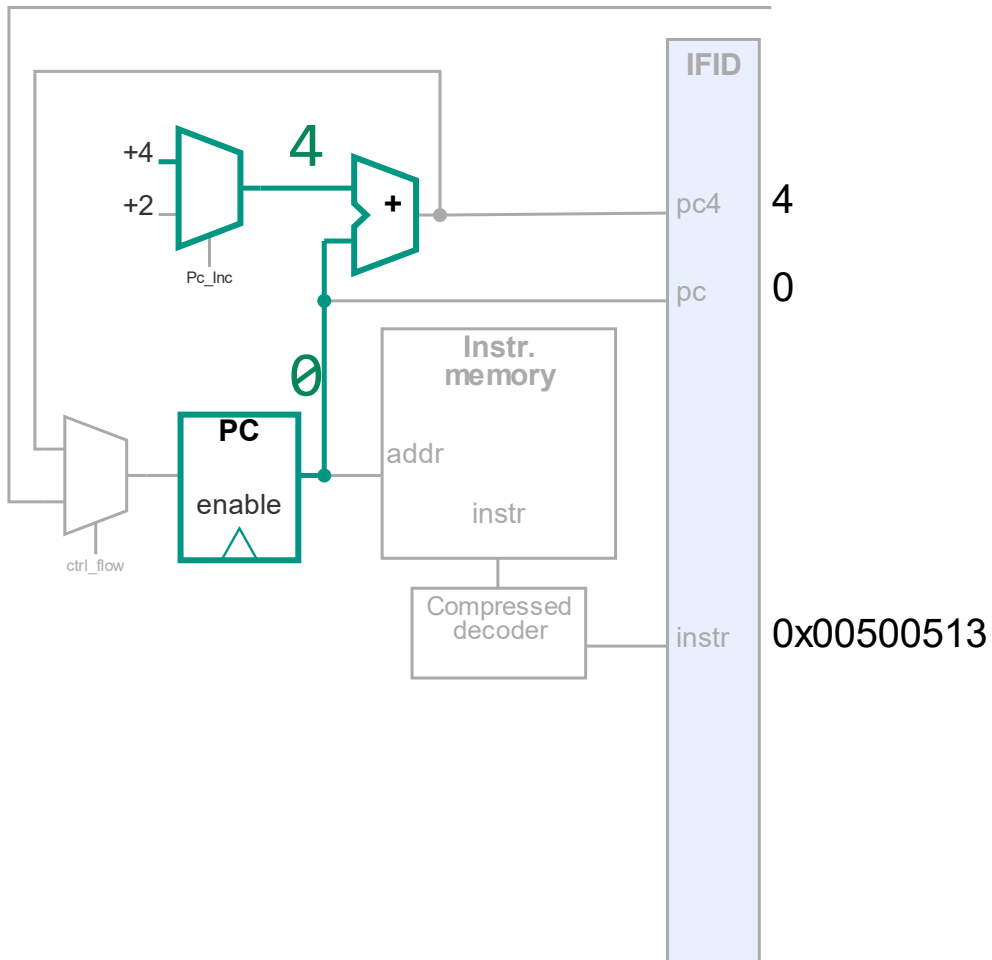
- Instruktion wird über ein Register für die nächste Stage zugänglich
- ProgramCounter ebenfalls

IF Stage im Detail



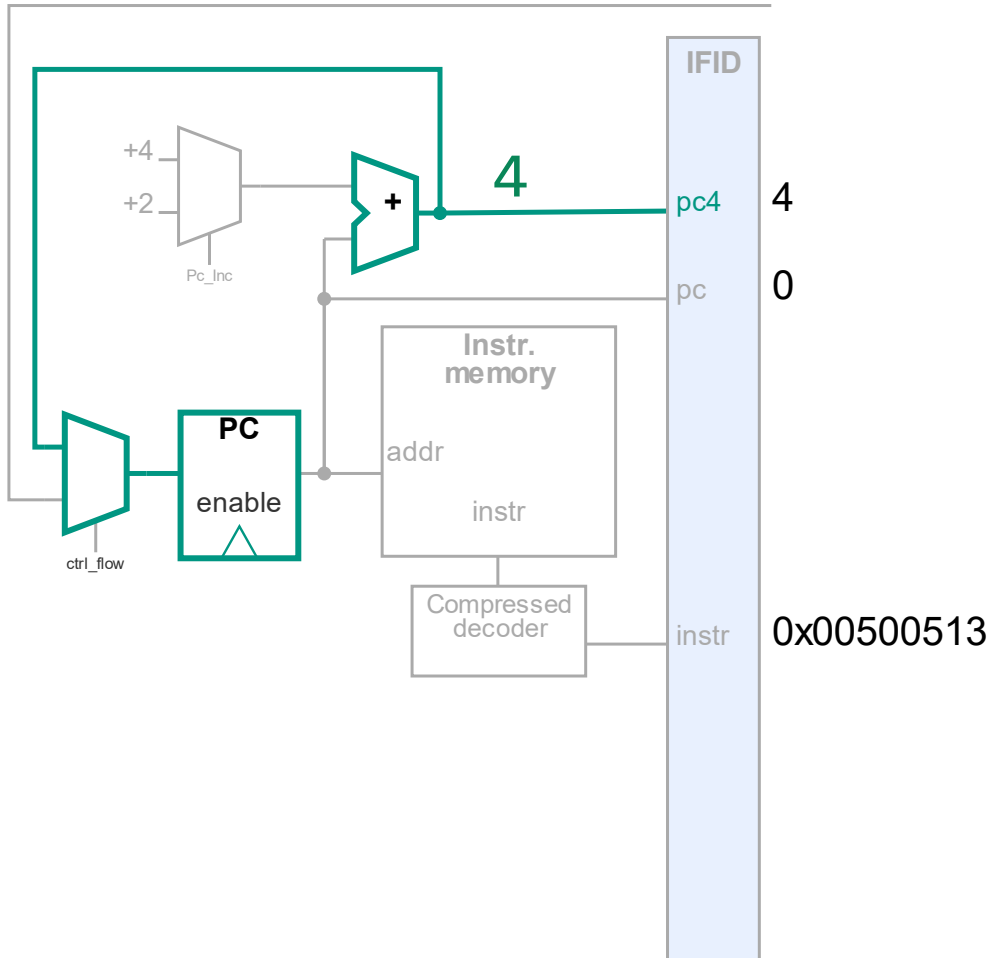
- Instruktion wird über ein Register für die nächste Stage zugänglich
- ProgramCounter ebenfalls
- Program Counter muss inkrementiert werden
- Dazu brauchen wir ein wenig zusätzliche Logik...

IF Stage im Detail



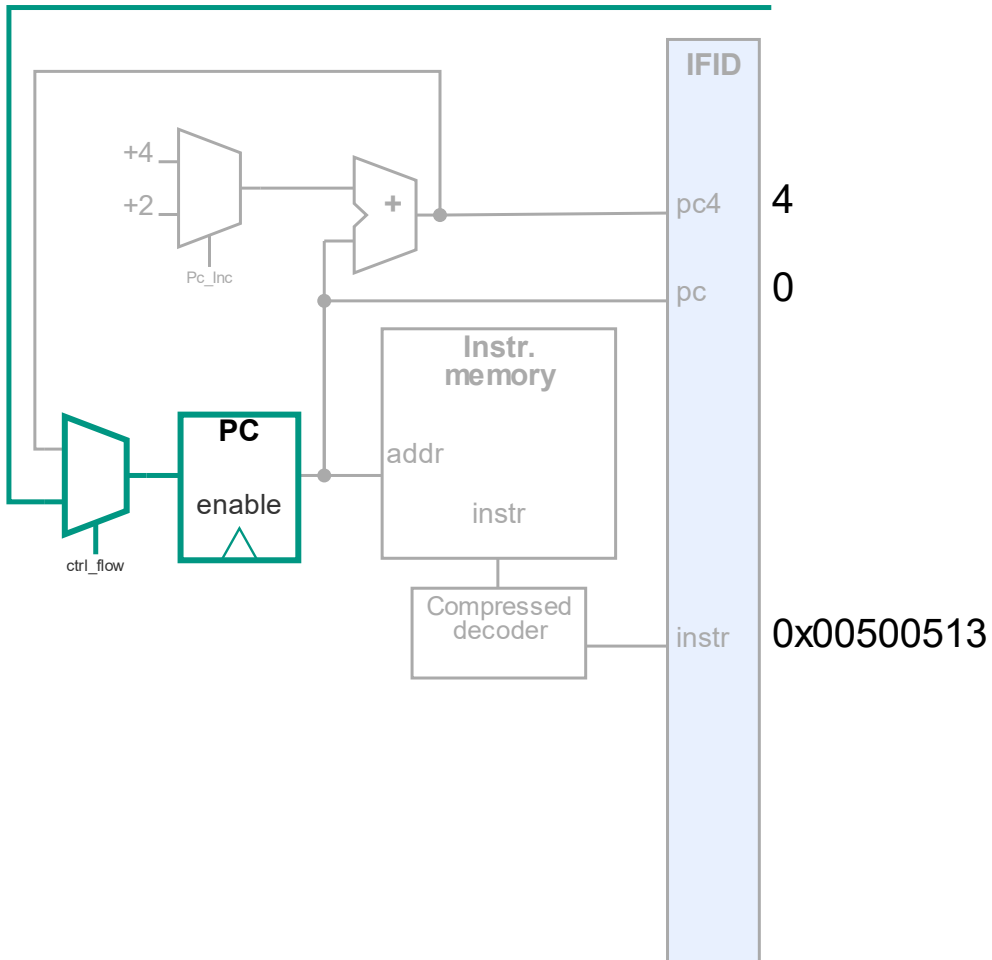
- Wir addieren PC + 4
(oder bei komprimierten Instruktionen +2)

IF Stage im Detail



- Wir addieren PC + 4
(oder bei komprimierten Instruktionen +2)
- Der neue Wert wird
zurückgeschrieben

IF Stage im Detail



- Wir addieren $PC + 4$
(oder bei komprimierten Instruktionen $+2$)
- Der neue Wert wird zurückgeschrieben
- Alternativ kann auch ein Wert aus einer späteren Pipeline Stage genommen werden (bspw. bei Sprüngen)

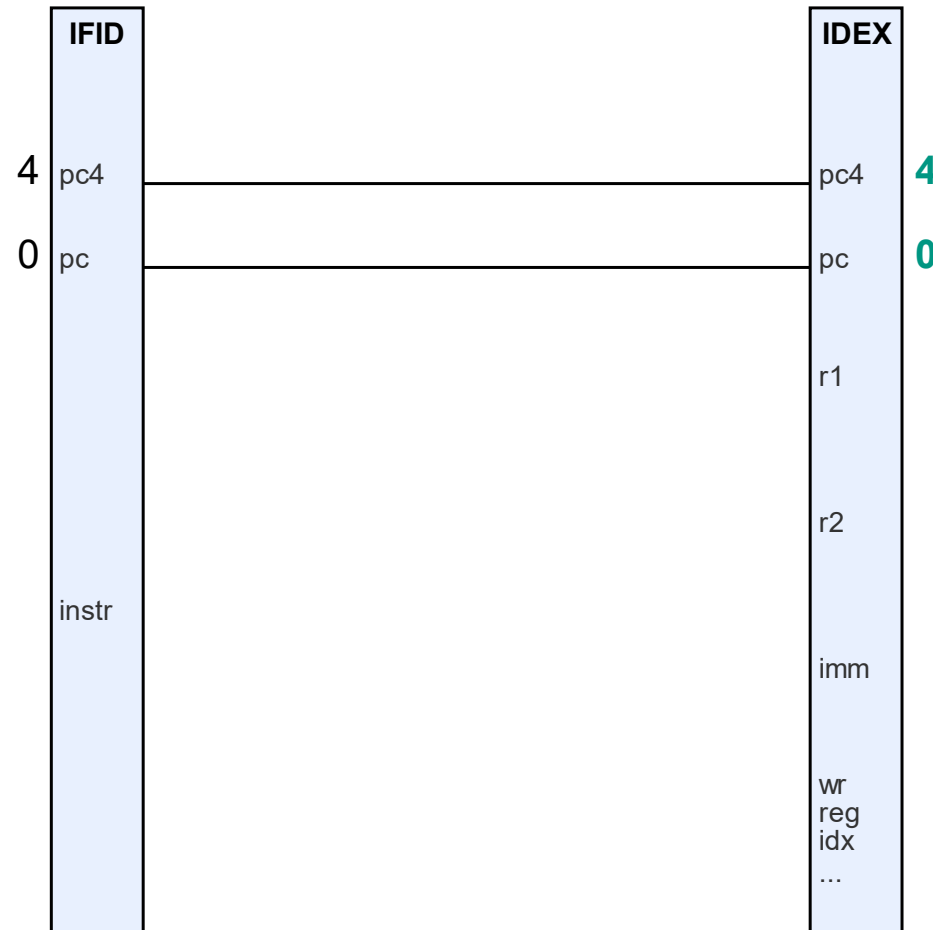
ID Stage im Detail

- **Ziel der Stage:** Instruktion dekodieren
 - Opcode bestimmen, Werte aus Registern laden, etc.

`jal ra, factorial` `addi a0, x0, 5`

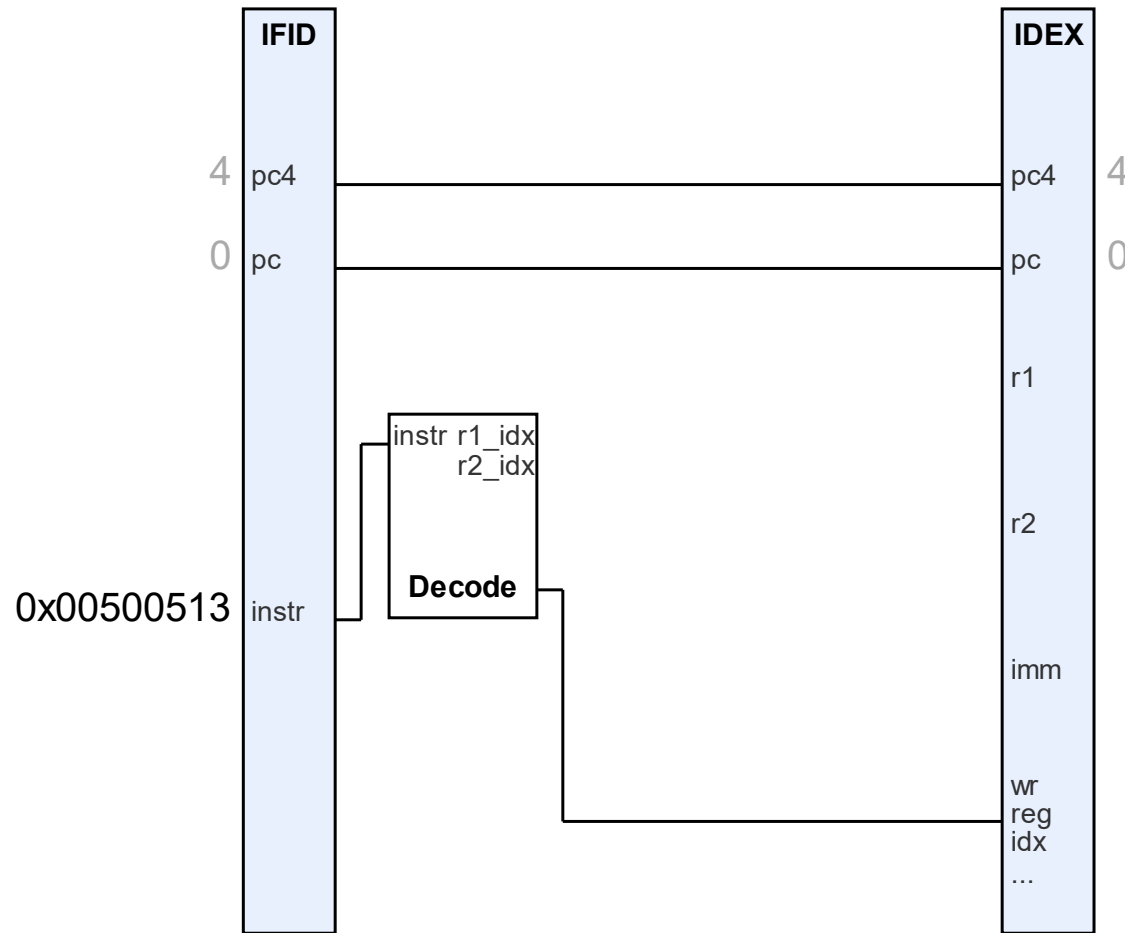


ID Stage im Detail



- Der aktuelle und nächste Program Counter werden einfach durchgereicht für spätere Stages

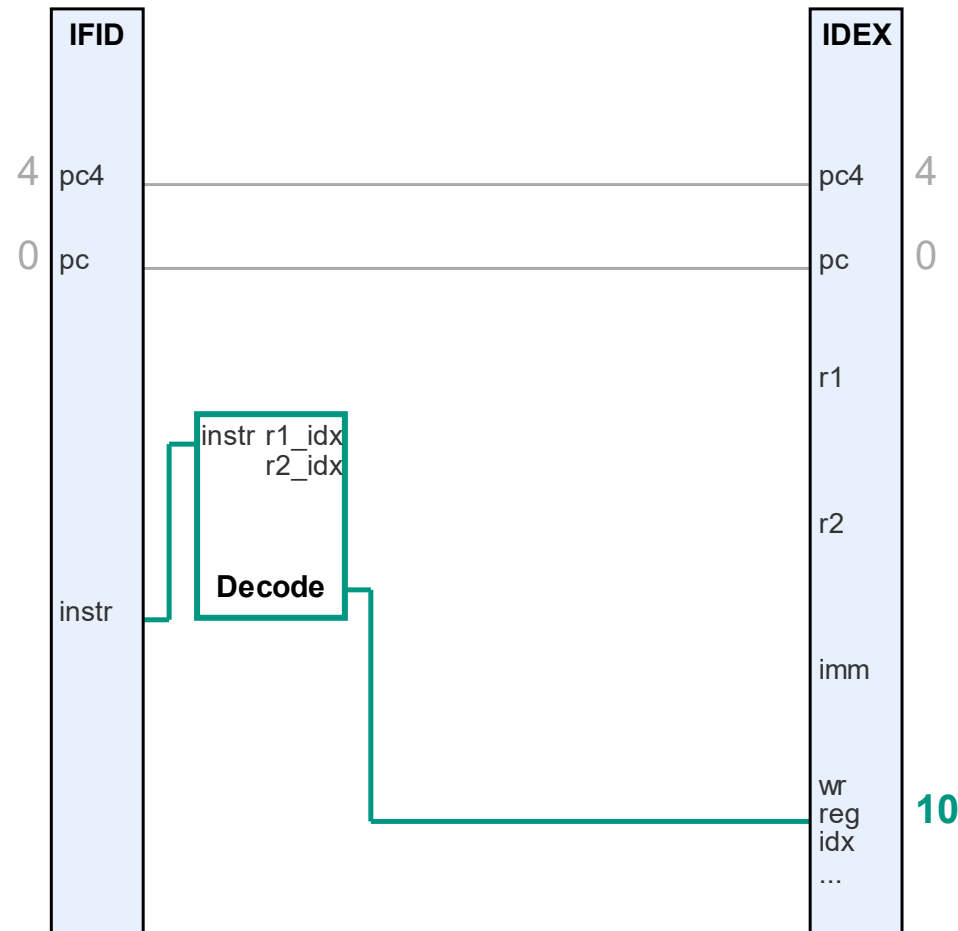
ID Stage im Detail



- Die Decode Komponente zieht das Zielregister (rd) aus der Instruktion

`addi x10 x0 5`

ID Stage im Detail



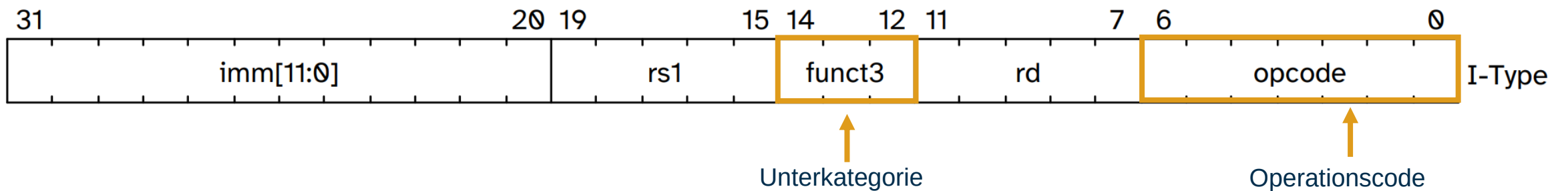
- Die Decode Komponente zieht das Zielregister (rd) aus der Instruktion
- Die Decode Komponente bestimmt den opcode aus der Instruktion

addi x10 x0 5

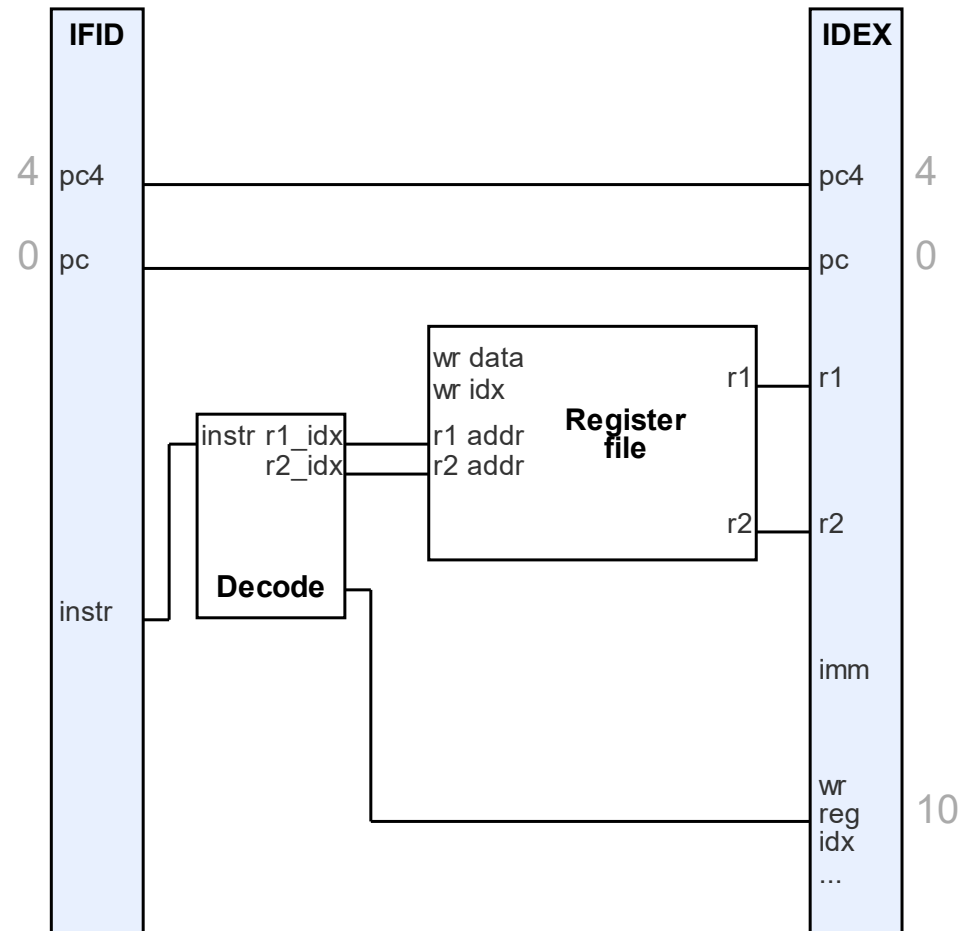
ID Stage im Detail

addi x10 x0 5

0000000001010000000010100010011
Addition Reg.-Imm.

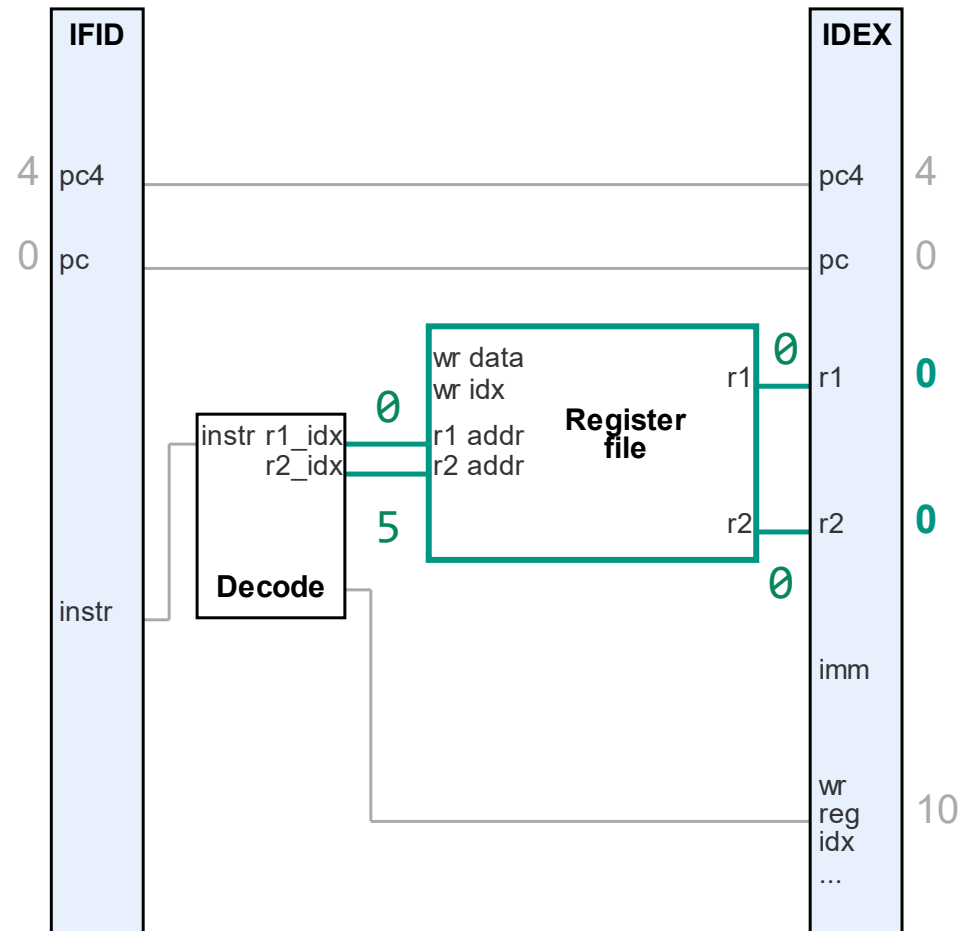


ID Stage im Detail



- Die Decode Komponente zieht das Zielregister (rd) aus der Instruktion
- Die Decode Komponente bestimmt den opcode aus der Instruktion
- Die benötigten Operanden werden aus den Registern geladen

ID Stage im Detail

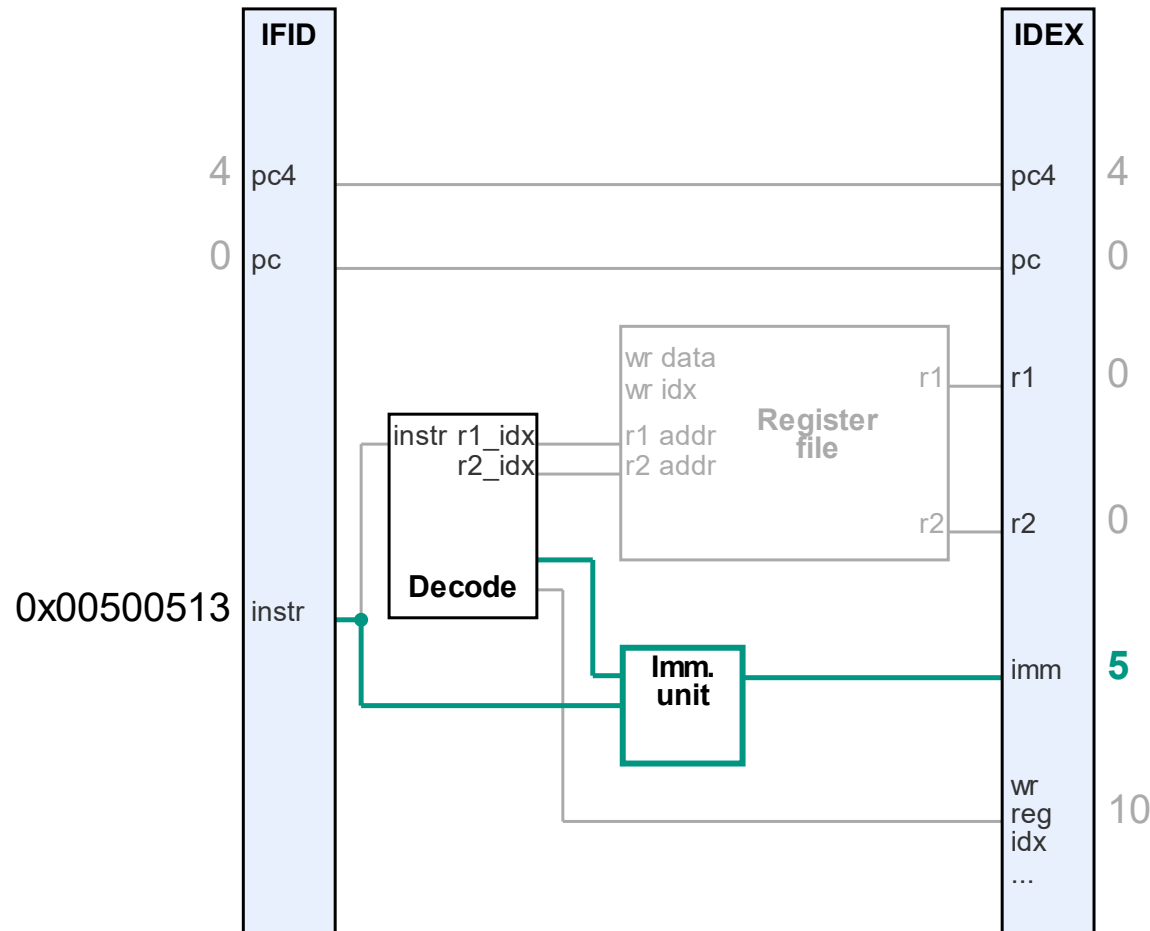


- Die Decode Komponente zieht das Zielregister (rd) aus der Instruktion
- Die Decode Komponente bestimmt den opcode aus der Instruktion
- Die benötigten Operanden werden aus den Registern geladen

addi x10 x0 5

000000000010100000000010100010011

ID Stage im Detail

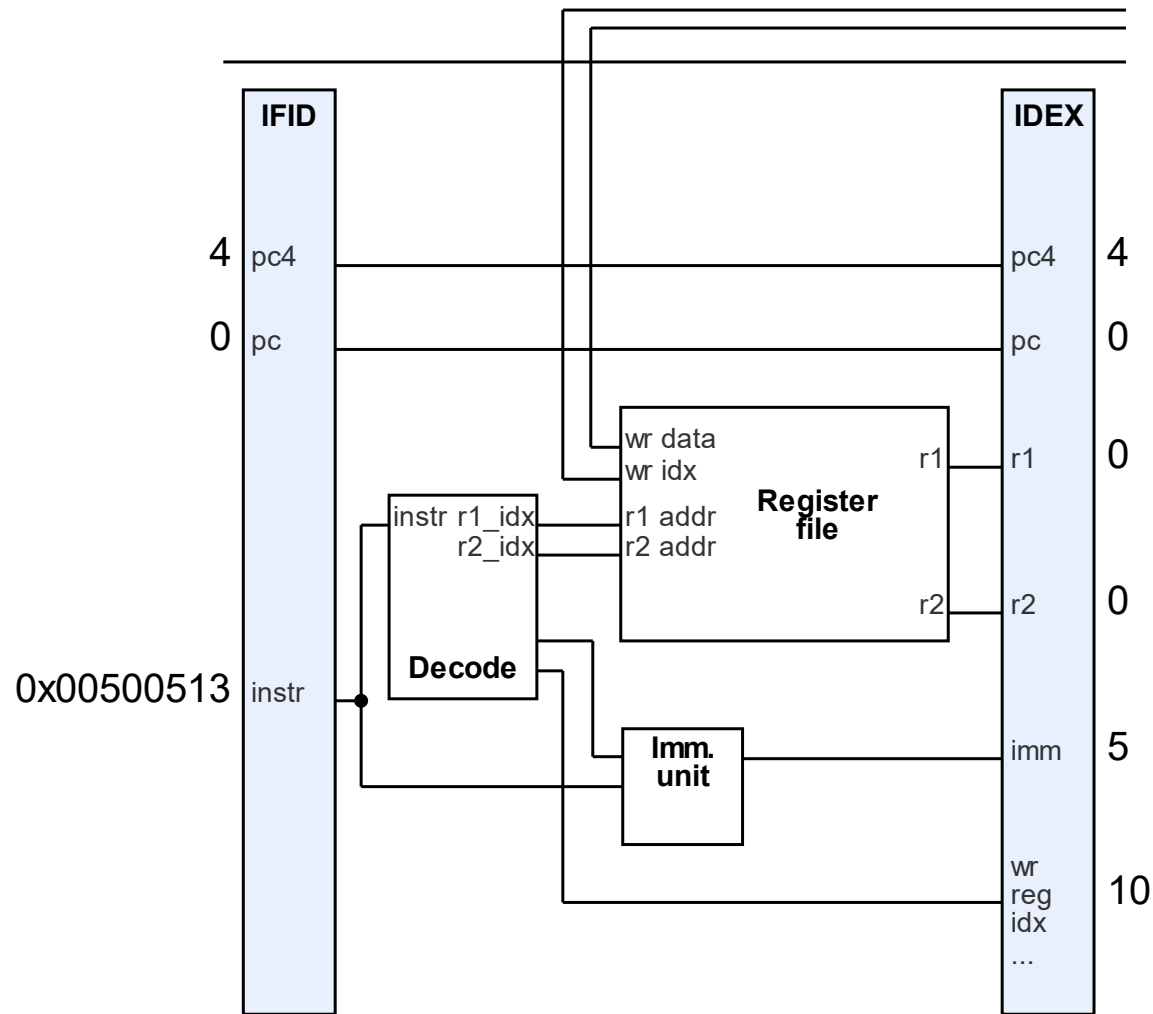


- Die Decode Komponente zieht das Zielregister (rd) aus der Instruktion
- Die Decode Komponente bestimmt den opcode aus der Instruktion
- Die benötigten Operanden werden aus den Registern geladen
- Der Immediate Wert wird rausgezogen

addi x10 x0 5

00000000010100000000010100010011

ID Stage im Detail



- **Wichtiger fehlender Baustein:**
 - **Control Unit (nicht dargestellt):** leitet aus Opcode eine Vielzahl von Kontrollsignalen ab, die in folgenden Stages gebraucht werden

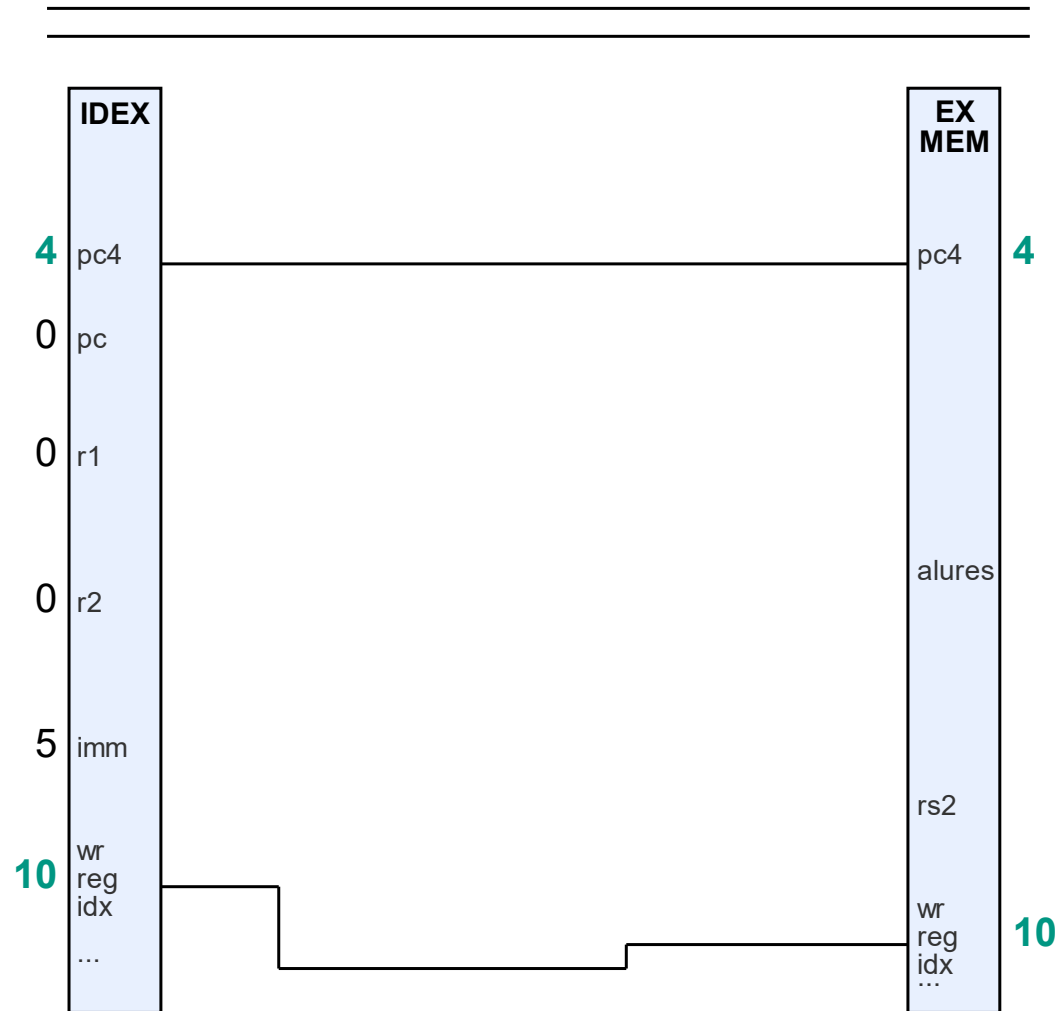
EX Stage im Detail

- **Ziel der Stage:** Instruktion ausführen
 - Addieren, Subtrahieren, Logische Operationen, ...

`addi a7, x0, 10` `jal ra, factorial` `addi a0, x0, 5`

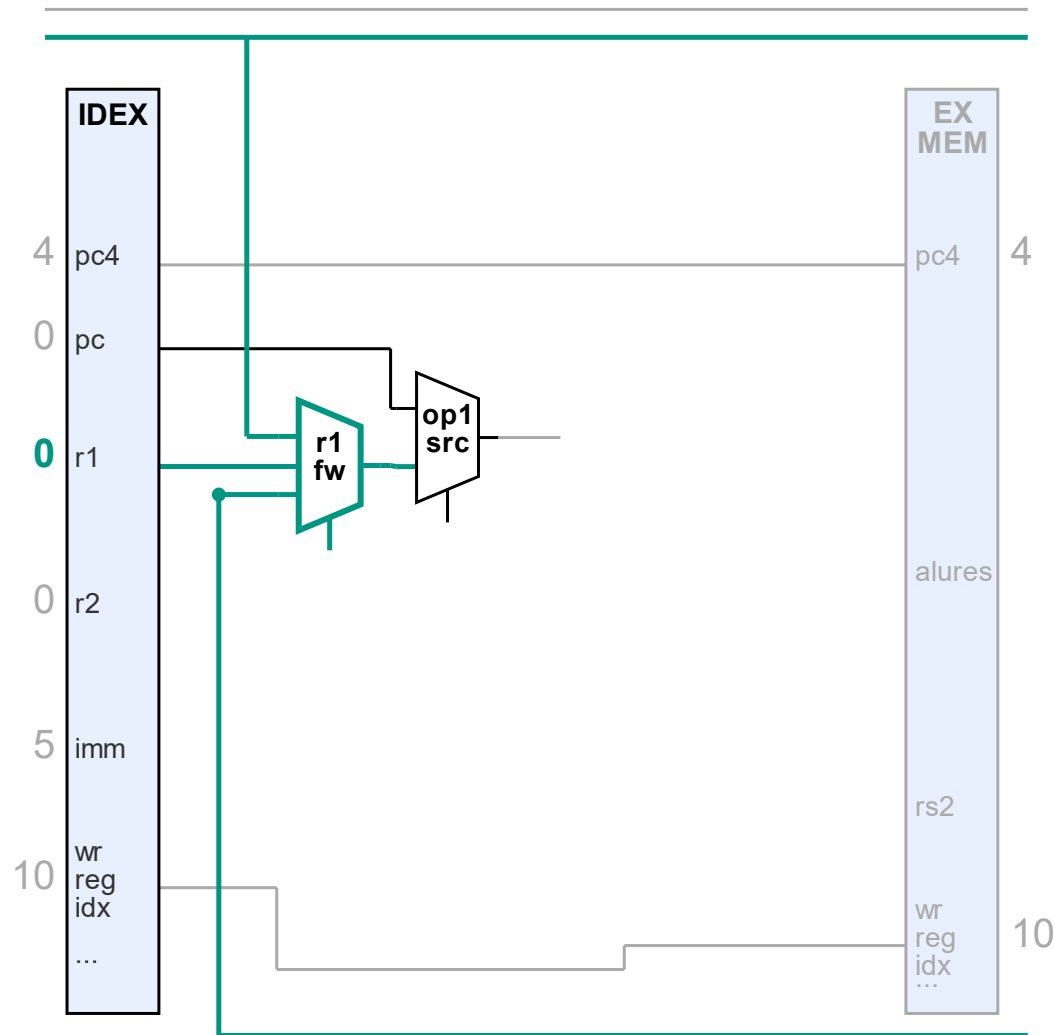


EX Stage im Detail



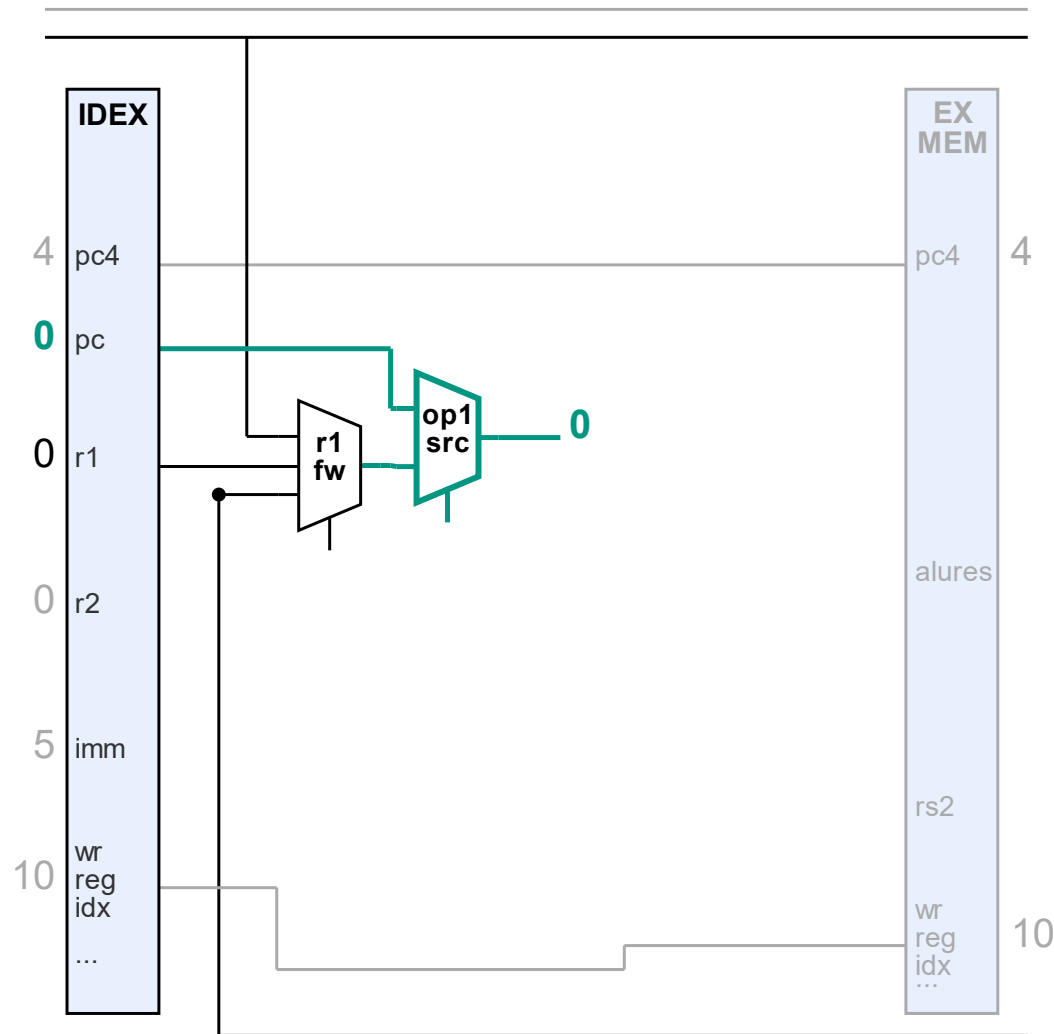
- Der inkrementierte Program Counter und das Ausgaberegister können durchgereicht werden

EX Stage im Detail



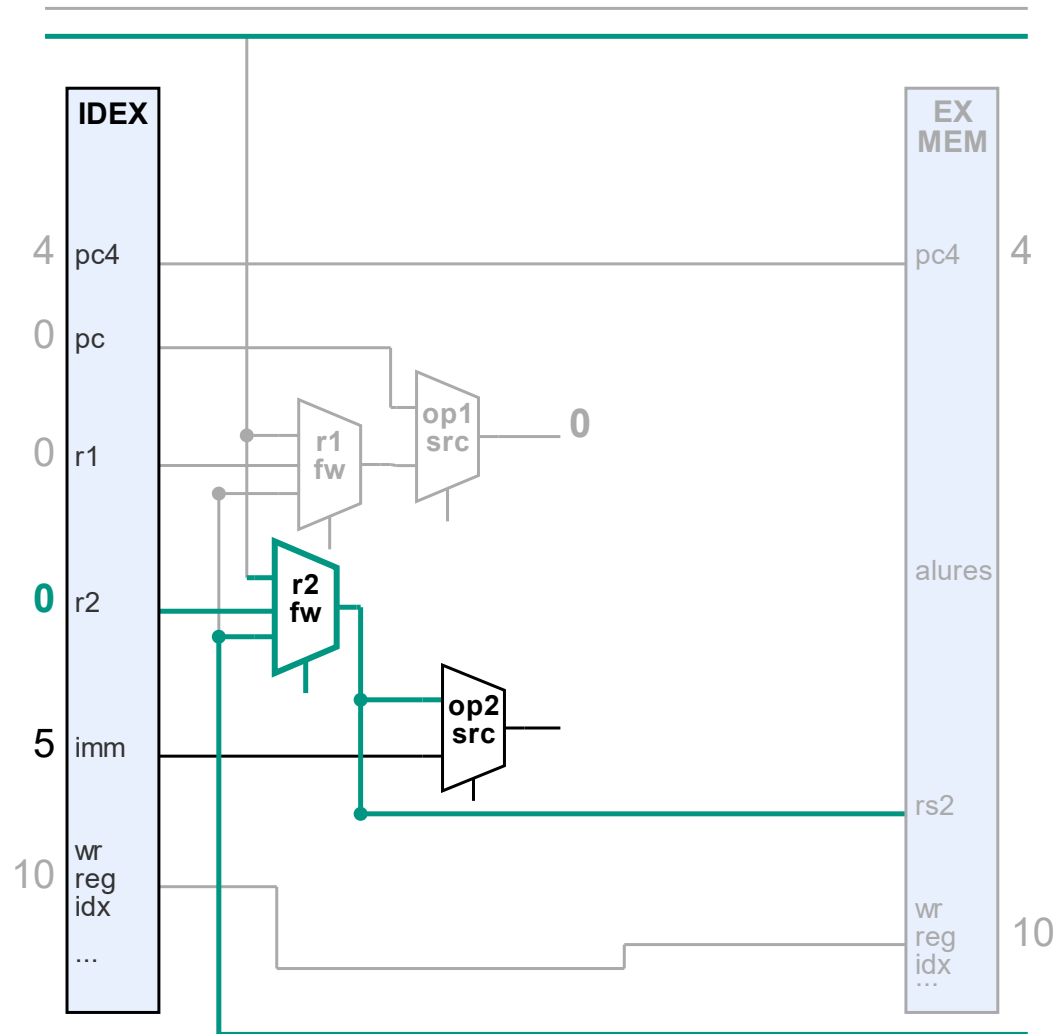
- Die Werte für den Program Counter und das Ausgaberegister können durchgereicht werden
- R1 Wert für ALU bestimmen
 - r1 fw (Multiplexer):
 1. Nimm R1 Wert aus Decode Stage
 2. Forwarding aus Memory Stage
 3. Forwarding aus Writeback Stage

EX Stage im Detail



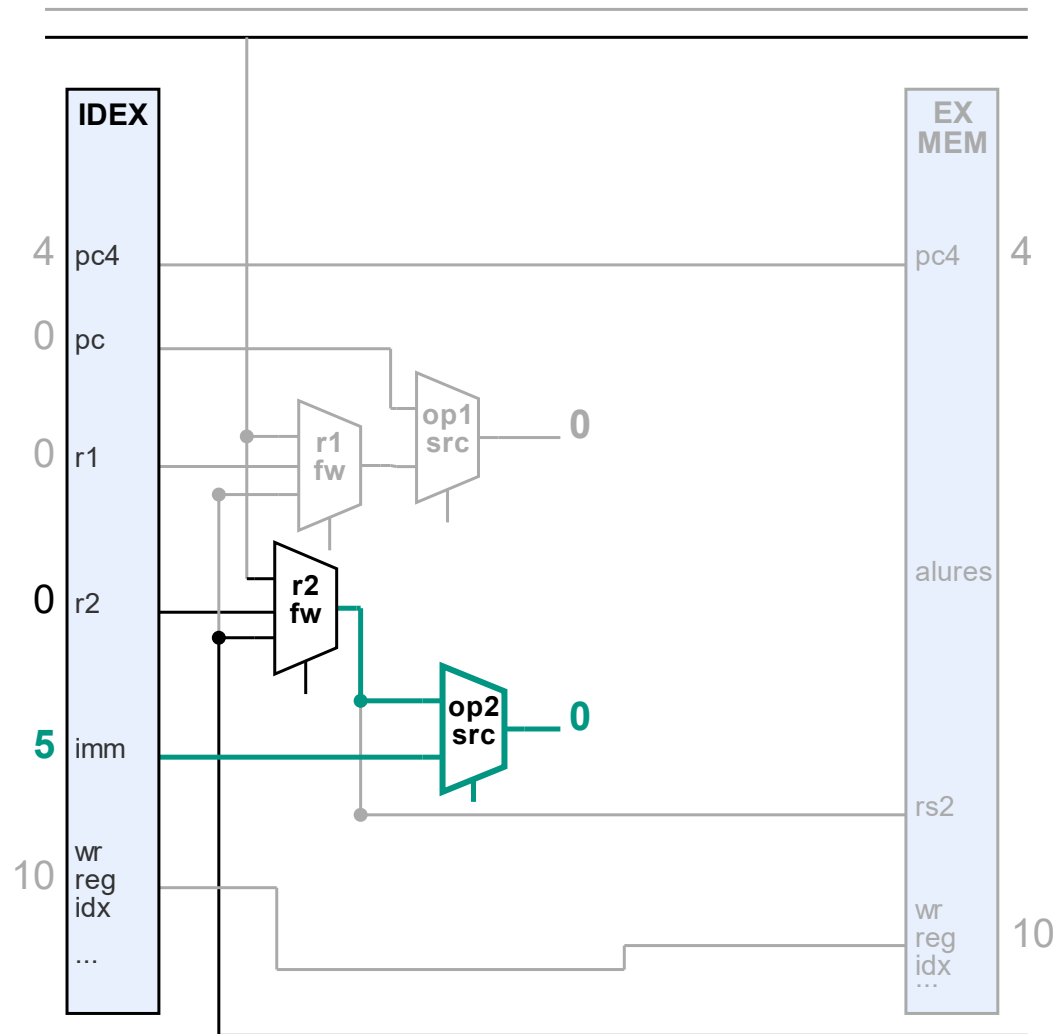
- Die Werte für den Program Counter und das Ausgaberegister können durchgereicht werden
- R1 Wert für ALU bestimmen
 - r1 fw (Multiplexer)
 - op1 src (Multiplexer):
 1. Nimm Wert aus r1 fw
 2. Nimm PC Wert (z.B. auipc Instruktion)

EX Stage im Detail



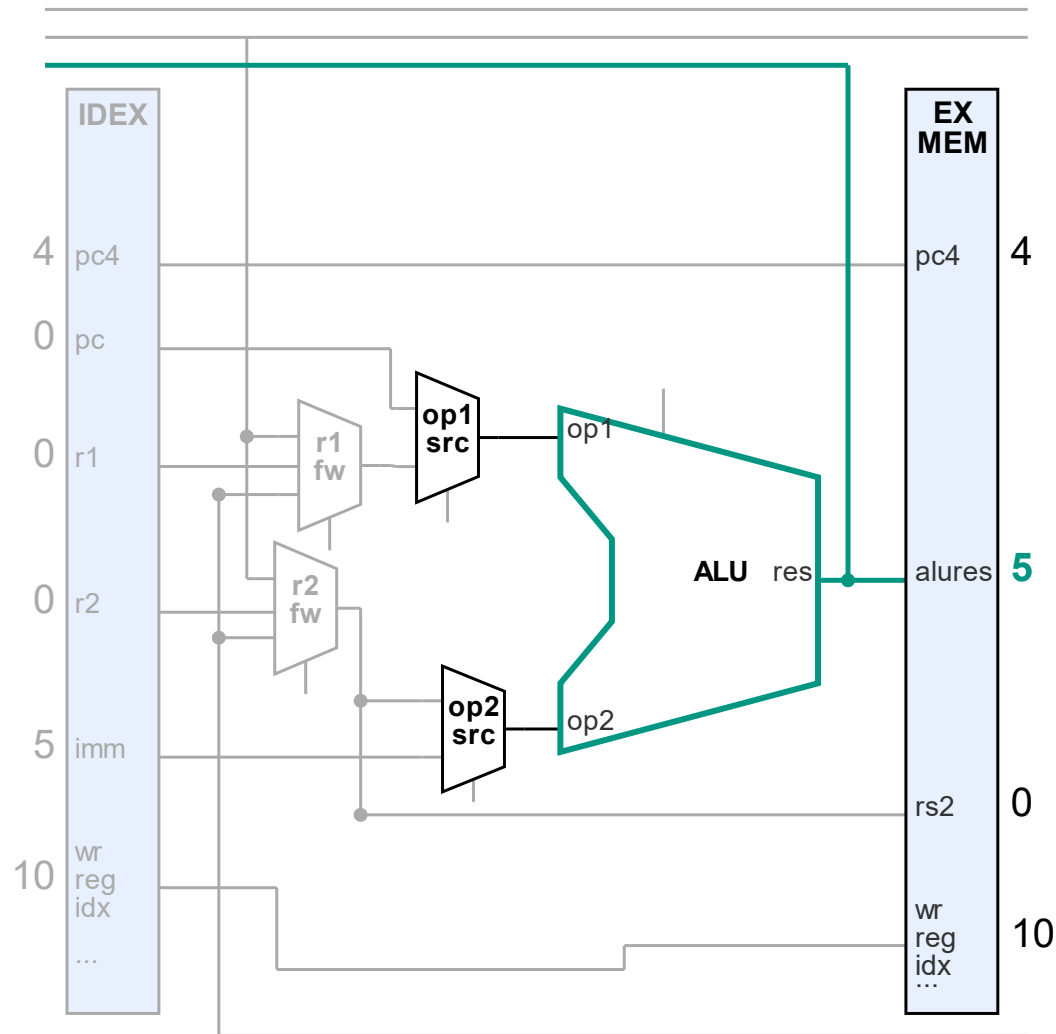
- Die Werte für den Program Counter und das Ausgaberegister können durchgereicht werden
- R1 Wert für ALU bestimmen
 - r1 fw (Multiplexer)
 - op1 src (Multiplexer)
- R2 Wert für ALU bestimmen
 - r2 fw (Multiplexer) (wie r1 fw):
 1. Nimm R2 Wert aus Decode Stage
 2. Forwarding aus Memory Stage
 3. Forwarding aus Writeback Stage

EX Stage im Detail



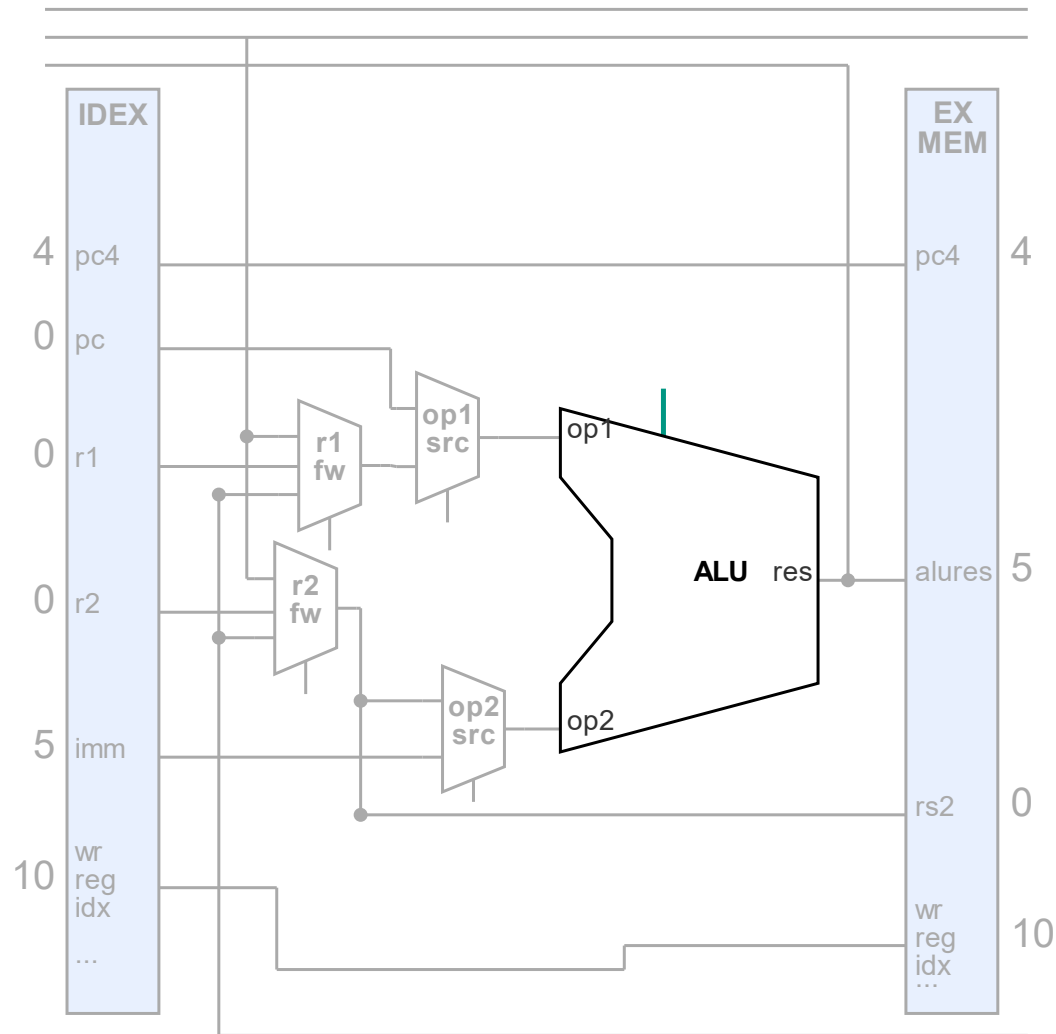
- Die Werte für den Program Counter und das Ausgaberegister können durchgereicht werden
- R1 Wert für ALU bestimmen
 - r1 fw (Multiplexer)
 - op1 src (Multiplexer)
- R2 Wert für ALU bestimmen
 - r2 fw (Multiplexer)
 - op2 src
 1. Für Register-Register Instruktionen (z.B. add) nimm den Wert aus R2
 2. Für alles andere (z.B. addi) nimm den Immediate Wert

EX Stage im Detail



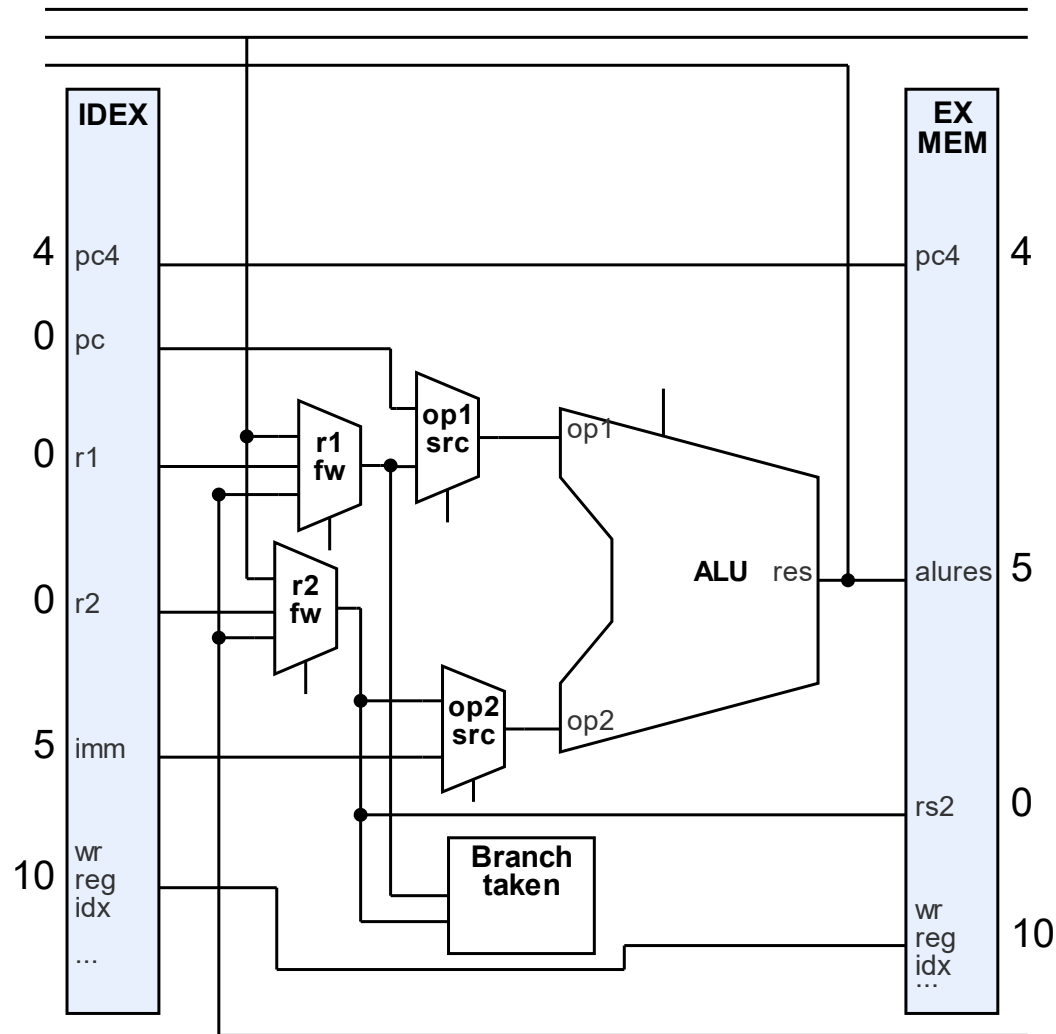
- Die Werte für den Program Counter und das Ausgaberegister können durchgereicht werden
- R1 Wert für ALU bestimmen
 - r1 fw (Multiplexer)
 - op1 src (Multiplexer)
- R2 Wert für ALU bestimmen
 - r2 fw (Multiplexer)
 - op2 src
- Die eigentliche Addition

EX Stage im Detail



- Die Werte für den Program Counter und das Ausgaberegister können durchgereicht werden
- R1 Wert für ALU bestimmen
 - r1 fw (Multiplexer)
 - op1 src (Multiplexer)
- R2 Wert für ALU bestimmen
 - r2 fw (Multiplexer)
 - op2 src
- Die eigentliche Addition
- Steuerung der ALU über Kontrollsignal

EX Stage im Detail



■ Wichtige fehlende Bausteine

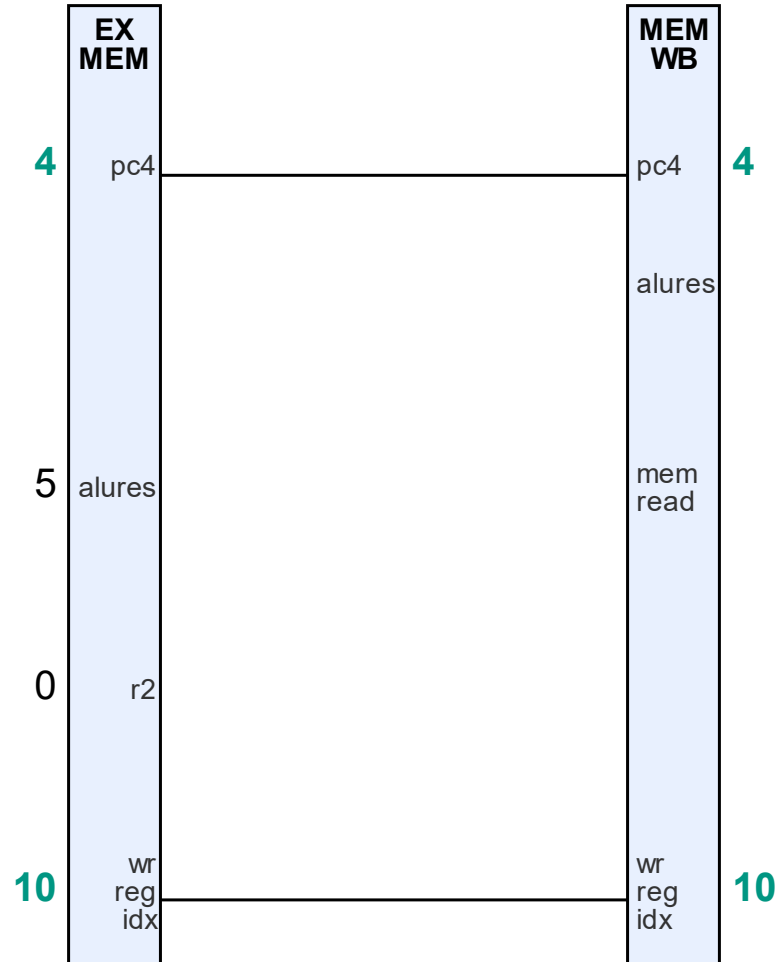
- Branch taken: entscheidet ob wir bei einer Branch Instruktionen springen oder nicht (hier unvollständig dargestellt)
- Forwarding Unit: setzt ggf. `r1fw` oder `r2fw` Kontrollbits damit der Forwarding Wert aus späterem Register genutzt wird
- Hazard Unit: Detektiert unlösbare Abhängigkeiten und löst Stalls aus

MEM Stage im Detail

- **Ziel der Stage: Speicherzugriffe**
 - In den Speicher schreiben / aus dem Speicher lesen
 - für die Addition nicht relevant

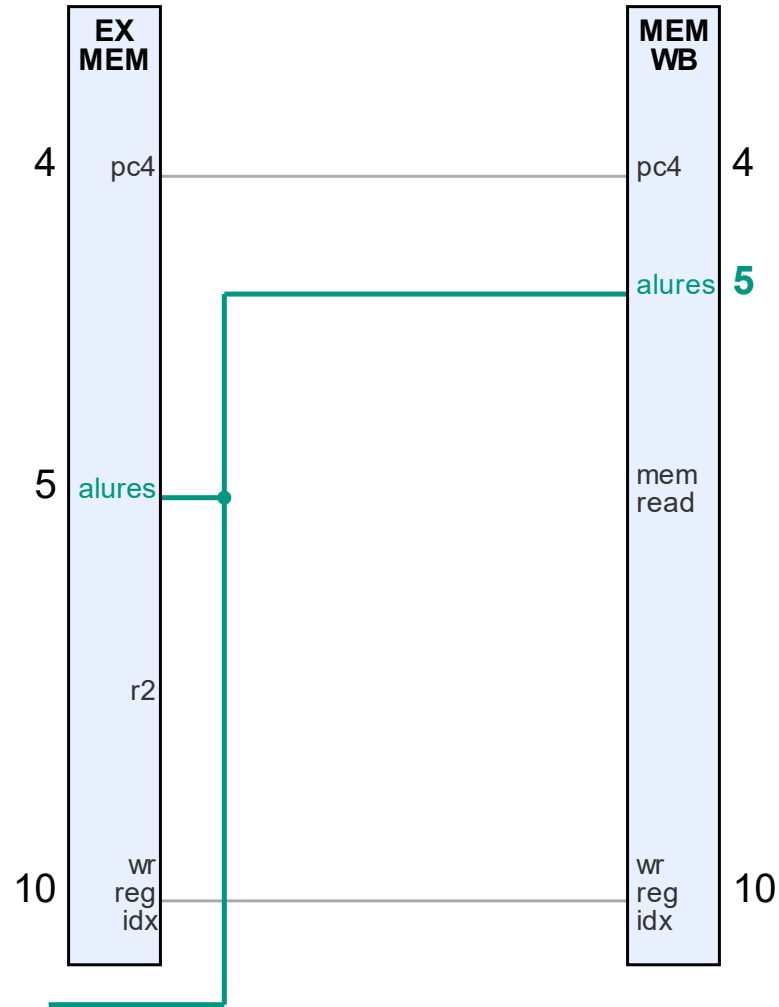


MEM Stage im Detail



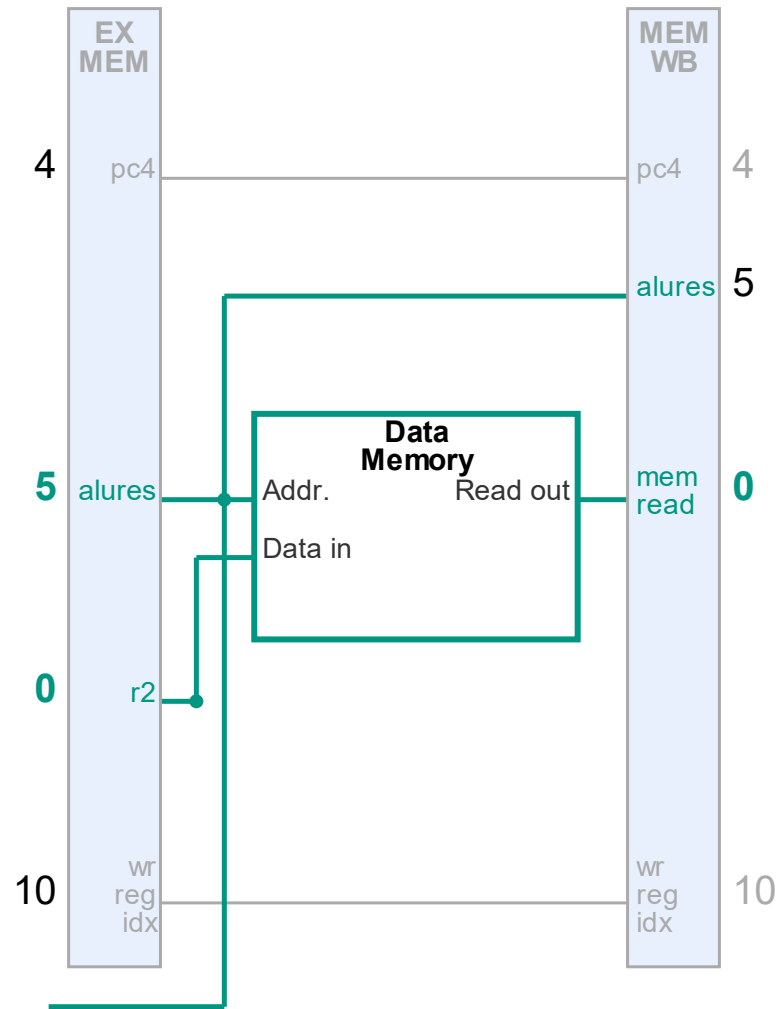
- PC4 und Zielregister werden durchgereicht

MEM Stage im Detail



- PC4 und Zielregister werden durchgereicht
- Ergebnis der ALU wird durchgereicht (und zurückgereicht für eventuelles Forwarding)

MEM Stage im Detail



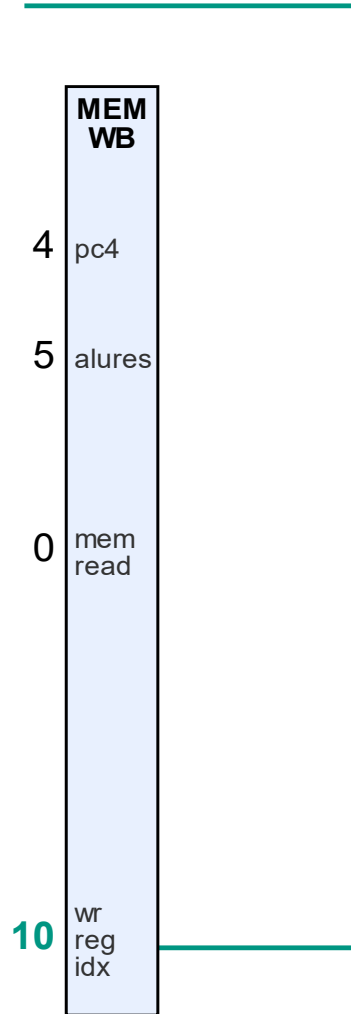
- PC4 und Zielregister werden durchgereicht
- Ergebnis der ALU wird durchgereicht (und zurückgereicht für eventuelles Forwarding)
- Falls Daten zu schreiben sind, gibt uns alures die Adresse und R2 den Wert
- Falls Daten zu lesen sind werden sie in mem read geladen

WB Stage im Detail

- **Ziel der Stage:** Write Back in das Register
 - Ergebnisse ins Zielregister speichern

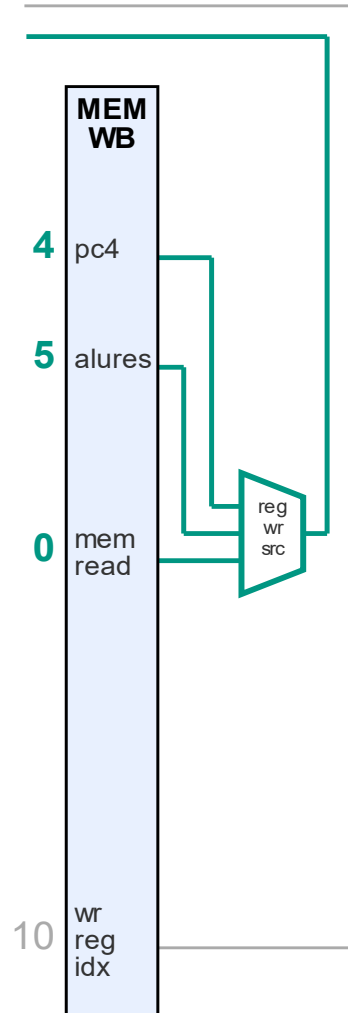


WB Stage im Detail



- Es muss angegeben werden in welches Register geschrieben wird

WB Stage im Detail



- Es muss angegeben werden in welches Register geschrieben wird
- Außerdem muss der zu schreibende Wert übergeben werden
 - Bei jal soll die return address pc4 geschrieben werden
 - Bei Arithmetik soll das ALU Ergebnis geschrieben werden
 - Bei Load Instruktionen soll der geladene Wert geschrieben werden

JAL Lebenszyklus

- Als nächstes vergleichen wir die Ausführung der *jal* Instruktion

```
main:
    addi    a0, x0, 5
    jal     ra, factorial

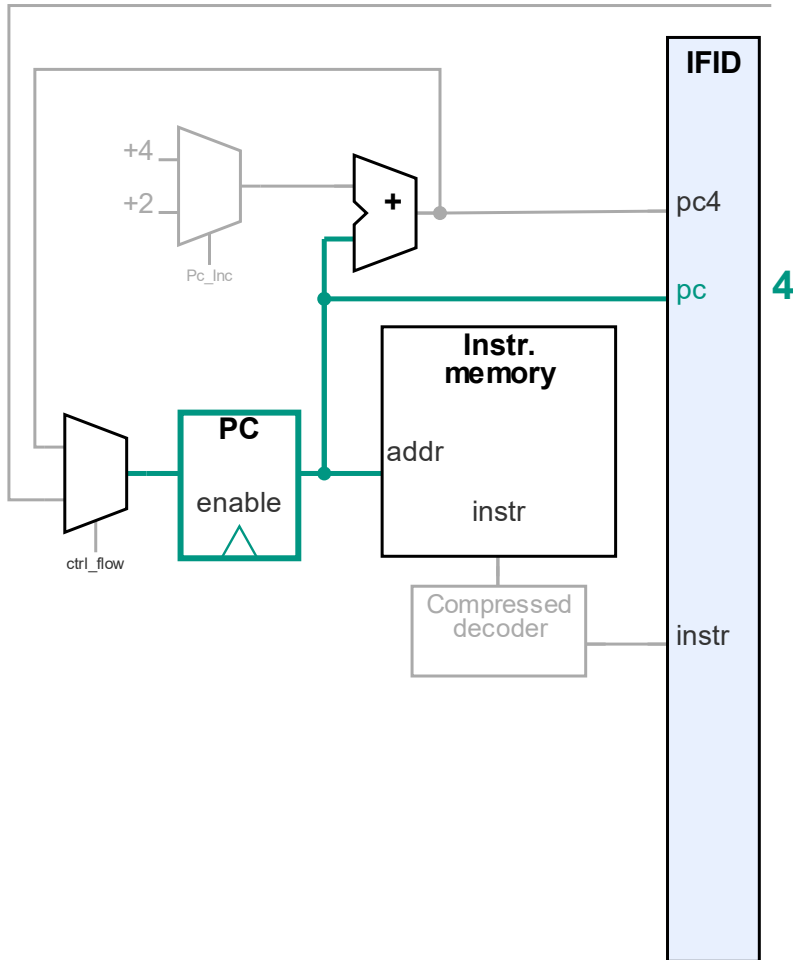
    addi    a7, x0, 10
    ecall
```

```
factorial:
    addi    a1, x0, 1
```

```
jal ra, factorial    addi a0, x0, 5
```

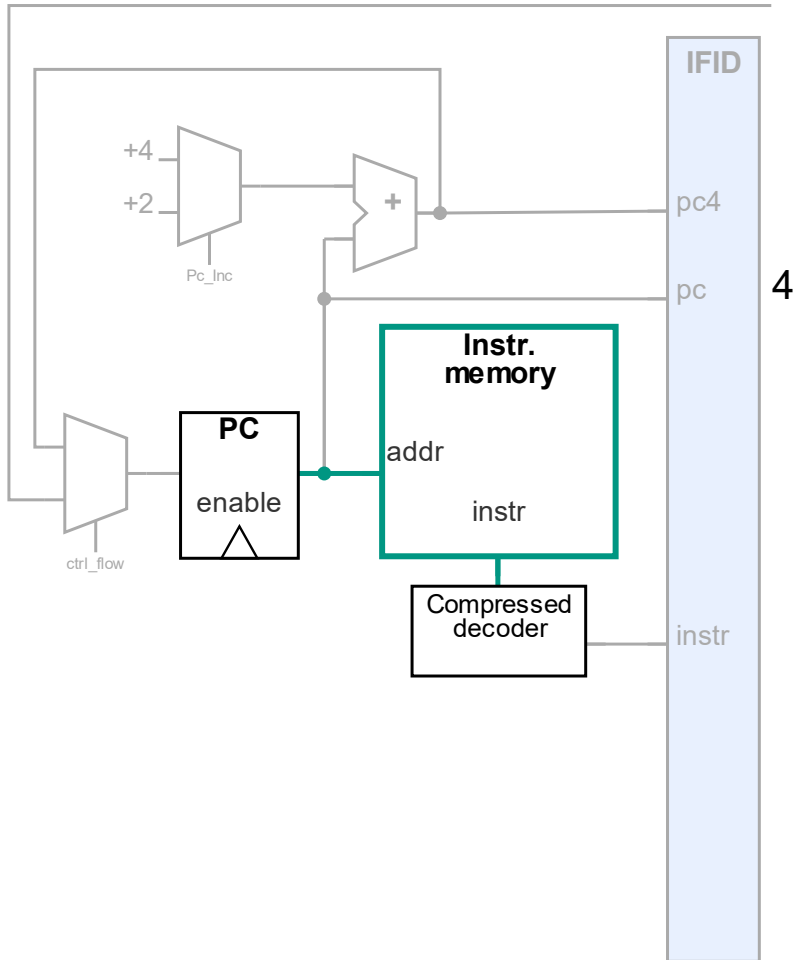


IF Stage im Detail



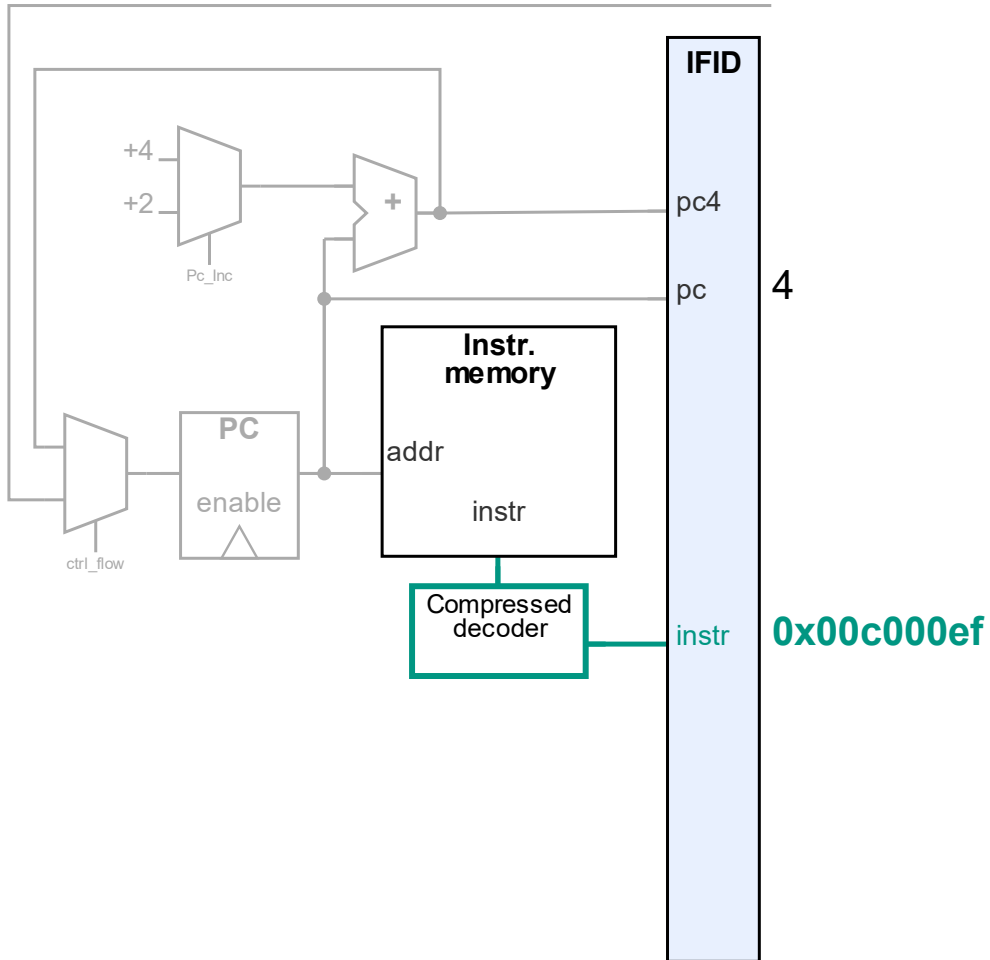
- Zunächst wird wieder eine Instruktion gefetched

IF Stage im Detail



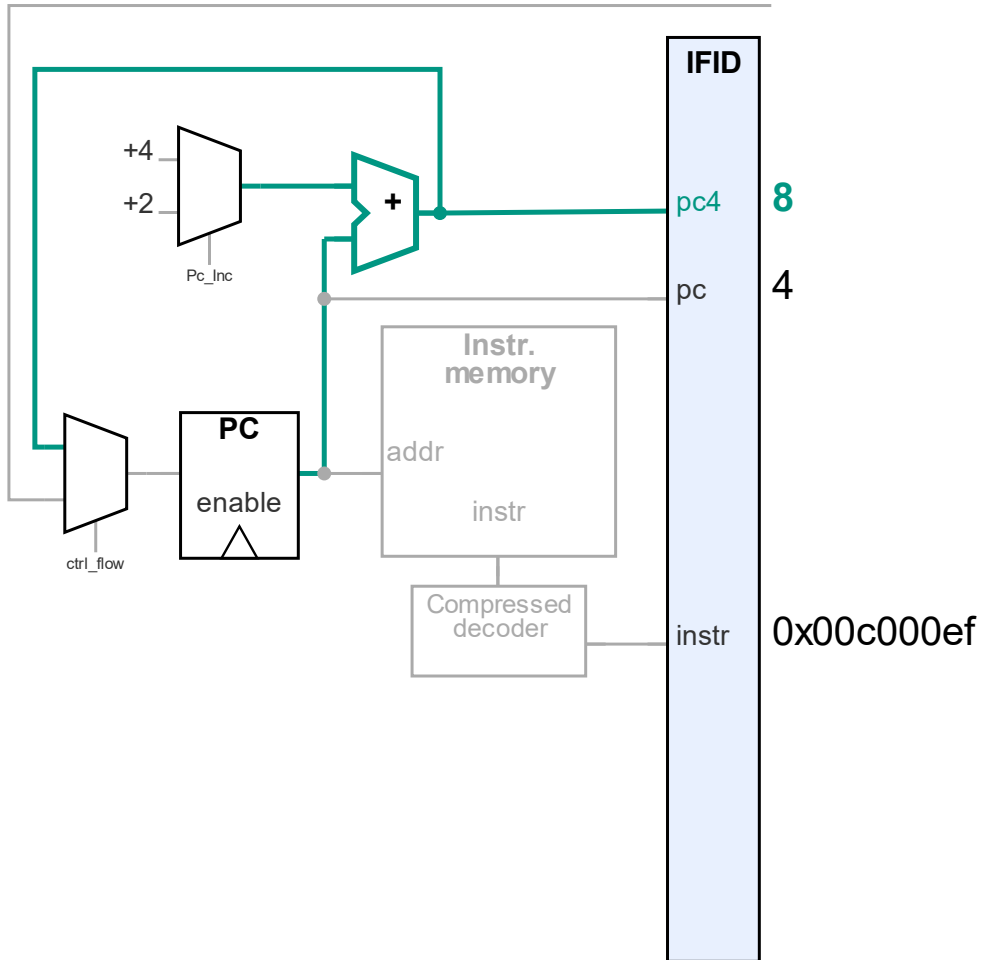
- Zunächst wird wieder eine Instruktion gefetched
 - Der erhöhte Program Counter gibt diesmal die jal Instruktion

IF Stage im Detail



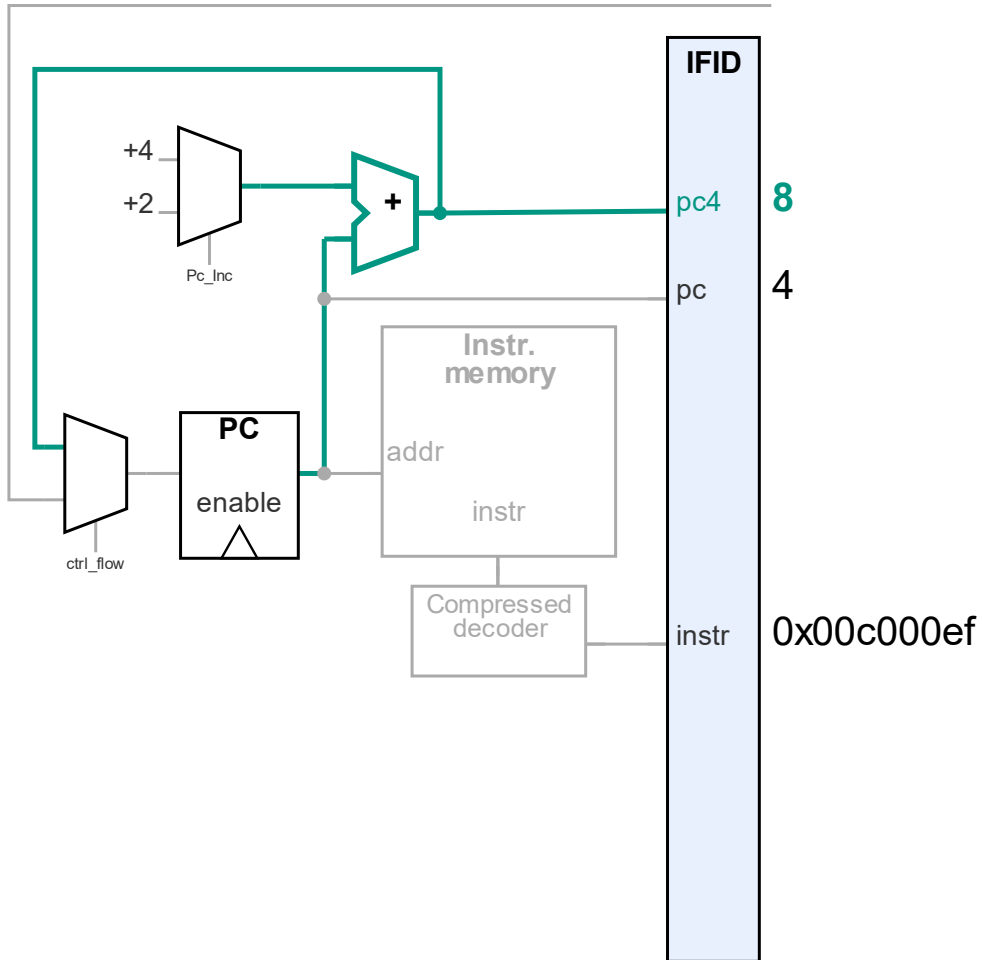
- Zunächst wird wieder eine Instruktion gefetched
 - Der erhöhte Program Counter gibt diesmal die jal Instruktion
 - (Wie zuvor ist sie nicht komprimiert)

IF Stage im Detail



- Zunächst wird wieder eine Instruktion gefetcht
- Der Program Counter muss inkrementiert werden

IF Stage im Detail



- Zunächst wird wieder eine Instruktion gefetcht
- Der Program Counter muss inkrementiert werden

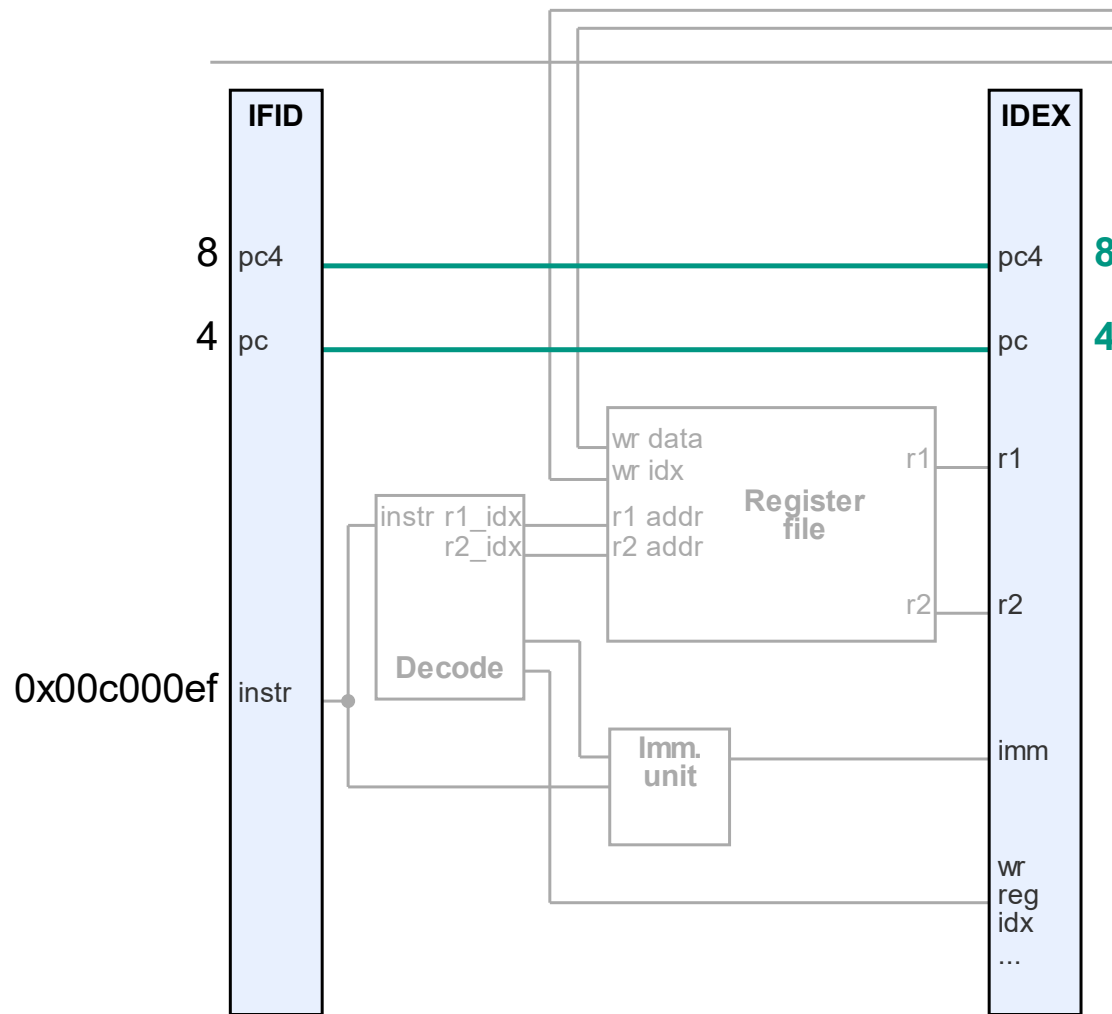
JAL Decode

- Andere Werte, gleiches Verfahren

`addi a7, x0, 10` `jal ra, factorial` `addi a0, x0, 5`

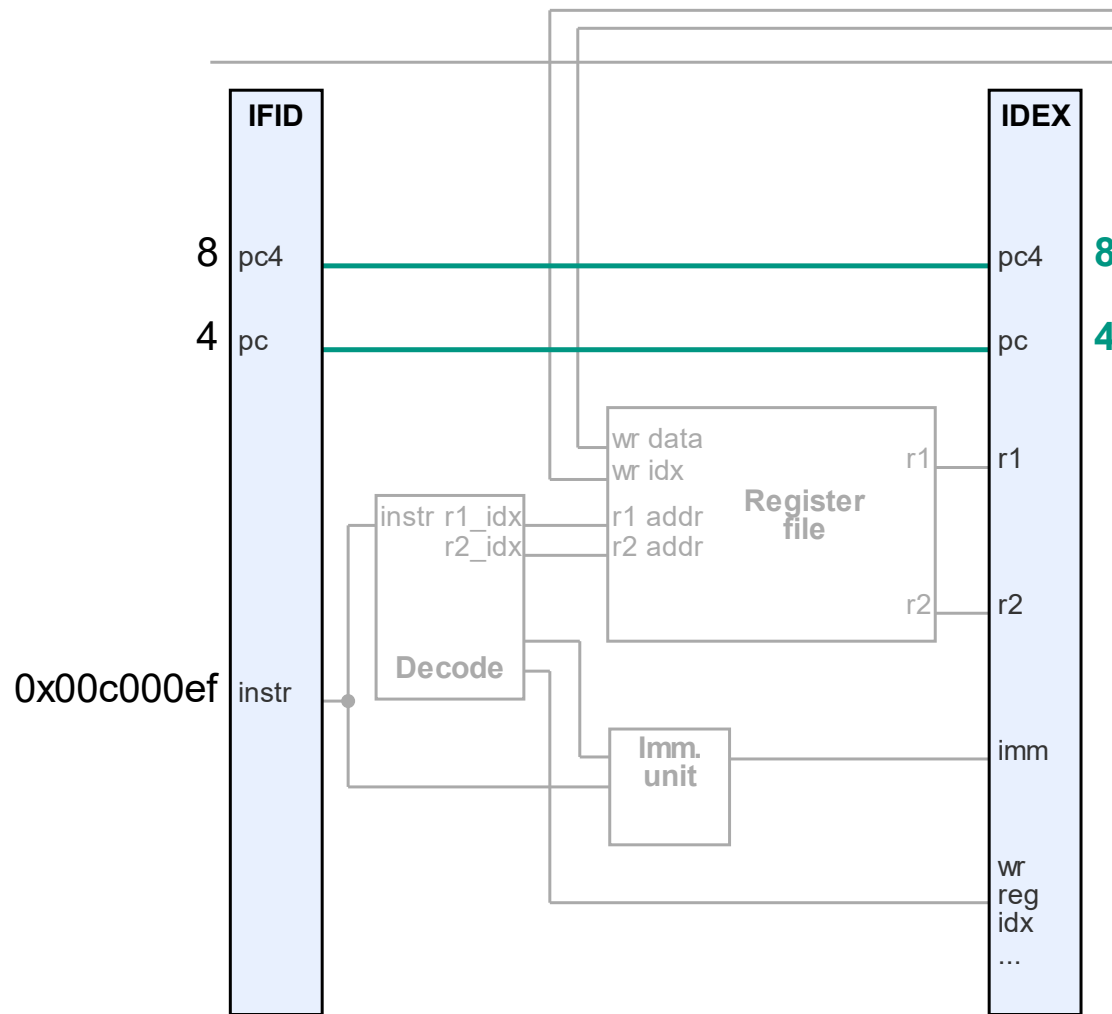


ID Stage im Detail



- PC Werte werden wieder durchgereicht

ID Stage im Detail



- PC Werte werden wieder durchgereicht
- Wie wird dekodiert?

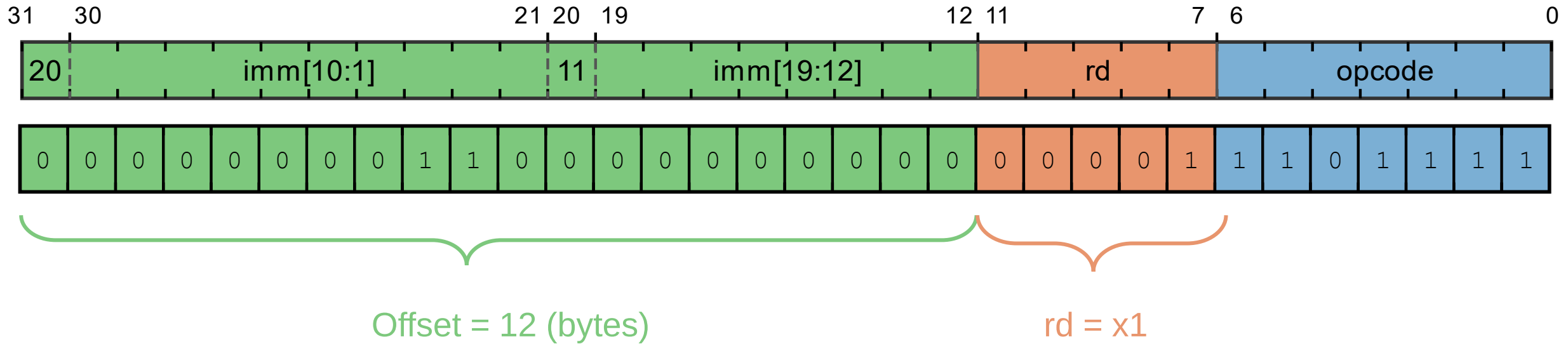
Wiederholung: Register Konventionen

- Wir erinnern uns an die Konventionen aus Vorlesung 2
- x1 wird typischerweise als Return address genutzt

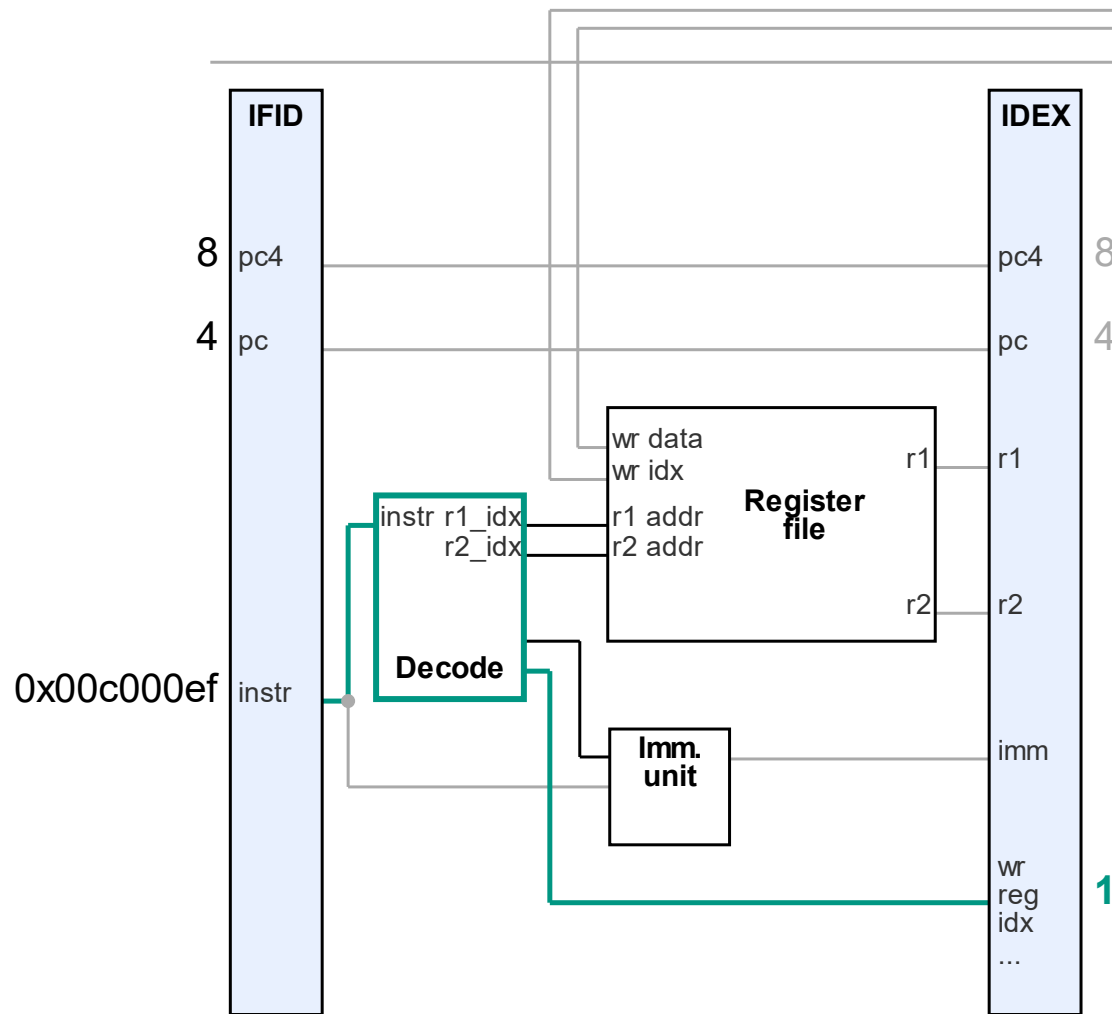
RISC-V Register Konvention

Name	ABI Mnemonic	Meaning
x0	zero	Zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5-x7	t0-t2	Temporary registers
x8-x9	s0-s1	Callee-saved registers
x10-x17	a0-a7	Argument registers
x18-x27	s2-s11	Callee-saved registers
x28-x31	t3-t6	Temporary registers

jal Instruktion

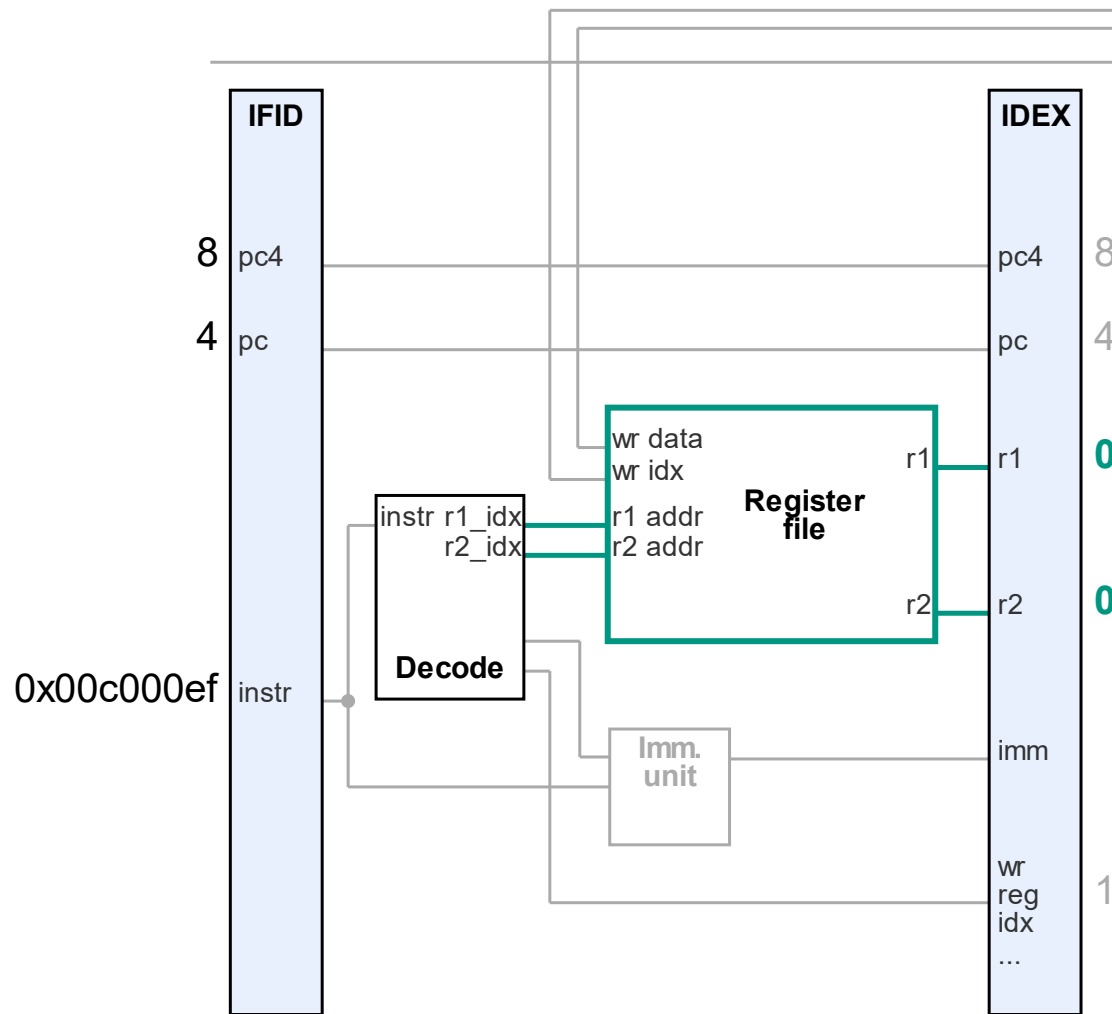


ID Stage im Detail



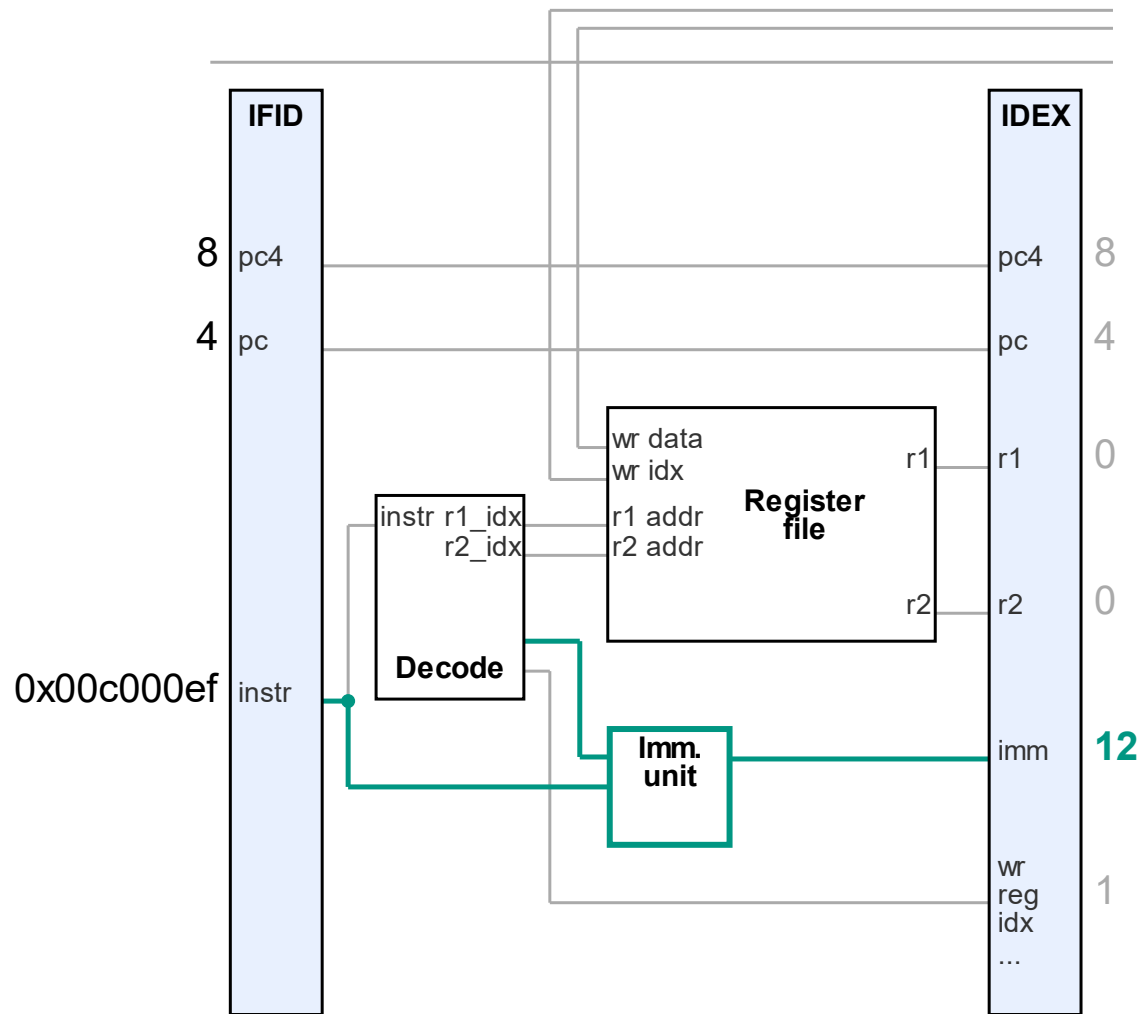
- PC Werte werden wieder durchgereicht
- Zielregister ist das Register für die Return address

ID Stage im Detail



- PC Werte werden wieder durchgereicht
- Die Decode Stage braucht diesmal keine Operanden dekodieren (versucht es aber trotzdem)
- Register Indexe bringen in beiden Fällen den Wert 0

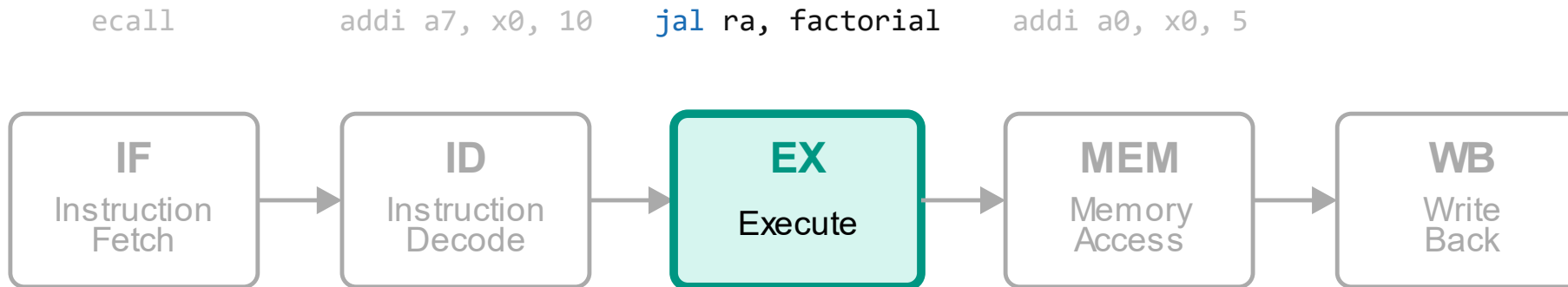
ID Stage im Detail



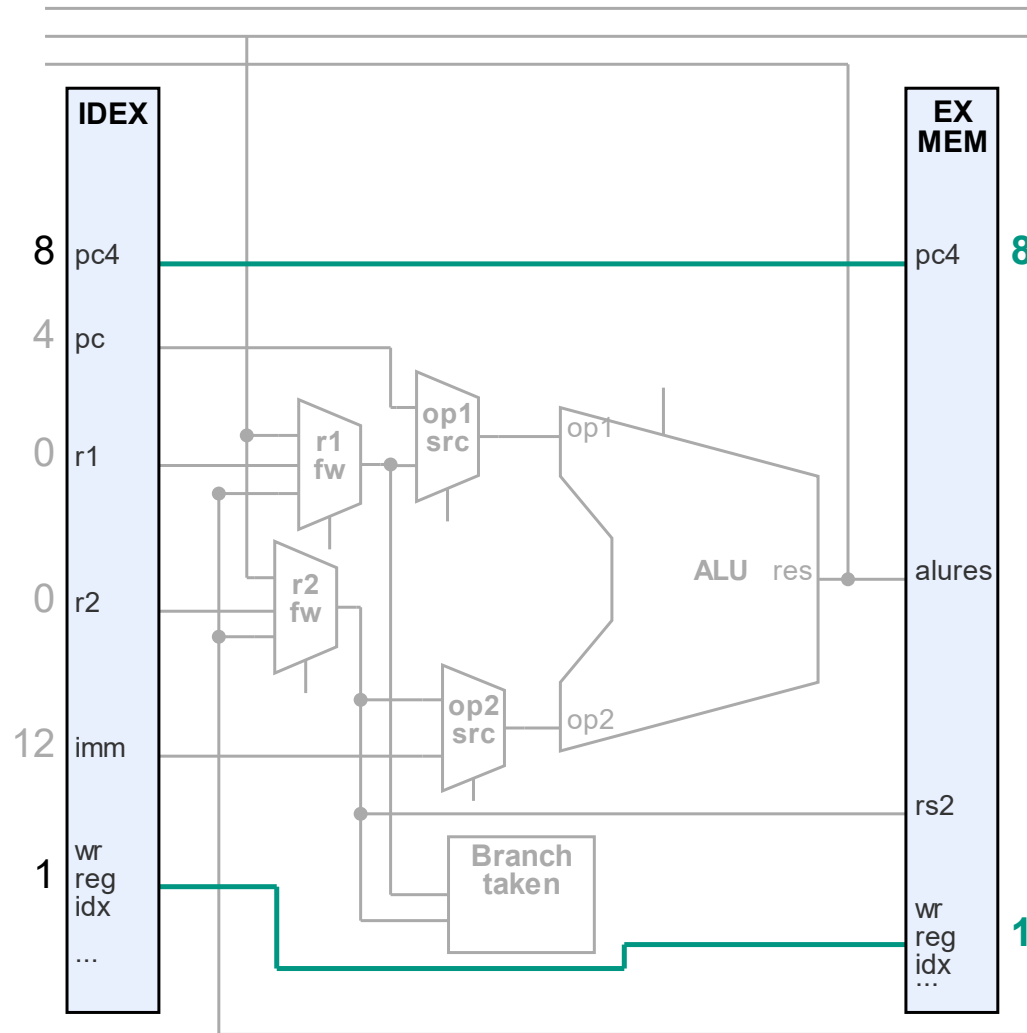
- PC Werte werden wieder durchgereicht
- Die Decode Stage braucht diesmal keine Operanden dekodieren (versucht es aber trotzdem)
- Register Indexe bringen in beiden Fällen den Wert 0
- Der Immediate Wert (Offset des Sprungs) wird ausgelesen

JAL EX

- **Ziel der Stage:** Instruktion ausführen
- Konkret:
 - Zieladresse des Sprungs berechnen
 - PC verändern

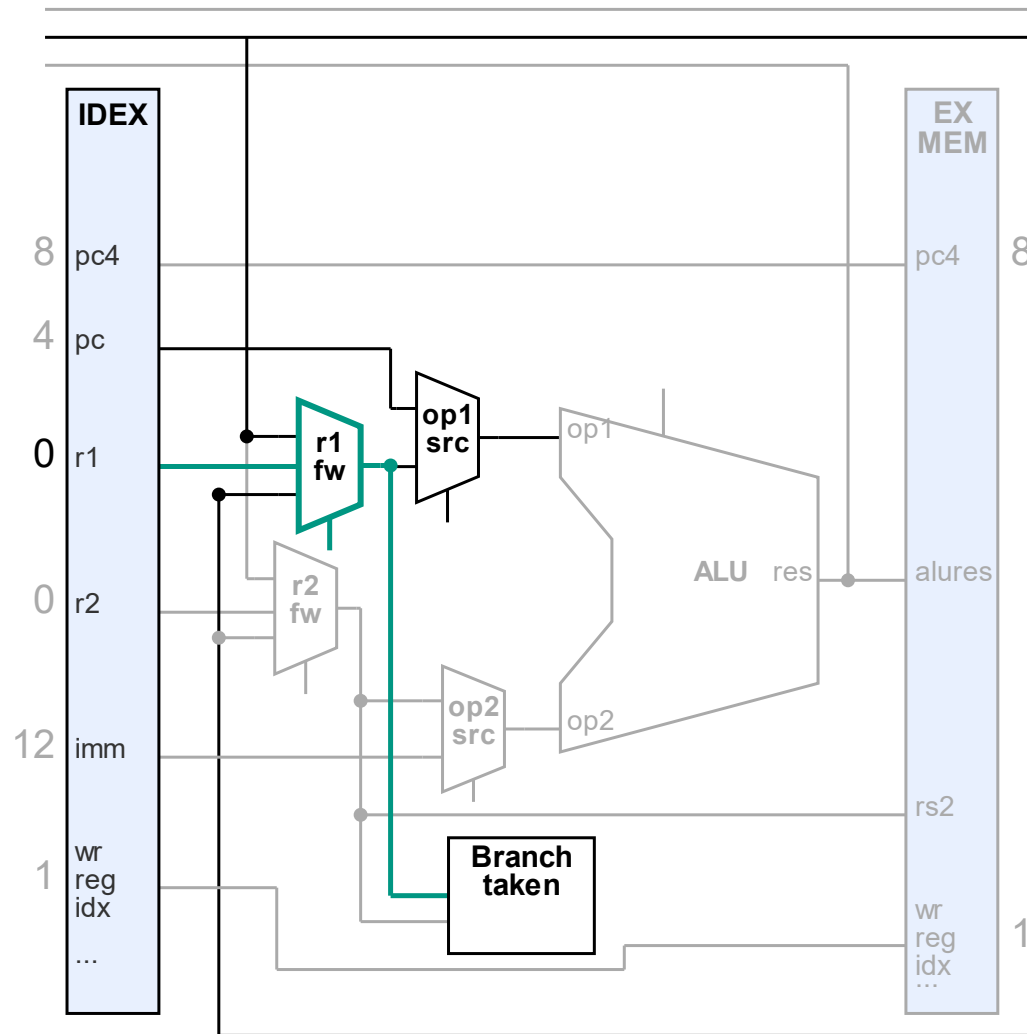


EX Stage im Detail



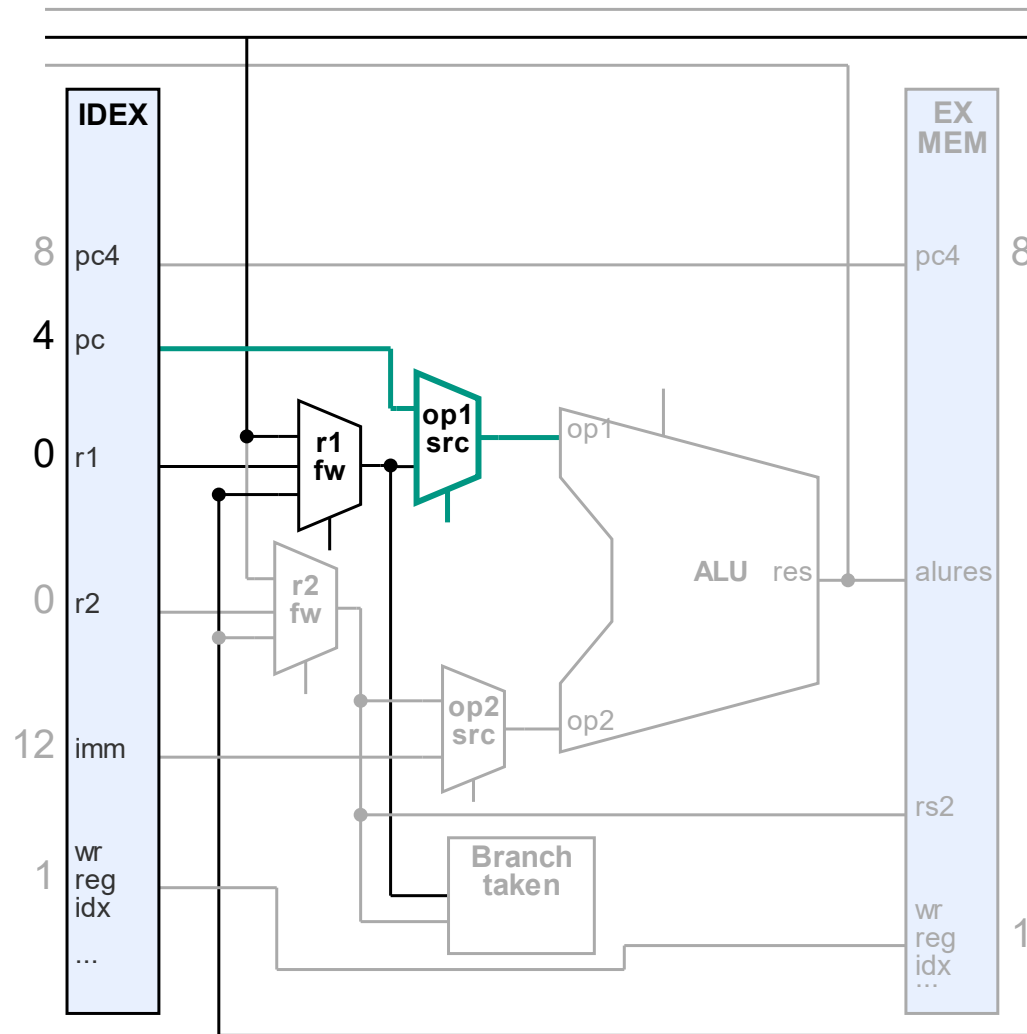
- PC und Zielregister werden wieder durchgereicht

EX Stage im Detail



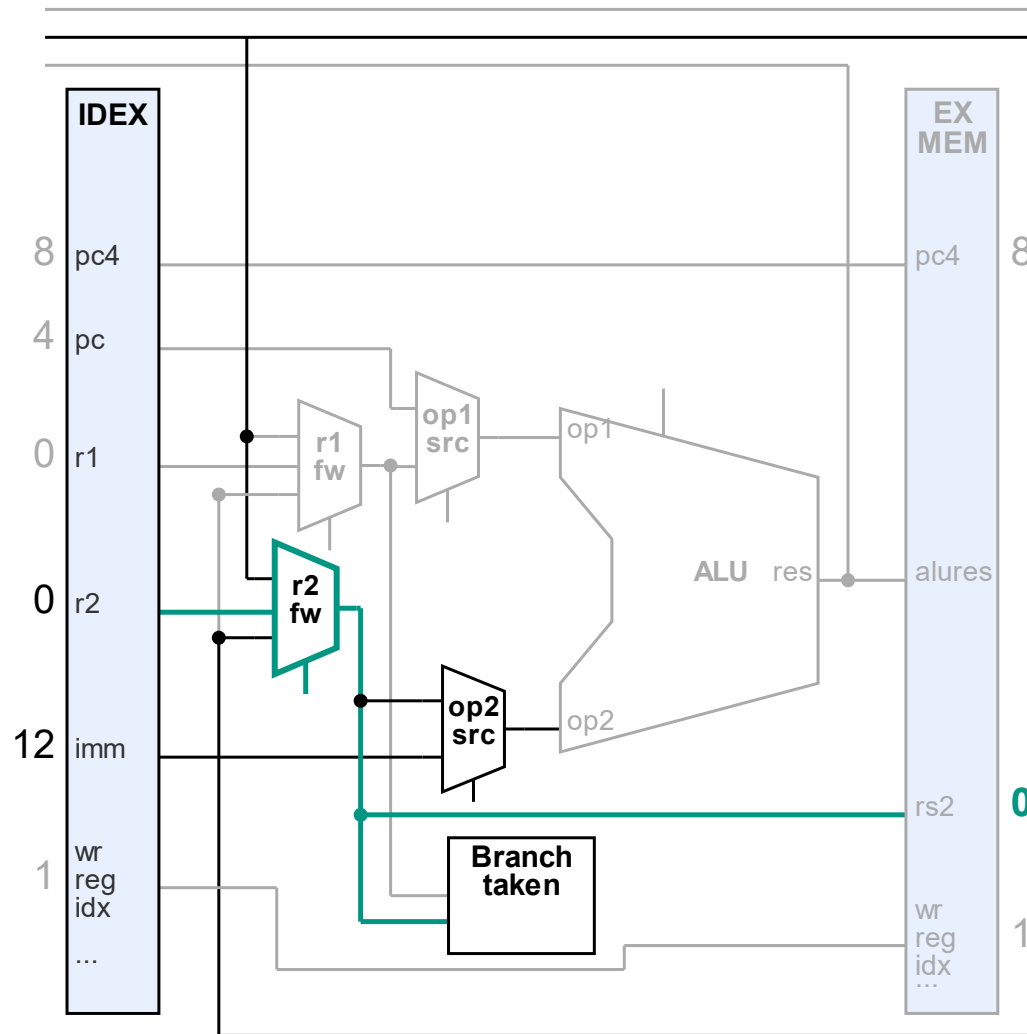
- PC und Zielregister werden wieder durchgereicht
- Nach wie vor kein Forwarding
 - Branch taken ist hier nicht relevant, weil ja ein unbedingter Sprung ist

EX Stage im Detail



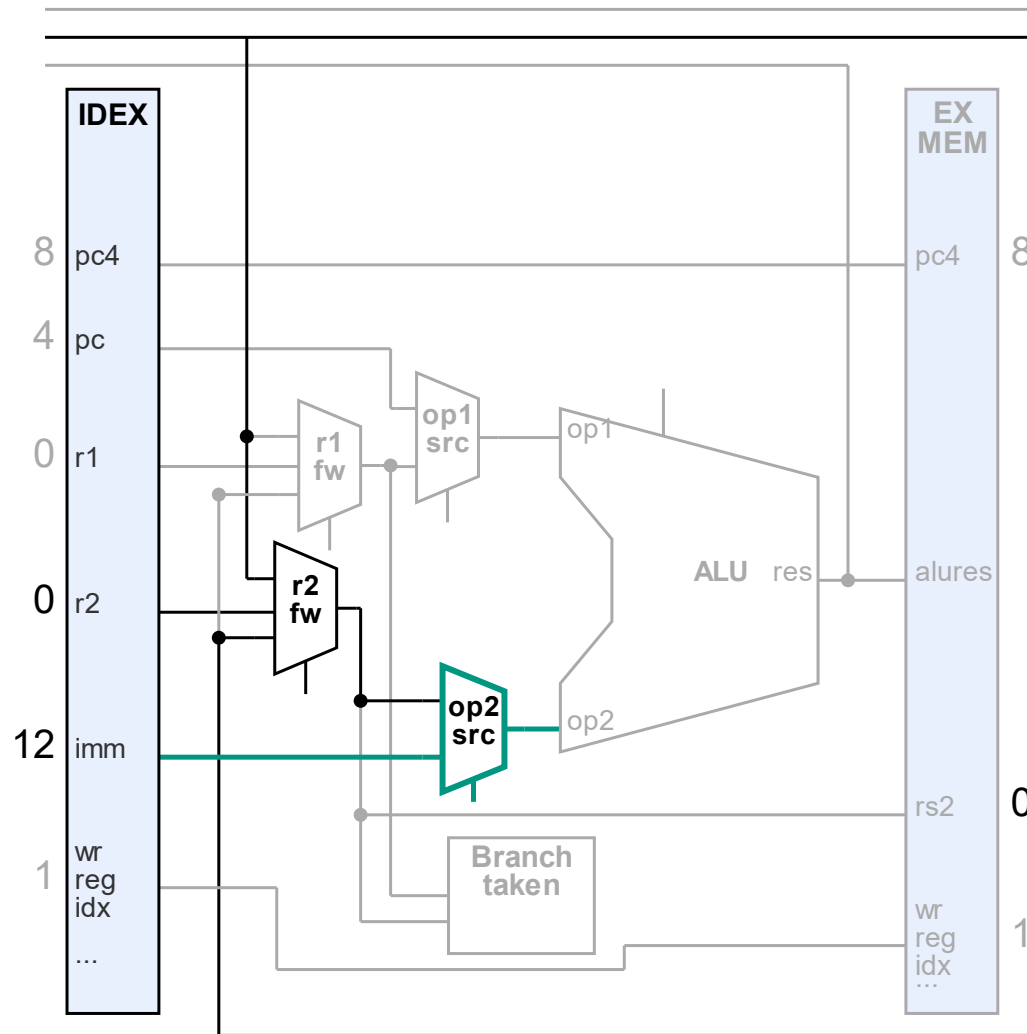
- PC und Zielregister werden wieder durchgereicht
- Nach wie vor kein Forwarding
- Ziel ist den Program Counter um einen Offset zu verschieben
 - Operand 1 muss der PC sein

EX Stage im Detail



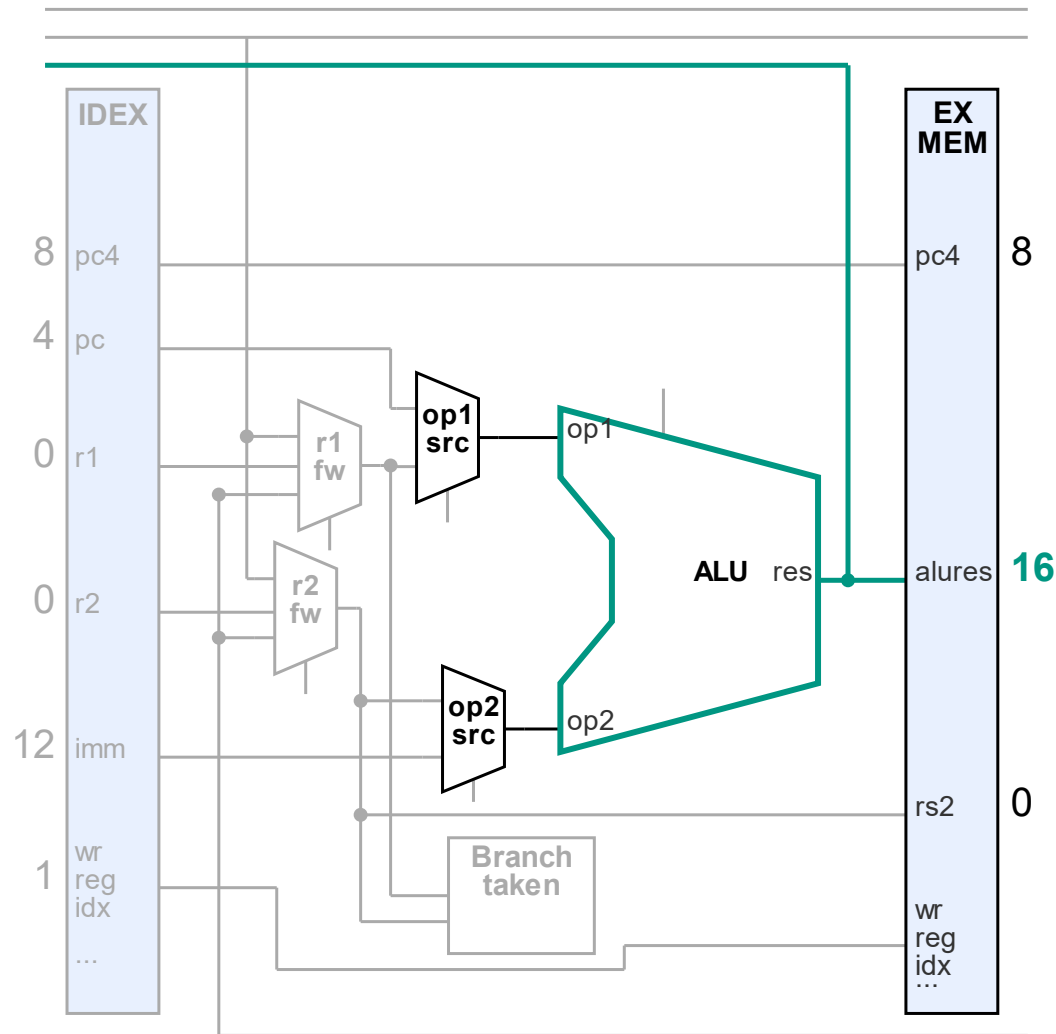
- PC und Zielregister werden wieder durchgereicht
- Nach wie vor kein Forwarding
- Ziel ist den Program Counter um einen Offset zu verschieben
- Auch für R2 kein Forwarding

EX Stage im Detail



- PC und Zielregister werden wieder durchgereicht
- Nach wie vor kein Forwarding
- Ziel ist den Program Counter um einen Offset zu verschieben
- Auch für R2 kein Forwarding
- Operand2 soll den Offset darstellen und ist in imm gespeichert

EX Stage im Detail



- PC und Zielregister werden wieder durchgereicht
- Nach wie vor kein Forwarding
- Ziel ist den Program Counter um einen Offset zu verschieben
- Auch für R2 kein Forwarding
- Operand2 soll den Offset darstellen und ist in imm gespeichert
- Die ALU berechnet die neue Speicheradresse

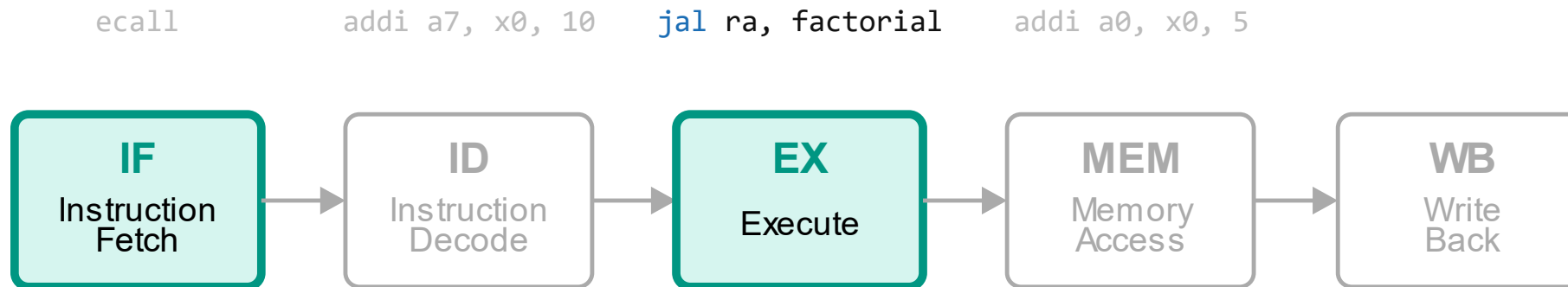
Der eigentliche Sprung

- Am Ende des Taktzyklus ist der neue Wert bereit, aber die IF Stage hat bereits den falschen PC Wert genommen
- Erst im darauffolgenden Taktzyklus kann der Sprung ausgeführt werden

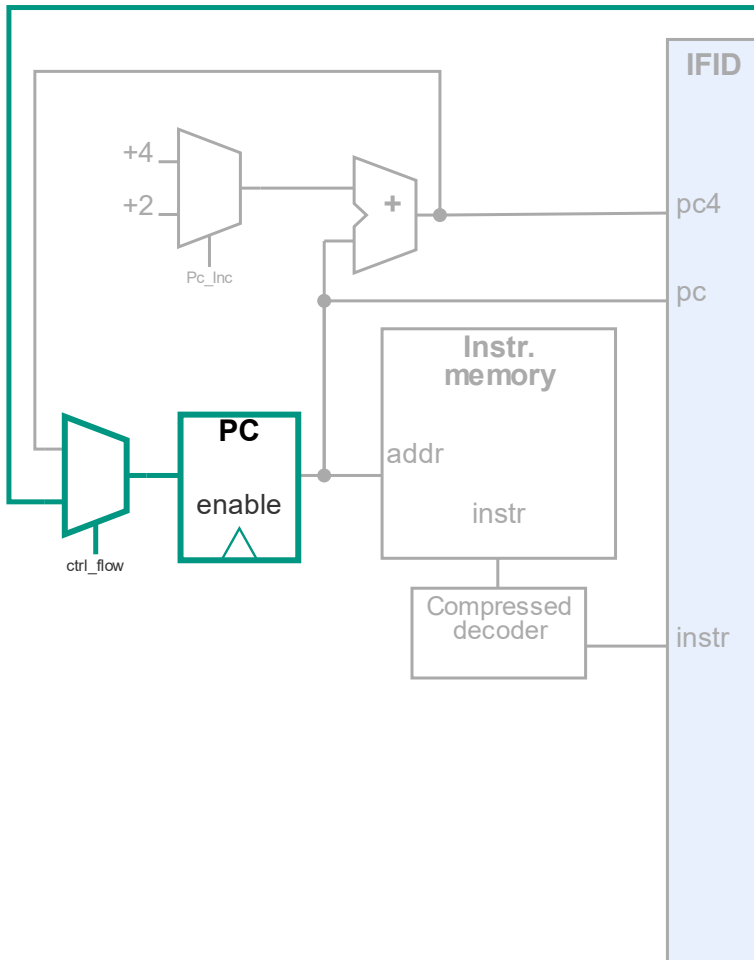
```
main:  
    addi    a0, x0, 5  
    jal     ra, factorial
```

```
    addi    a7, x0, 10  
    ecall
```

```
factorial:  
    addi    a1, x0, 1  
    addi    a2, x0, 1
```



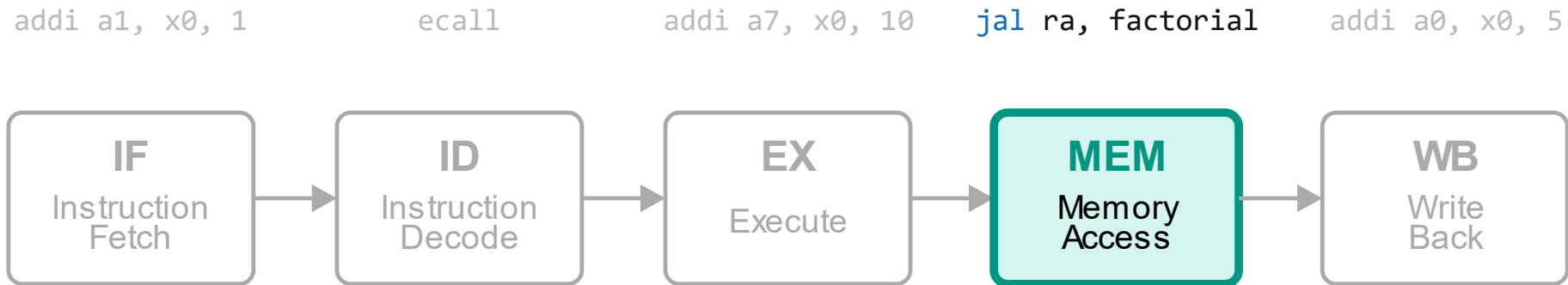
Der eigentliche Sprung



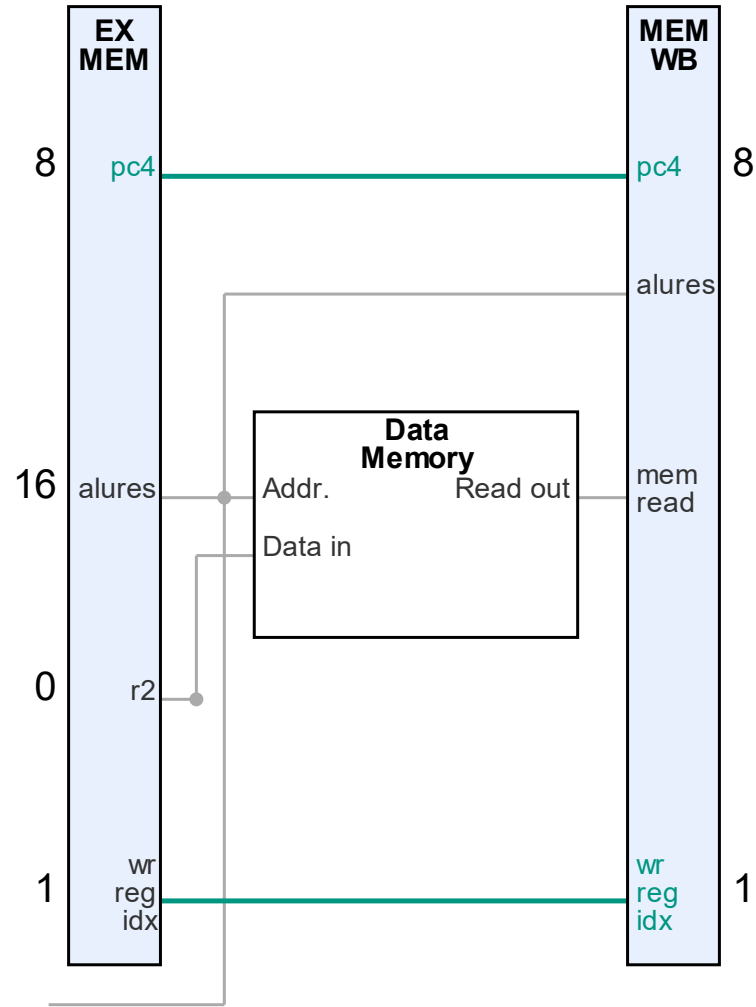
- Neuer Wert liegt bereit für Program Counter zu Beginn des nächsten Taktzyklus

JAL MEM

- Externe Speicherzugriffe
- Wie zuvor: nicht zutreffend

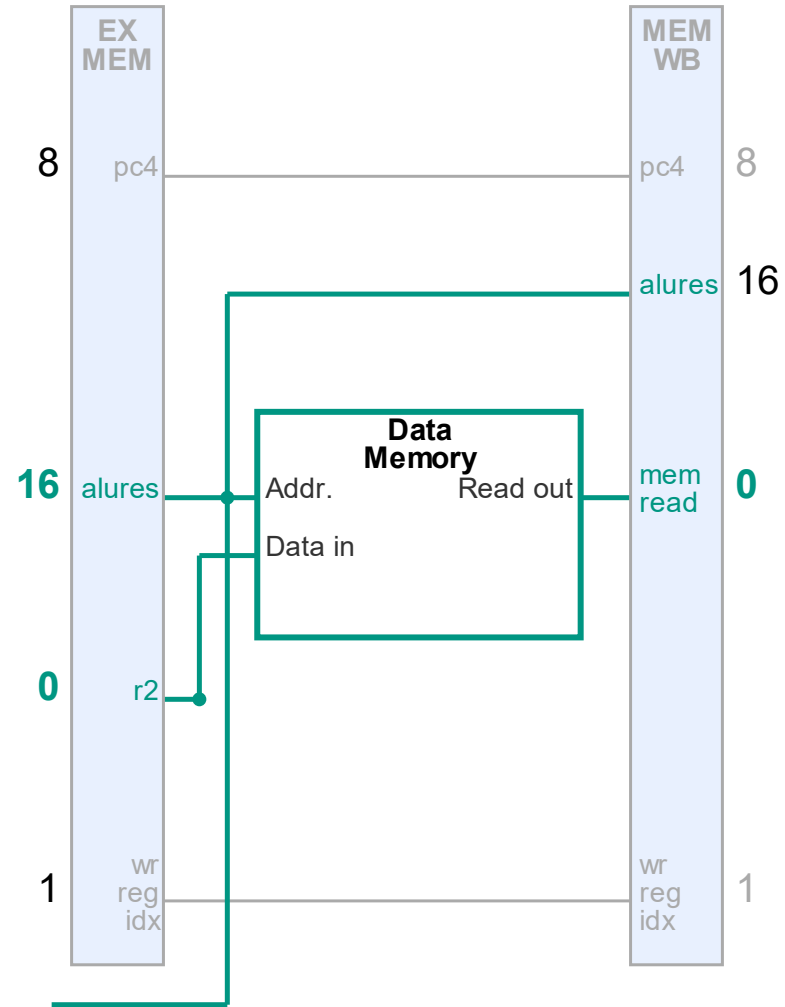


MEM Stage im Detail



- PC4 und Zielregister können durchgereicht werden

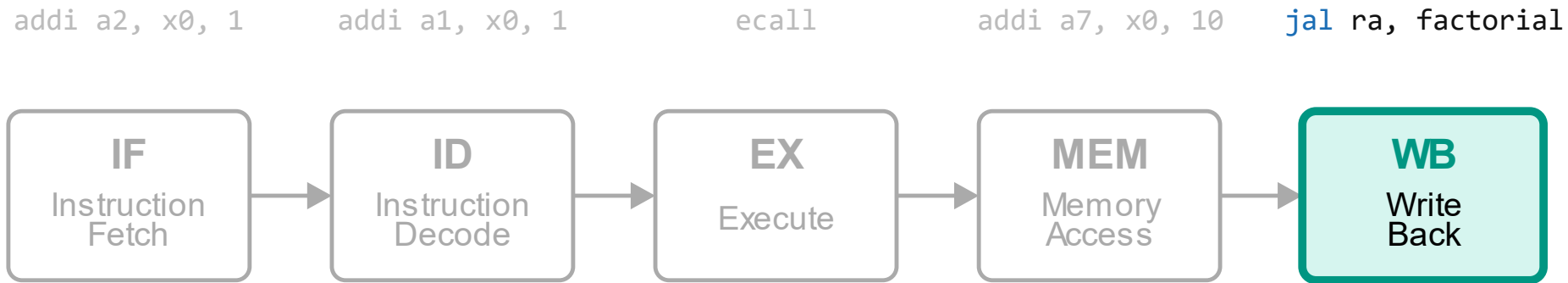
MEM Stage im Detail



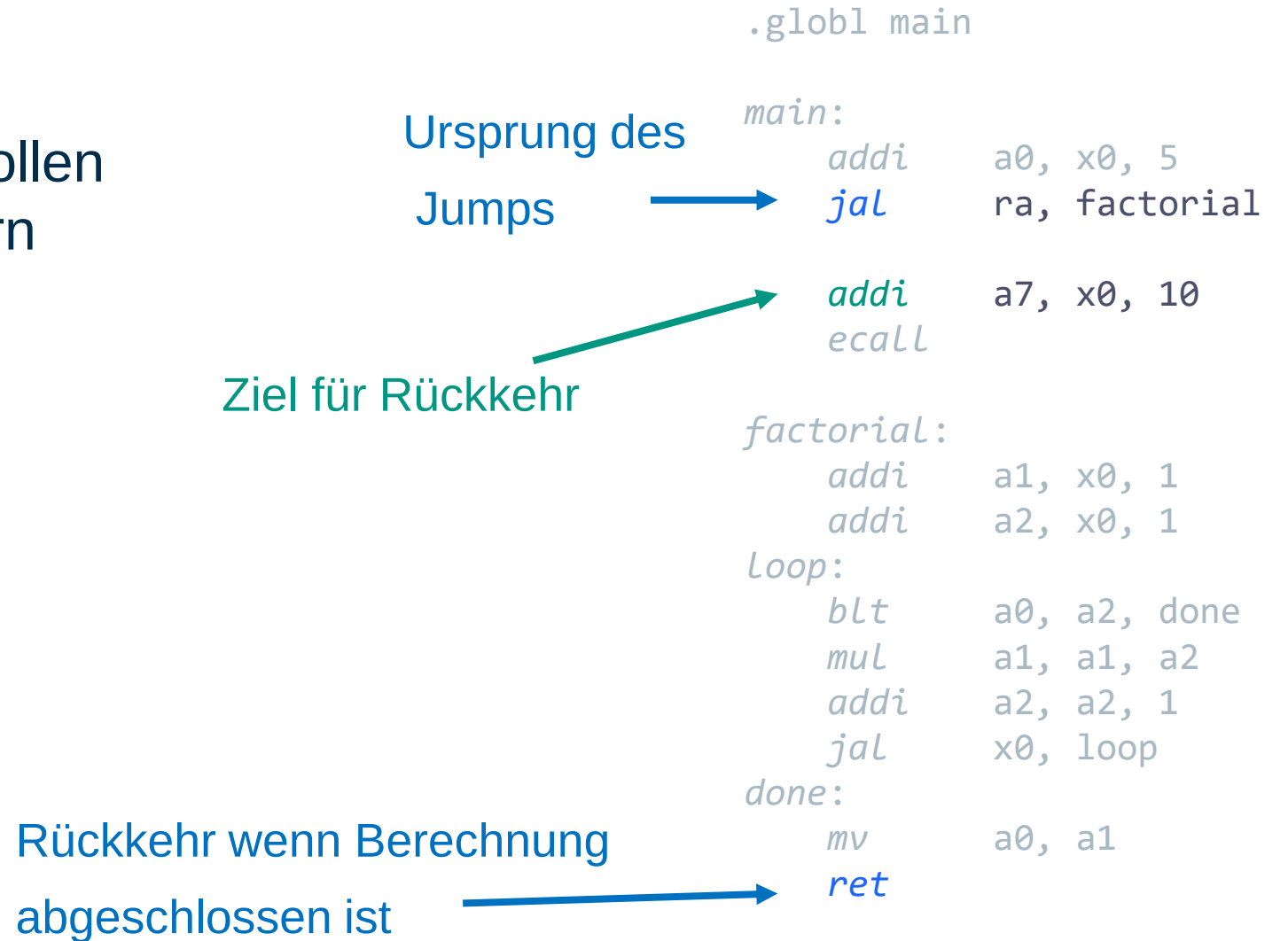
- PC4 und Zielregister können durchgereicht werden
- Keine Speicherzugriffe nötig

JAL WB

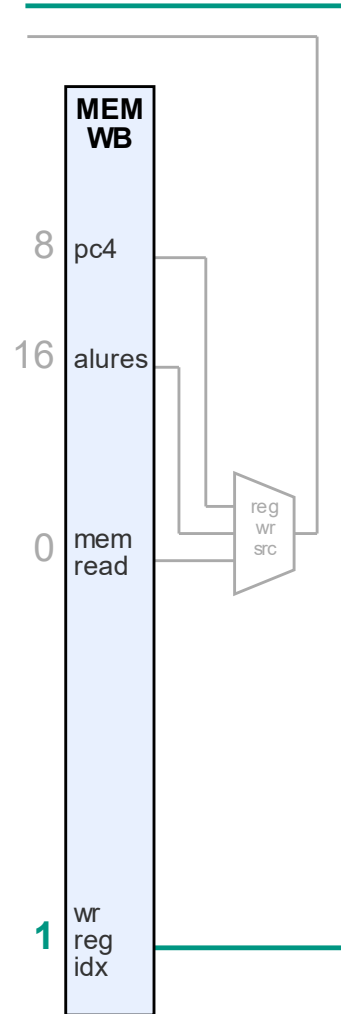
- Writeback ins Zielregister
- Hier: Writeback der return address



- Zu einem späteren Zeitpunkt wollen wir zurückspringen an die Return address

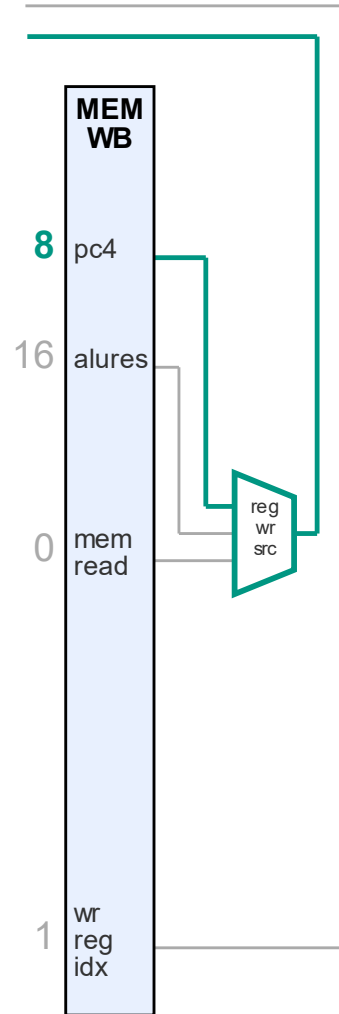


WB Stage im Detail



- Ziel des Writebacks ist die Return address (per Konvention x1)

WB Stage im Detail



- Ziel des Writebacks ist die Return address (per Konvention x1)
- Als Wert wird pc4 geschrieben (die Instruktion unmittelbar nach der *jal* Instruktion)

Übersicht – Vorlesung Nr. 05

- Ripes Case Study
- Ausblick:
 - Moderne CPUs und heterogene Systeme

RISC-V-Kerne

- Proprietäre RISC-V-Kerne

- Codasip

- Hauptsitz in München
 - News 04/2026: Strategische Neuausrichtung, fokussiert sich künftig stärker auf sicherheitsorientierte Architekturen

- SiFive

- News 04/26: 400-Millionen-Dollar-Finanzierung für den Ausbau einer High-Performance-RISC-V-Roadmap für Rechenzentren
 - Offene CPU-IP, welche eng mit GPU-Interconnects verzahnt sein soll (NVIDIA)

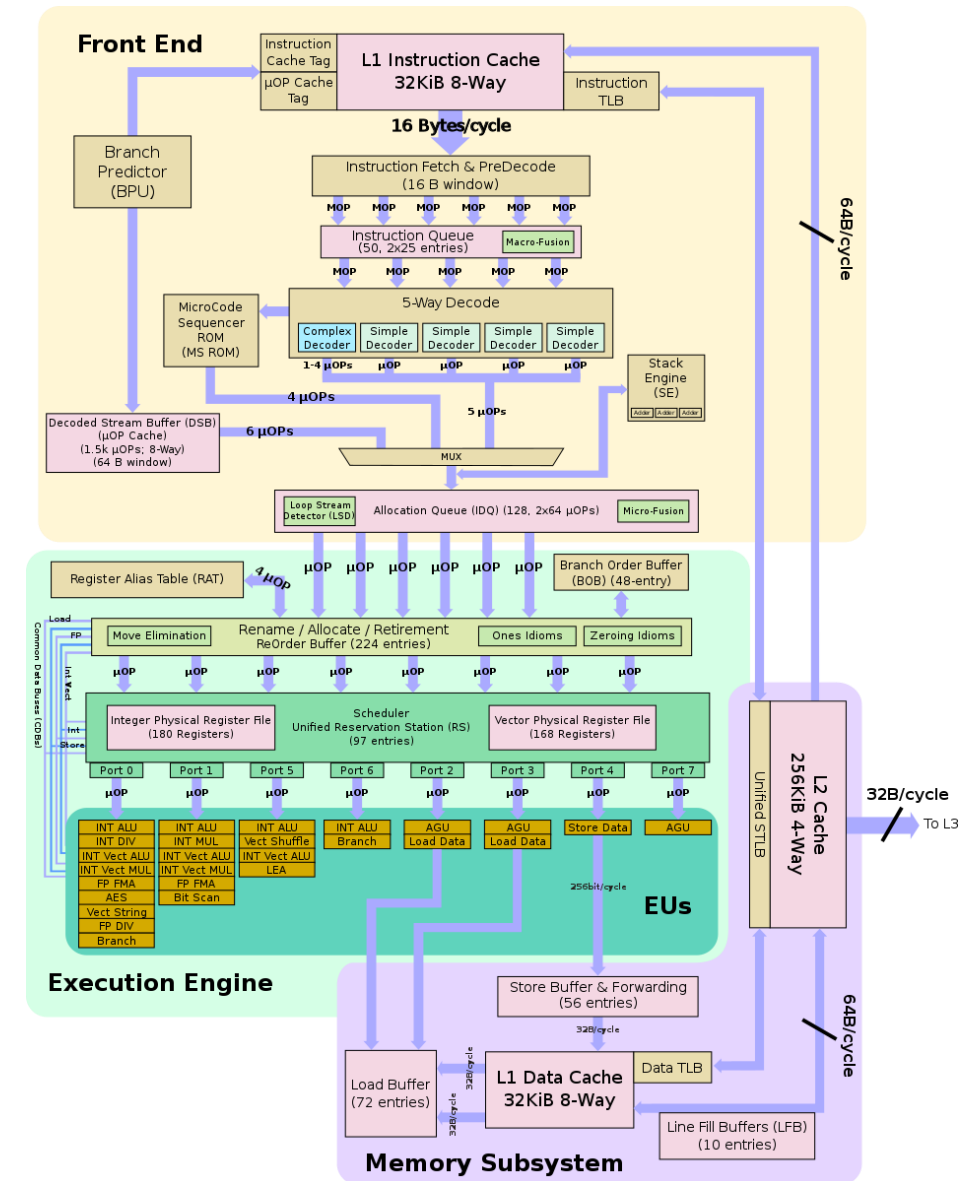
- Quelloffene RISC-V-Kerne

- University of California (Berkeley): Rocket Chip (Chisel)
 - ETH Zürich / Universität Bologna: Parallel Ultra Low Power (PULP) Platform (SystemVerilog)

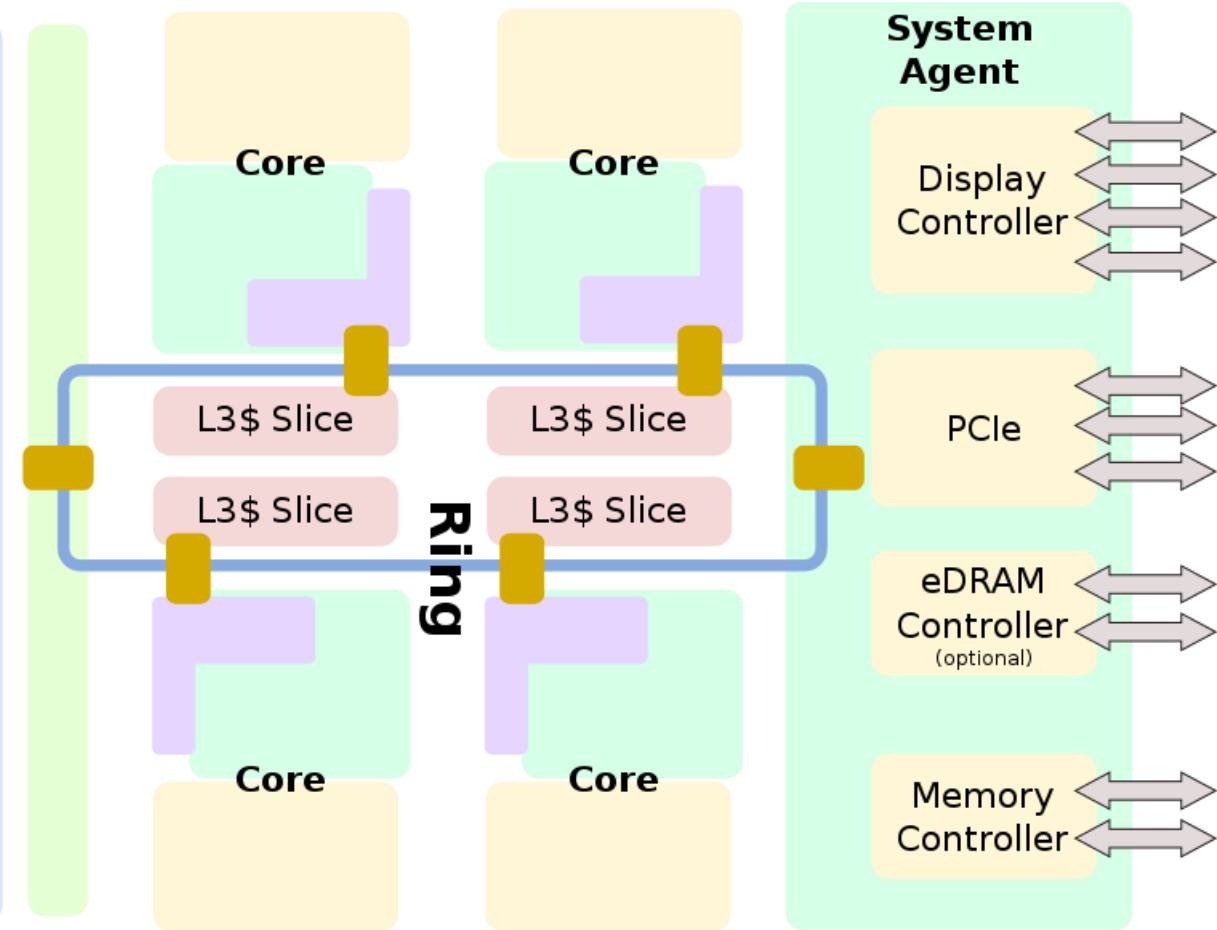
Beispiel Intel Kaby Lake

- Architektur eines einzelnen Cores
- 14-19 Pipeline Stufen
- Superskalar
- Spekulative Ausführung
- SIMD Instructions
- Caches
 - L1I 8-way, 32KiB
 - L1D 8-way, 32KiB
 - L2 4-way, 256KiB
 - L3 16-way, 2 MiB/core

ISA	x86-16, x86-32, x86-64
Extensions	MOVBE, MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, POPCNT, AVX, AVX2, AES, PCLMUL, FSGSBASE, RDRND, FMA3, F16C, BMI, BMI2, VT-x, VT-d, TXT, TSX, RDSEED, ADCX, PREFETCHW, CLFLUSHOPT, XSAVE, SGX, MPX



Gen9.5

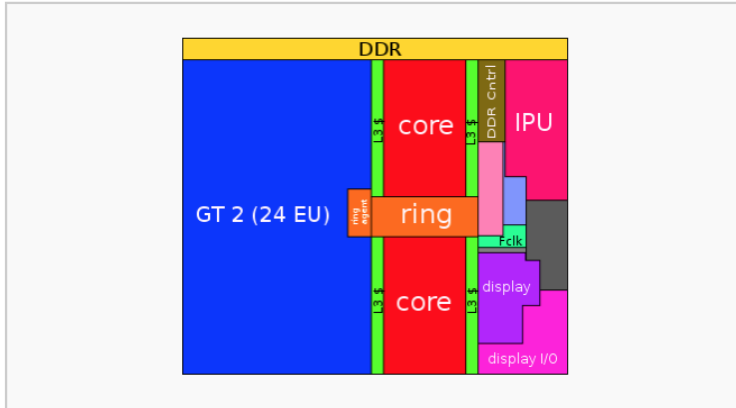


Beispiel Intel Kaby Lake

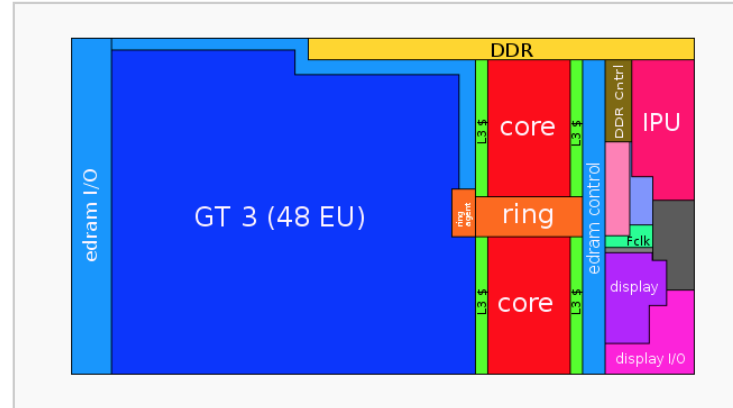
- Konfiguration mit 4 Kernen und GPU

Beispiel Intel Kaby Lake Konfigurierbarkeit

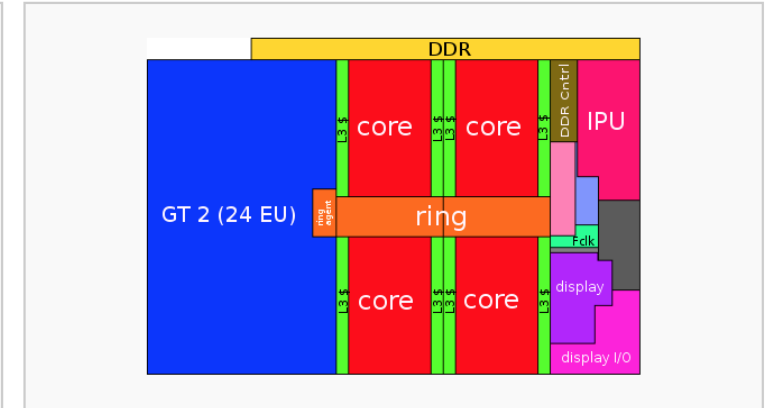
Physical Layout Breakdown



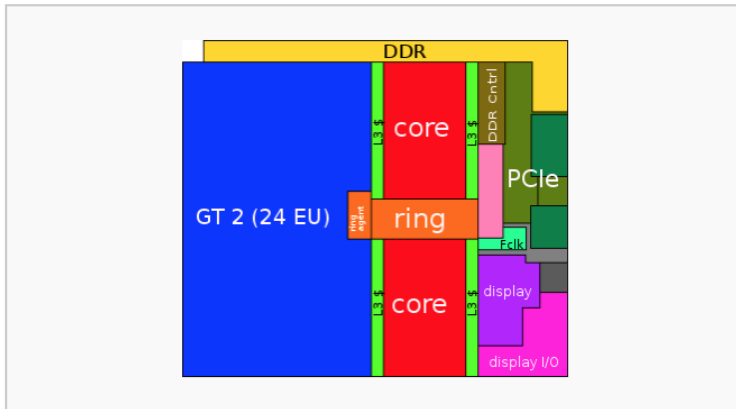
Dual-core die, GT2 GPU, Low Power



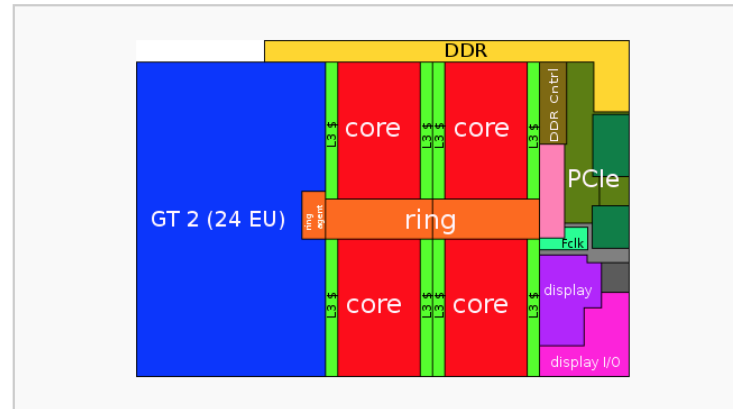
Dual-core die, GT3 GPU, Low Power



Quad-core die, GT2 GPU, Low Power

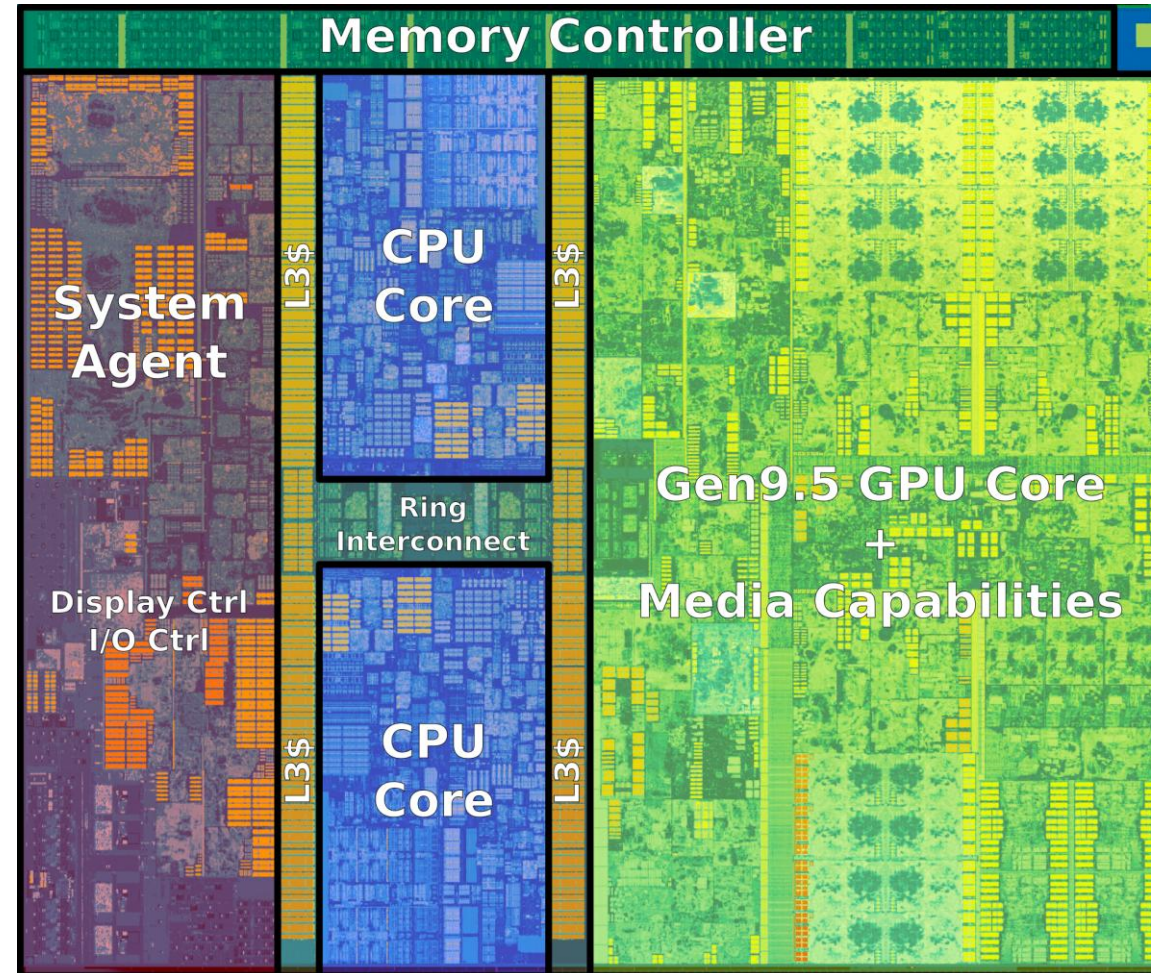


Dual-core die, GT2 GPU, High Power

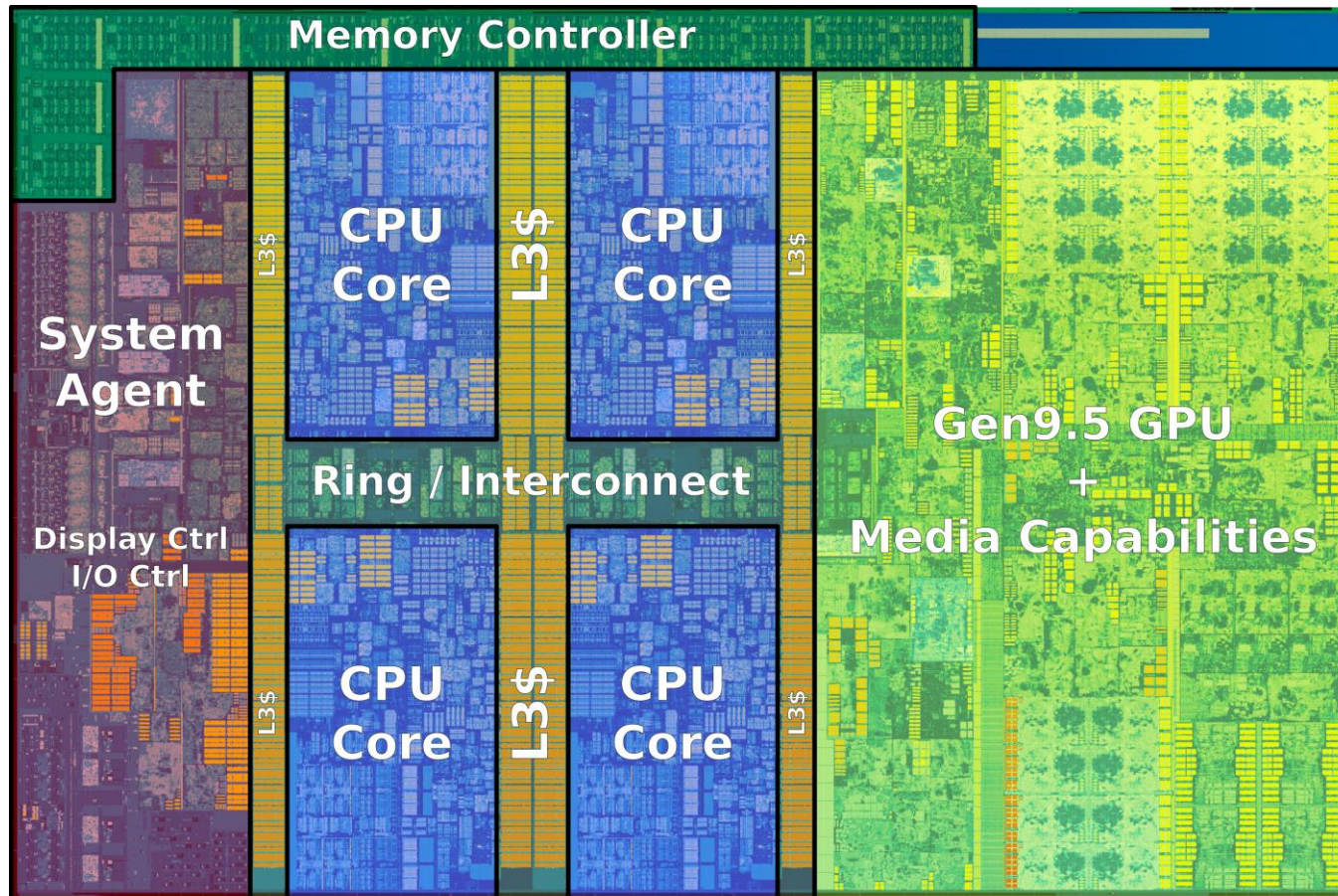


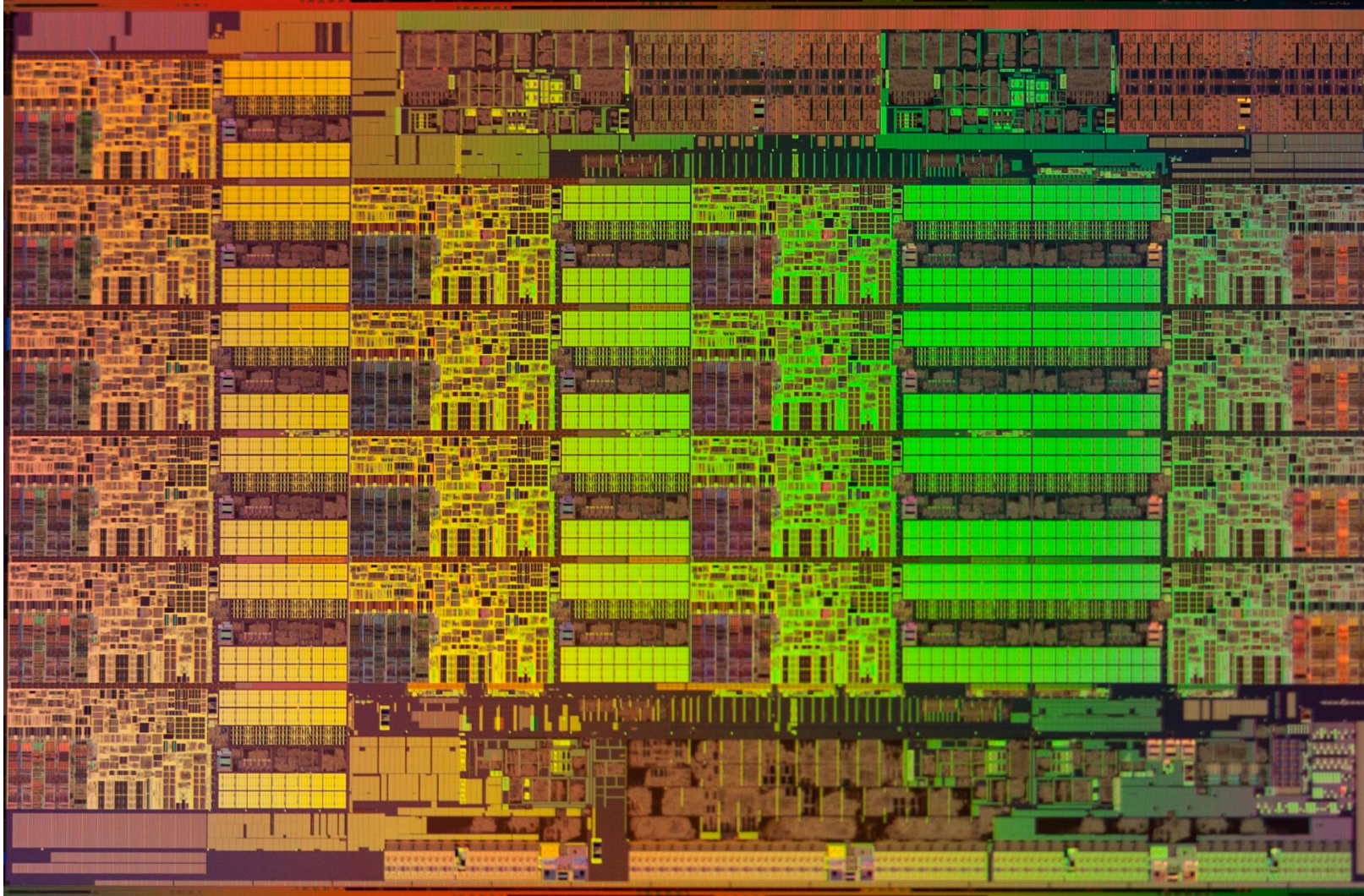
Quad-core die, GT2 GPU, High Power

Beispiel Intel Kaby Lake - Dual Core



Beispiel Intel Kaby Lake - Quad Core

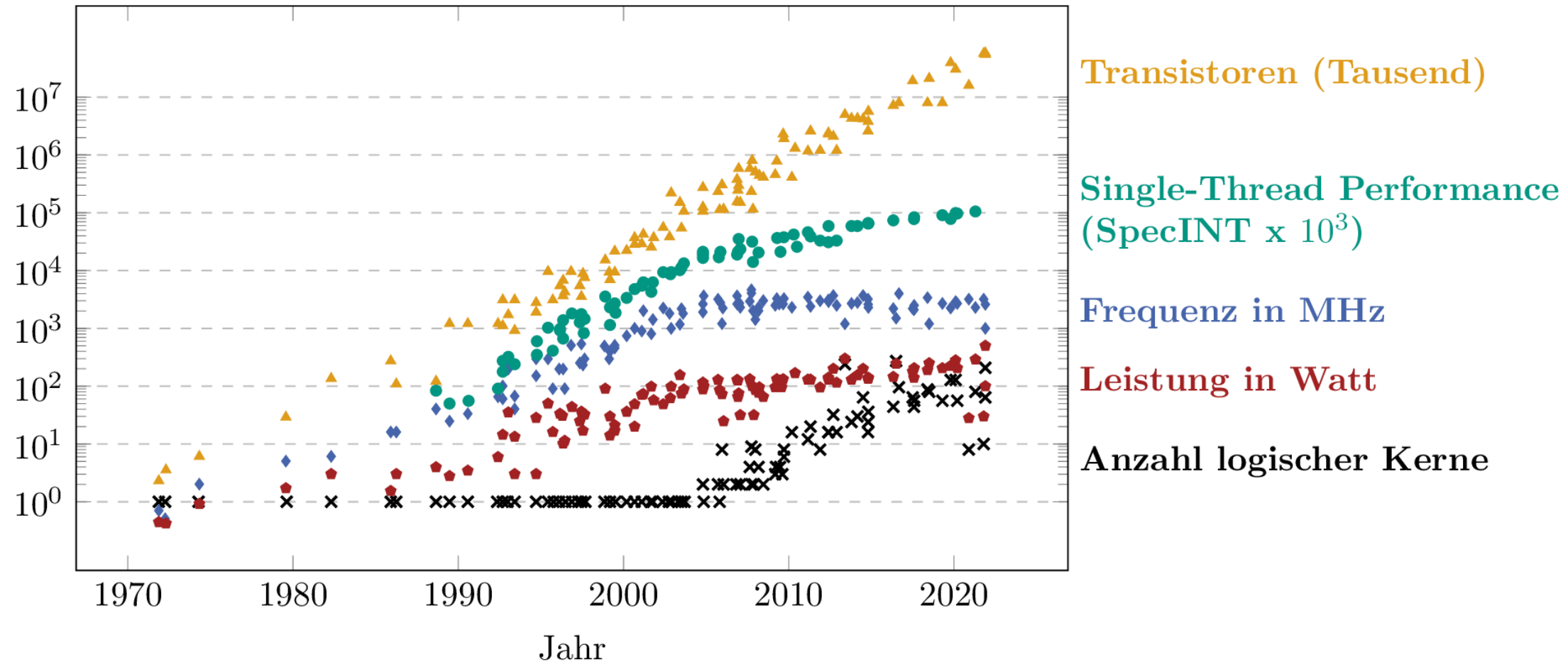




Beispiel: Intel Haswell Octadeca Core

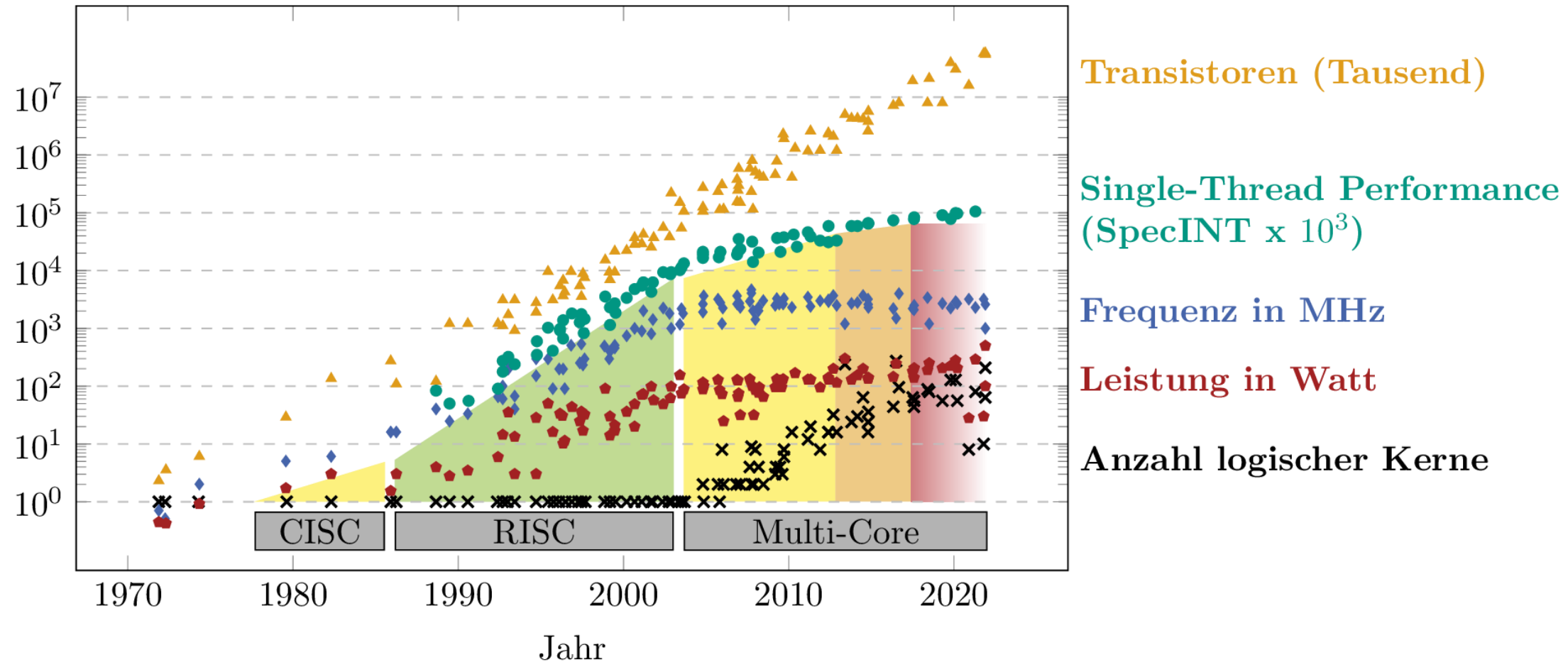
- 18 cores, 5,690,000,000 transistors, 622 mm² die size, 22 nm

50 Jahre Entwicklung von Mikroprozessoren



Source: <https://github.com/karlrupp/microprocessor-trend-data/>

50 Jahre Entwicklung von Mikroprozessoren



Source: <https://github.com/karlrupp/microprocessor-trend-data/>

Free Lunch is over

- **Moore's Law** (1965)
 - Anzahl der Transistoren verdoppelt sich alle 18 Monate
- **Dennard Scaling** (1974)
 - Konstante Leistungsdichte bei Reduzierung der Transistorgröße
 - Leistungsverbrauch im Verhältnis zur Fläche unverändert
- **Amdahl's Law** (1967)
 - Geschwindigkeitszuwachs bei Parallelisierung vor allem durch den sequentiellen Anteil des Problems beschränkt

Ansätze nach Ende des Dennard Scalings

- Neue Technologien
 - Quantencomputer, Neuromorphic und Nanotechnologie
 - 3D Technologie
- Mehrkernarchitekturen
- General Purpose Prozessoren mit dedizierten Beschleunigern
 - Graphic Processing Units (GPU)
 - Heterogene Architekturen
- Applikationsspezifische Prozessoren
 - Dynamisch zur Laufzeit
- Optimierte Speicherarchitekturen
 - Near/In Memory Computing
 - Datengetrieben

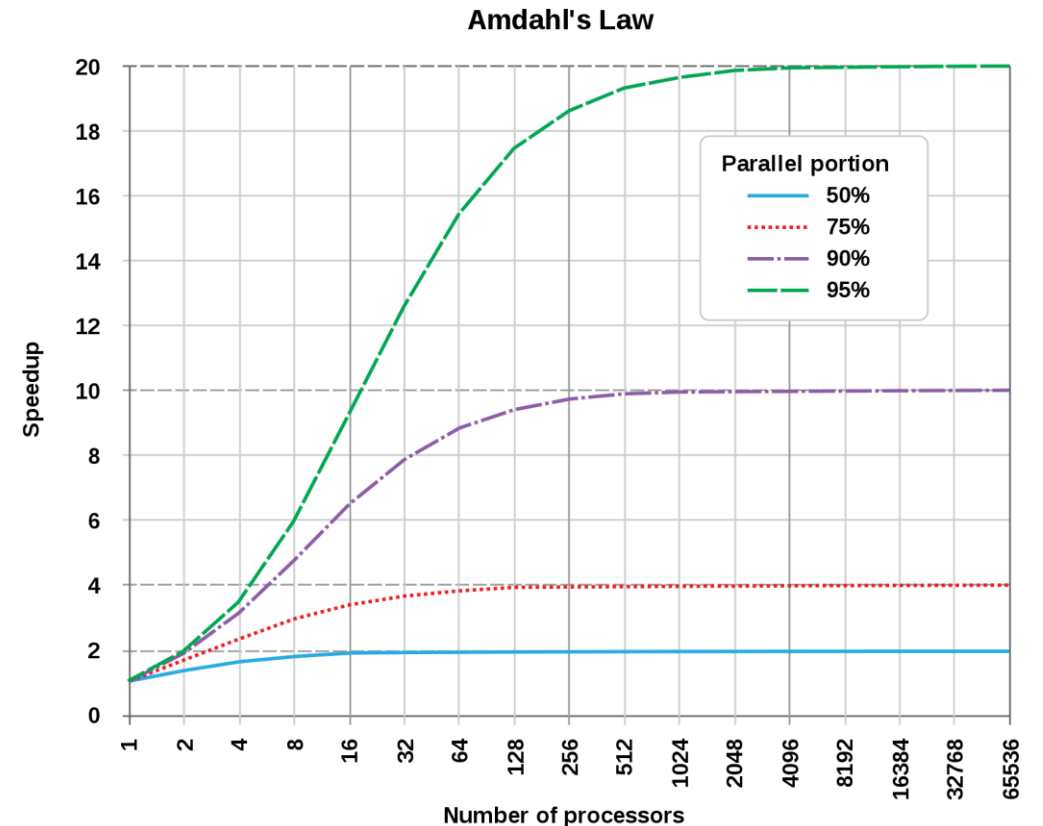
Amdahl's Law

- Modell für die Beschleunigung von Programmen durch parallele Ausführung

- Der Geschwindigkeitszuwachs wird hauptsächlich durch den sequenziellen Anteil begrenzt

$$\eta s = \frac{T}{s + \frac{p}{n}} \leq \frac{T}{T - p}$$

- ηs Speedup-Faktor
- $T = s + p$ Gesamtlaufzeit
- s Laufzeit eines seriellen Programmabschnitts
- p Laufzeit eines parallelen Programmabschnitts



Weiterführende Vorlesungen im Master am ITIV

- Hardware Synthesis and Optimization
 - Formalen und methodischen Grundlagen für den automatisierten Entwurf optimierter elektronischer Systeme
- Hardware/Software Co-Design
 - Gleichzeitige und verzahnte Entwicklung der Hardware- und Softwarekomponenten eines Systems
- Hardware Modeling and Simulation
 - Einblicke in die verschiedenen Abstraktionsebenen von Hardwarebeschreibungssprachen
- Communication Systems and Protocols
 - Physikalische und technische Grundlagen für das Design und Konzeption von Kommunikationssystemen

